

Diving Deep Into UC: Uncovering and Resolving Issues in Universal Composability

Céline Chevalier^{1,2} and Éric Sageloli^{2,3,4}

¹ CRED, Paris-Panthéon-Assas University, Paris, France

² DIENS, École normale supérieure, CNRS, PSL University, Inria, Paris, France

³ Thales, Gennevilliers, France

⁴ eric.sageloli@protonmail.com

Abstract. Introduced by Canetti in 2001, Universal Composability (UC) is a widely adopted security model that enables the specification and proof of security for a broad range of protocols, offering strong security guarantees. At its core lies the universal composition theorem (UC theorem), which ensures that protocols proven secure within the framework remain secure even when deployed in real-world environments with multiple instances of them.

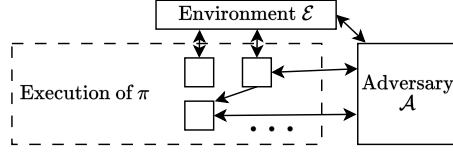
In this work, we present two key contributions. First, we identify several problems with the UC framework, in particular the UC Theorem. They include counterexamples, limitations that make it unusable for important classes of protocols, and weaknesses in its proof. These problems reveal flaws in nearly all the fundamental concepts of UC.

Secondly, we update the main concepts of UC to address these problems. Although these revisions are nontrivial, our updated definitions are intended to stay as closely aligned with the original model as possible, while providing greater simplicity overall. To ensure the validity of these updates, we present a proof of the updated UC theorem, which is more detailed and modular than the original.

1 Introduction

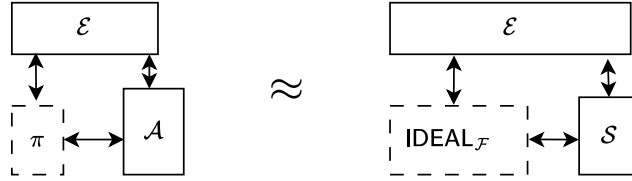
Introduced in 2001 by Canetti in [8], Universal Composability (UC) is a security framework renowned for its ability to define and prove the security of a wide range of protocols, as well as for the strong security guarantees it provides. A protocol proven secure within this framework remains secure even when deployed in real-world environments, including in multiple concurrent instances. To illustrate the variety of protocols recently studied under UC, we can cite PAKEs [22,19,17,4], non-interactive zero-knowledge arguments [16], oblivious transfer [5], the WhatsApp backup protocol [18], the Signal protocol [14] and the Bitcoin protocol [3].

In UC, it is considered that if a protocol contains a vulnerability, this vulnerability should have an impact on some environment of execution, attacked by some adversary. Consequently, when studying a protocol π , we consider its execution alongside two machines: one modeling an adversary and the other modeling the execution environment.



To define the security of a protocol, such as a PAKE [22,19,17,4], or an oblivious transfer [5], we must first define a special machine $\mathcal{F}$, known as an ideal functionality. This ideal functionality is then used to define the protocol $\text{IDEAL}_{\mathcal{F}}$, referred to as the ideal protocol associated with $\mathcal{F}$, which represents a perfectly secure version of that protocol (*e.g.*, a perfectly secure PAKE or oblivious transfer). The references listed at the beginning of this paragraph provide examples of ideal functionalities.

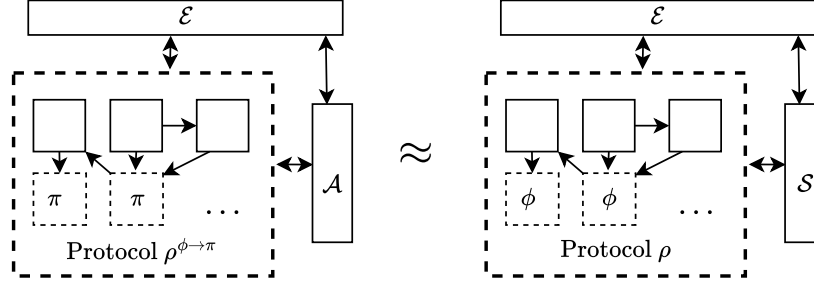
Then, the security of a protocol π is proven by showing that π is “as secure as” $\text{IDEAL}_{\mathcal{F}}$ through a notion called UC-emulation. π is said to UC-emulate $\text{IDEAL}_{\mathcal{F}}$ if, for any adversary $\mathcal{A}$ of π , there is an adversary $\mathcal{S}$ of $\text{IDEAL}_{\mathcal{F}}$ such that no environment $\mathcal{E}$ can efficiently distinguish if it is interacting with π and $\mathcal{A}$ or $\text{IDEAL}_{\mathcal{F}}$ and $\mathcal{S}$.



While the aim of $\mathcal{E}$ is to model any execution environment, such environments are typically not composed of a single machine, nor do they involve only one instance of the protocol. Instead, they can be viewed as protocols ρ with multiple machines, each using multiple instances of the studied protocol.

To justify modeling the environment as a single machine that can only use one instance of the protocol π , UC must ensure that this modeling is sufficient to cover situations involving complex execution environments. This is the goal of the main theorem of the framework, called the universal composition theorem [11, (Theorem 22)] (UC theorem).

The UC Theorem states that for protocols ϕ , π , and ρ , under certain hypotheses, if π UC-emulates ϕ , then a modification of ρ , denoted by $\rho^{\phi \rightarrow \pi}$, where uses of ϕ are replaced by uses of π , will also UC-emulate ρ .



Taking $\phi = \text{IDEAL}_{\mathcal{F}}$, this implies that if π is secure within a simple execution environment – *i.e.*, if π UC-emulates $\text{IDEAL}_{\mathcal{F}}$ – it remains secure when used in complex execution environments.

1.1 Our Contributions

As a first contribution, we identify several problems with the UC theorem [11, (Theorem 22)], classified as follows:

- *Issues (I)*: problems that lead to counterexamples of the UC theorem.
- *Limitations (L)*: difficulties that limit the applicability of the UC theorem.
- *Weaknesses (W)*: gaps in the proof of the UC theorem.

These problems affect all security concepts associated with the UC Theorem, with multiple concerns identified for each. Our focus will be on these concepts within initial and most detailed formulation of the UC framework that was last updated in 2020 [11], which we refer to as “Plain UC”. However, these problems naturally extend to its generalizations, which encompass it. These include UC with joint state [15], UC with global setup [13], and multi-protocol UC [6].

Since the UC Theorem ensures that protocols proven UC-secure maintain their security during composition, our findings reveal that the current UC framework does not guarantee the preservation of security under composition – the very goal of UC.

To re-establish these security guarantees, in a second contribution, we update each security concept within Plain UC. These revised definitions address the identified issues and broaden the applicability of the UC Theorem to protocols that were previously incompatible with the framework. To support these updates, we present an updated UC Theorem with a more modular and detailed proof than the original.

Moreover, although this is outlined for future work, we are confident that these updates provide a solid basis for updating the main generalizations of Plain UC.

Problems in Plain UC and its generalizations. The identified issues, limitations, and weaknesses presented in the main body exhibit problems with all the properties required by the UC Theorem. They are listed and classified into four categories in Table 1, and outlined in the Technical Overview (Section 2).

Table 1. Issues, Limitations and Weaknesses described in the main body.

Notions	Issues	Limitations	Weaknesses
Extended Session and Subroutine Respecting	(I.1) (I.2) (I.3)	(L.1)	(Section 5.1)
UC Operator	(I.4)		(W.1) (Section 5.2)
Subroutine Exposing		(L.2)	(W.2) (Section 5.3)
Compliance			(W.3) (W.4) (Section 5.4)

As a side contribution, Appendix F describes additional problems related to other concepts, such as the time-complexity notion used in UC and the concept of a Balanced Environment (See Technical Overview, Section 2.9).

Updated UC. To address the identified problems, we propose updated definitions for Plain UC, as summarized in Table 2, where even similarly named notions exhibit differences. We refer to the framework with these updated definitions as Updated UC.

Although most of these updates introduce non-trivial changes, they still rely on the same computational model and core ideas. Consequently, while verification on a case-by-case basis is necessary, we believe that protocols proven secure under Plain UC meet the hypotheses of the UC Theorem in Updated UC. Moreover, we claim that these definitions are simpler than those in Plain UC.

Table 2. Correspondence between Plain UC and Updated UC notions.

Plain UC [11]	Updated UC (this work)
Extended Session [11, (Section 3.1.2)]	Extended Session (Definition 3)
–	Panoptic Session (Definition 5)
Subroutine Respecting [11, (Section 5.2)]	Sub-party Respecting (Definition 9)
Universal Composition Operator [11, (Section 6.1)]	Universal Composition Operator (Section 6.7)
Subroutine Exposing [11, (Section 6.1)]	Sid-Exposing (Definition 8)
Compliant [11, (Section 6.1)]	Compliant (Definition 10)
–	Disjunction (Definition 6)
–	Separation (Definition 7)
Balanced Environment [11, (Section 4.2)]	–
–	Replacement Execution (Section 6.2)

The justification for these updates is twofold.

First, we support each definitional change by explaining how it addresses one of the identified problems. These justifications show that, despite some changes being nontrivial, Updated UC is not a distinct model; rather, it represents a necessary update of Plain UC to ensure its correctness.

Second, we provide in Appendix C a comprehensive proof of the UC theorem. While the main ideas remain those of Plain UC, adapted to Updated UC, our version incorporates several improvements to ensure correctness:

- We propose a definition of the UC Operator that allows the UC Theorem to be stated and proven independently of any specific implementation. This adds a layer of modularity, simplifying its analysis.
- The proof is more detailed. For example, in Plain UC, Equality [11, (Equation 7, page 68)], a crucial component, was simply asserted through inspection of the ITMs involved. In contrast, our proof presents a detailed analysis of the equivalent equality (Lemma 2).

Finally, to assist those seeking to prove the security of a protocol under Updated UC, without having to dig into the proof of the updated UC Theorem, a short guide is provided in Appendix B.

1.2 Related Work

Aside from the current version of Plain UC, widely used in this work, we can classify previously known publications into three categories:

- Generalizations of Plain UC, such as UC with joint state [15], UC with global setup [13], and multi-protocol UC [6]. While the problems we have identified in Plain UC can be directly adapted to these generalizations, updating their UC theorems requires additional work. Although further research is needed, we are confident that the updates to Plain UC provide a solid foundation for updating these generalizations. This confidence is partly justified by [2], which demonstrates that the UC theorem from Plain UC can be used to prove the UC theorem for UC with global setup.
- Alternative frameworks to Plain UC that rely on different computational models, such as iUC [7] (based on the IITM model [23,25]), CC [24], and GNUC [20]. Both [20] and [23] identify significant issues in older versions of Plain UC (specifically the 2005 and 2013 revisions), leading to counterexamples of the UC theorem. These issues are addressed by the current version of Plain UC that we analyze, and they arise from different ideas than the problems we present in this work. Appendix J of [25] highlights minor issues distinct from those we identified, which can be resolved by slightly refining Plain UC’s protocol-execution definition; our description of Plain UC incorporates those refinements. Given that the problems of Plain UC identified in our work are new, it may be worthwhile to investigate if any of them can be adapted to these alternative frameworks. Finally, we emphasize that our aim is not to create a new framework, but to ensure the correctness of Plain UC by updating its definitions.

- Simplification of Plain UC, such as the one described in [12], which presents a simplified variant within the context of standard multiparty computation. This variant deliberately restricts the expressiveness of Plain UC, for instance, by removing the possibility for protocols to have sub-parties [12, (Section 3.2)]. Since its security is based on embedding this variant into Plain UC, we believe some of the issues we identified may also be relevant to this simplified version. Moreover, our updates to Plain UC pursue different goals: they aim to preserve as much expressiveness as possible.
- Comparison of UC with other frameworks, such as [25] that compare the UC and IITM frameworks. It would be interesting to explore how the updated definitions of UC impact such comparisons.

Acknowledgements

This work was supported in part by the French ANR project SecNISQ (ANR-21-CE47-0014) and through the France 2030 program under the RESQUE project.

We express our deepest appreciation to Thomas Ricosset, whose insightful comments improved the presentation of the paper throughout its creation; Hugo Beguinet, whose feedback enhanced the flow of the main body; Erkan Tairi, for his constructive advice that refined the introduction and technical details; and Paula Celles, for her invaluable assistance in elevating the overall writing quality.

Outline of the paper

Section 2 gives a technical overview of the article.

The following sections elaborate on our contributions. First, Section 3 provides preliminary notations, followed by a description of Plain UC in Section 4. Then, Section 5 identifies various issues, limitations, and weaknesses within Plain UC. Section 6 introduces Updated UC, and Section 7 presents the UC Theorem associated with Updated UC.

Additionally, the appendices include further contributions. Appendix E defines a UC Operator that addresses issues associated with Plain UC’s UC Operator and aligns with our general definition of a UC Operator (Definition 13). Furthermore, Appendix F outlines additional issues, limitations, and weaknesses that, while not essential for understanding the main content, offer valuable insights. These include discussions of the notion of time complexity within Plain UC, the concept of a balanced environment, and the possibility of specifying the receiver of an external write via a predicate rather than by its extended identity.

2 Technical Overview

In this technical overview, we provide a detailed account of the main results of our article. Due to the technical nature of our contributions, some familiarity

with Plain UC is required. To facilitate understanding, a comprehensive overview of Plain UC is provided in Section 4. Furthermore, from this point onward and throughout the rest of the article, we focus exclusively on Plain UC and the update proposed in this paper.

2.1 A Flaw in the Extended Session Definition Making the UC Theorem Unusable for Practical Protocols

In Plain UC, the extended session of a protocol π and session identifier s for a fixed execution is a set denoted by $\text{Ext}(\pi, s)$.⁵ It is constructed interactively during the execution by observing the ITIs that are invoked. All ITIs with code π and session identifier s , known as the main parties, are included and form the subset $\text{MainParty}(\pi, s) \subset \text{Ext}(\pi, s)$. Additionally, ITIs that serve to perform computations for the main parties or other members of the extended session are also added. These are referred to as sub-parties and form the subset $\text{ExtSub}(\pi, s) \subset \text{Ext}(\pi, s)$.

According to the precise definition of an extended session, an ITI N cannot be part of $\text{ExtSub}(\pi, s)$ if it has not been invoked by a member of $\text{Ext}(\pi, s)$ – see the discussion on page 21 in the overview of Plain UC.

In our work, we establish that this condition is excessively restrictive. Consider for example an ITI $(\pi, (s, 0))$, which exclusively sends inputs to an ITI N , requesting computations. The only outputs N provides are to $(\pi, (s, 0))$, delivering the results of these computations. Moreover, N rejects any messages from sources other than $(\pi, (s, 0))$, ignoring them as though they were never received. This scenario is illustrated in Figure 1.

In any execution, when $(\pi, (s, 0))$ sends a message to N , we assert that N should belong to the set $\text{ExtSub}(\pi, s)$; it could even serve as a textbook example of a sub-party. However, if any malicious ITI U invokes N before it receives its first message from $(\pi, (s, 0))$, the strict definition of the extended session implies that N will not be included in $\text{ExtSub}(\pi, s)$.

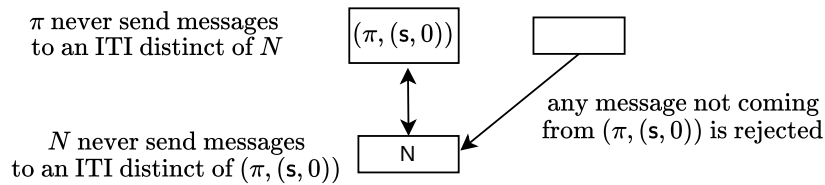


Fig. 1. Illustration of ITIs $(\pi, (s, 0))$ and N described in Section 2.1.

In Limitation (L.1) of Section 5.1, we demonstrate that this counterintuitive situation prevents π from meeting the conditions of the UC Theorem. More

⁵ Formally, the notation should indicate the specific execution being considered.

broadly, we show that this problem makes the use of the UC Theorem impossible in most practical contexts, including most ideal protocols.

To rectify this, we introduce the concept of “sleeping ITI” in Section 6.1. A sleeping ITI is one that has either not been invoked or has rejected all received messages thus far. In the earlier example, after being invoked by U , N remains a sleeping ITI because it rejects U ’s message. Based on this, we revise the definition of an extended session in Section 6.1, aligning it with its intuitive meaning.

2.2 Protocols with Dynamic Sub-Parties: A Family of Protocols Unfit for Plain UC

Before delving into the details, we summarize our contribution in a single paragraph. We show that the standard notion of subroutine respecting is insufficiently restrictive, allowing the UC Theorem to be applied to a family of protocols that do not properly fit the UC framework. Specifically, Issue (I.2) (Section 5.1) utilizes one such protocol to construct a counterexample to the UC Theorem.

In simple terms, a protocol π is considered subroutine respecting if for any execution and any sid s , any communication between an ITI within $\text{Ext}(\pi, s)$ and any other ITI must go through the main parties of $\text{Ext}(\pi, s)$. In particular, for any execution⁶:

- Any input or output sent by an ITI $U \notin \text{Ext}(\pi, s)$ to an ITI in $\text{ExtSub}(\pi, s)$ is rejected.
- No ITI in $\text{ExtSub}(\pi, s)$ sends an input or output to an ITI $U \notin \text{Ext}(\pi, s)$.

This concept is of crucial importance for the UC Theorem because it allows us to “isolate” the members of $\text{ExtSub}(\pi, s)$ from any execution, simulating any other ITI of the execution by an environment $\mathcal{E}_{\text{sim}}$, as shown in Figure 2.⁷

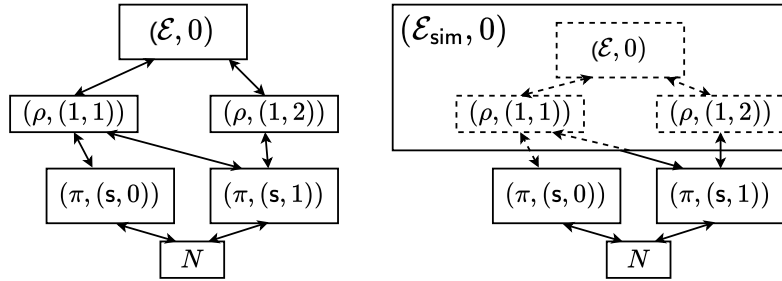


Fig. 2. An example of “isolation”: all ITIs are simulated except those within $\text{Ext}(\pi, s)$

⁶ For simplicity, we do not describe all the conditions.

⁷ Strictly speaking, achieving isolation requires additional concepts, such as compliance. However, the subroutine respecting notion remains the principal component.

However, we discovered that the conditions required for being subroutine respecting are not sufficiently restrictive. This lack of constraint allows protocols that are not well-suited for analysis within the UC framework to satisfy the hypotheses of the UC Theorem, leading to counterexamples to the theorem.

We refer to these as protocols with “dynamic subparties”. A protocol π has a dynamic sub-party N if there exists a session identifier s such that:

- (a) In some executions, $N \in \text{ExtSub}(\pi, s)$.
- (b) In other executions, N is not sleeping (i.e., it exists and has not rejected all received messages) and does not belong to $\text{ExtSub}(\pi, s)$.

While we believe that no real-world protocols have dynamic sub-parties, it is possible to construct simple protocols with this characteristic that lead to counterexamples of the UC Theorem. Such a protocol is constructed in Issue (I.2), as detailed in Section 5.1.

These protocols do not fit the UC framework because, during the execution of a protocol π with any adversary and environment, the execution rules prevent instances of condition (b) from occurring. These instances are only observable when considering the execution of another protocol ρ that uses π . As a result, the full behavior of π is unobservable when examining its UC-emulation by another protocol.

In order to exclude protocols with dynamic sub-parties from being considered subroutine respecting, we updated the notion of subroutine respecting protocols to develop the concept of sub-party respecting protocols (see Section 6.5) used in Updated UC. This definition uses the new concept of Panoptic Session, which is detailed in the next section.

2.3 Panoptic Session: A Static Variant of Extended Session

The concept of an extended session is dynamic because it is tied to a specific execution of the protocol. However, as discussed in the previous section on dynamic sub-parties, it might be beneficial to consider not only the *actual* sub-parties of an execution but also the *potential* sub-parties – that is, those that appear in some execution, without specifying which one.

To capture this idea, we introduce the notion of Panoptic Session (Section 6.3). For a protocol π and session identifier s , the panoptic session $\text{Pan}(\pi, s)$ is defined as the union of all extended sessions $\text{Ext}(\pi, s)$ across all executions of π with any environment and adversary. Therefore, the panoptic session is a static concept that provides information about the potential sub-parties of a protocol π .

For example, in the situation depicted in Figure 3, $\text{Ext}(\pi, 0) = \{((\pi, (0, 0)), N_i)\}$ for each execution such that $(\pi, (0, 0))$ sampled i , and $\{(\pi, (0, 0))\} \cup \{N_i : i \in \{1, \dots, 100\}\} \subset \text{Pan}(\pi, 0)$.

We assert that computing the panoptic sessions is straightforward for practical protocols. In fact, for any protocol analyzed under Plain UC, the panoptic sessions are already effectively determined, since checking the subroutine respecting condition requires computing the extended sessions that comprise it.

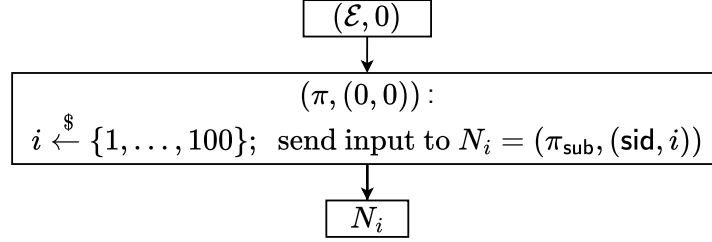


Fig. 3. Example to illustrate the difference between Extended and Panoptic Sessions.

In the previous section, we explained that panoptic sessions are used to update the notion of subroutine respecting. In the upcoming section, we will demonstrate their applicability in addressing a flaw related to the subroutine exposing notion.

2.4 Problems with the Subroutine Exposing Property

In this section, we identify two flaws in the subroutine-exposing property – a requirement of the UC Theorem – and explain how they are addressed by the sid-exposing property, which updates it in Updated UC.

For a protocol α , the subroutine exposing property requires that for each sid s , one of the ITIs in the extended session $\text{Ext}(\alpha, s)$ can inform – upon request – the adversary whether an ITI N belongs to the extended session $\text{Ext}(\alpha, s)$ at the time of the request. We refer to this ITI as the exposing ITI of the extended session $\text{Ext}(\alpha, s)$, or simply the exposing ITI when the context is clear.⁸ It is also required that the extended identity of the exposing ITI for the extended session $\text{Ext}(\alpha, s)$ can be determined using only α and s .

A Flawed Generic Transformation for Adding the Subroutine Exposing Property. There is a straightforward mechanism, described in [11, (Section 6.1)], that transforms any protocol ϕ into a subroutine exposing protocol ϕ_{SE} . In summary, ϕ_{SE} introduces in any extended session $\text{Ext}(\phi_{\text{SE}}, s)$ a middleman $M_{\phi, s}$ which is aware of all ITIs belonging to the extended session $\text{Ext}(\phi_{\text{SE}}, s)$ and can therefore act as an exposing ITI, telling the adversary whether any ITI N belongs to $\text{Ext}(\phi_{\text{SE}}, s)$.

To our knowledge, most protocols in the UC framework are not subroutine-exposing. Consequently, applying this transformation is essential to grant them that property and ensure they enjoy the UC Theorem’s security guarantees—yet its use is almost always implicit.

⁸ Note that the exposing ITI is sometimes called a "directory ITI" in Plain UC.

For the transformation to be sound, it must preserve UC-emulation: whenever π UC-emulates ϕ , then π_{SE} should UC-emulates ϕ_{SE} . However, Limitation (L.2) (Section 5.3) shows this preservation is not guaranteed.

As a consequence, protocol relying on this transformation lack the formal assurance of composable security ensured by the UC Theorem.

We illustrate the problem with a simple example depicted in Figure 4. In this example, ϕ^* performs cryptographic operations without utilizing any subroutines, whereas the protocol π^* executes the same operations using two specialized subroutines. These subroutines are selected based on the type of input – for example, α might use an algorithm optimized for inputs of type 1, while β is more efficient for inputs of type 2.

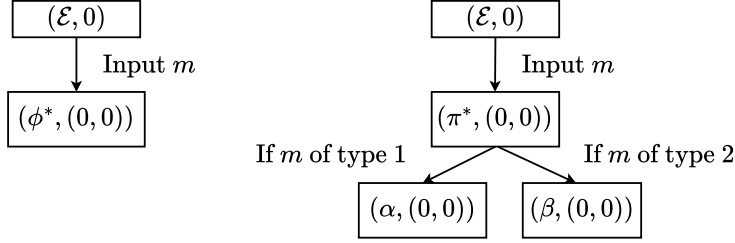


Fig. 4. Protocols ϕ^* and π^* used to describe a flaw in the transformation $\mu \rightarrow \mu_{SE}$.

By definition of π^* , $(\alpha, (0, 0))$ (*resp.* $(\beta, (0, 0))$) is part of the extended session $\text{Ext}(\pi, 0)$ if and only if the environment sent to $(\pi, (0, 0))$ an input of type 1 (*resp.* of type 2). This implies that an adversary against π_{SE}^* can leverage the exposing ITI to determine whether the environment sent an input of type 1 or 2. In contrast, an adversary against ϕ_{SE}^* does not gain additional information by using the exposing ITI.

This asymmetry in the information provided by the transformation illustrates why one cannot conclude that π_{SE}^* UC-emulates ϕ_{SE}^* simply because π^* UC-emulates ϕ^* .

To resolve this problem, we leverage the Panoptic Session concept to refine the subroutine exposing property, yielding the sid-exposing property of Updated UC (Section 6.4). Under this notion, the exposing ITI can only determine whether an ITI is in $\text{Pan}(\pi, 0)$ rather than in $\text{Ext}(\pi, 0)$. Informally, the “updated” exposing ITI – its equivalent in the sid-exposing property – reveals if a subroutine *could* belong to $\text{Ext}(\pi, 0)$, whereas the “original” exposing ITI indicates if it *actually* belongs to $\text{Ext}(\pi, 0)$.

Applying this updated definition to the example in Figure 4, the updated exposing ITI is of no use in discerning the type of message the environment sent. It responds affirmatively for both ITIs $(\alpha, (0, 0))$ and $(\beta, (0, 0))$, since both belong to $\text{Pan}(\pi, 0)$.

proof for Plain UC studies some executions of extended systems that are not protocol executions – recall that a protocol execution is just a special case of an extended-system execution. We call these *replacement executions*.

Informally, a replacement execution of a protocol ρ is one in which some subroutines with code ϕ are substituted by other subroutines with code π . Figure 6 illustrates this: the left-hand side shows a normal execution, while the right-hand side depicts a replacement execution.

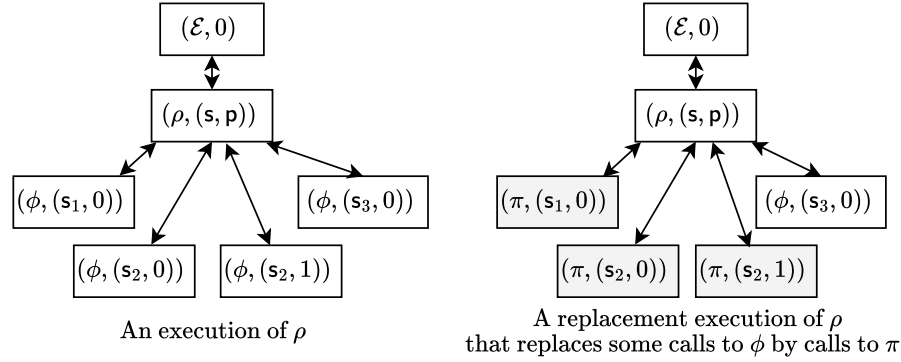


Fig. 6. Illustration of a replacement execution.

In this work, we demonstrate that the conditions required by the subroutine-respecting and compliance notions should apply not only to protocol executions but also to certain replacement executions.

In particular, Issue (I.3) (Section 5.1) exploits this omission in the subroutine-respecting notion to construct a counterexample to the UC Theorem of Plain UC.¹²

In Updated UC, we update both the subroutine-respecting and compliance notions to enforce their conditions on these additional executions as well. To that end, we formally define the notion of replacement execution in Section 6.2.

2.6 Identifying Concerns in the UC Operator and Reimagining It for Enhanced Modularity

As explained in the introduction, the UC Theorem requires the construction of a protocol $\rho^{\phi \rightarrow \pi}$ for protocols ρ , ϕ , and π . Simply stated, $\rho^{\phi \rightarrow \pi}$ is a modification of ρ that replaces any call to ϕ by a call to π . We refer to the function that, given ρ, ϕ, π , returns $\rho^{\phi \rightarrow \pi}$ as the UC Operator.

¹² Regarding the compliance, the corresponding problem we found is treated as a weakness (Weakness (W.4) in Section 5.4), since we have not attempted to build a counterexample based on it.

While the purpose of the UC Operator can be simply described, its detailed construction [11, (Section 6.1)] is intricate, demanding careful attention to various challenges. This intricacy is the origin of two main contributions of our article. First, we identified a subtle issue in its conception, leading to a complete counterexample of the UC Theorem (Issue (I.4) in Section 5.2). Second, we found a problem in how the UC Operator is used within the proof of the UC Theorem (Weakness (W.1) in Section 5.2).

To mitigate these two problems, it would be sufficient to create a new UC Operator and revisit the proof of the UC Theorem. However, we believe these issues illustrate that the UC Theorem should not directly depend on any specific UC Operator construction. Instead, a more modular formulation is necessary.

To achieve this, we introduce in Section 6.7 a general definition of the UC Operator. Our definition outlines the necessary properties for a UC Operator, regardless of how it is precisely constructed. This enables us to provide a formulation of the UC Theorem that depends solely on this abstract definition (Theorem 1). Any operator that satisfies this definition can then be utilized, resulting in a “concrete” version of the UC Theorem using this operator (Corollary 1).

This approach enhances modularity, offering several advantages. The proof of the UC Theorem becomes simpler because it does not need to address the construction nuances of a particular UC Operator, effectively addressing Weakness (W.1). Moreover, the correctness of the theorem remains unaffected by any potential errors in an individual UC Operator.

As a supplementary contribution, we provide in Appendix E the construction of a UC Operator that conforms to our definition. Notably, our construction is unaffected by Issue (I.4).

2.7 Revealing Hidden Scope Constraints in the UC Theorem

The UC Theorem states that for protocols ϕ , π , and ρ , under specific hypotheses, if π UC-emulates ϕ , then $\rho^{\phi \rightarrow \pi}$ UC-emulates ρ . Recall that $\rho^{\phi \rightarrow \pi}$ is a modification of ρ where instances of ϕ are replaced by instances of π .

In this section, we explain that the UC Theorem does not impose sufficient restrictions on which protocols ρ can be used.

Indeed, while some restrictions apply to ρ , it is possible for ρ to invoke ITIs with both codes ϕ and π . In $\rho^{\phi \rightarrow \pi}$, ITIs with code ϕ are replaced with π , whereas ITIs with code π remain unchanged. Consequently, ITIs N with code π in $\rho^{\phi \rightarrow \pi}$ can be classified into two types:

- Type 1: N was invoked due to a replacement $\phi \rightarrow \pi$,
- Type 2: N is not the result of such a replacement.

This is illustrated by Figure 7.

The existence of these two types of ITIs gives rise to a subtle weakness in the proof of the UC Theorem (Weakness (W.3), Section 5.4). Specifically, the proof

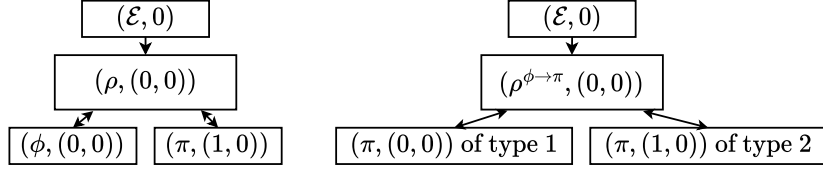


Fig. 7. An illustration of the two types of ITIs with code π described in Section 2.7.

requires one adversary to distinguish between Type 1 and Type 2 ITIs, yet it lacks the information to make this distinction.¹³

As a result, contrary to its stated claim, the UC Theorem applies only to protocols ρ whose executions exclude any Type 2 ITI.¹⁴ This restriction still applies to Updated UC, and is captured by the update of the compliance notion (see Section 6.6).

2.8 Proof of the UC Theorem Accounting for the Updated Definitions

In Section 7, we state the UC Theorem of Updated UC (Theorem 1), with its full proof provided in Appendix C.

The argument adapts the core ideas from Plain UC to Updated UC, with increased precision to ensure correctness. For instance, Equality [11, (Equation 7, page 68)], previously stated without detailed justification, is proved as Lemma 2.

As already explained in Section 2.6, the proof is independent of any particular UC operator. Furthermore, to improve its clarity and verifiability, it is more structured – it is organized around a central lemma (Lemma 1), supported by two auxiliary lemmas (Lemmas 2 and 3).

2.9 Additional Contributions

As a side contribution, we discuss additional problems in Appendix F. These are listed in Table 3 and cover three concepts not addressed by the problems described in the main body.¹⁵ These concepts are:

- The ability of a machine to *designate a message receiver using a predicate* rather than specifying its full code and identity [11, Section 3.1.2].

¹³ In specific cases – where additional information about ρ is available – this restriction could be relaxed to distinguish between Type 1 and Type 2. However, such information is not always available, which creates the problem.

¹⁴ It also formally applies to protocols excluding any Type 1 ITI, but that case is vacuous – $\rho^{\phi \rightarrow \pi}$ would never replace ϕ with π .

¹⁵ Note that the category “other” used in the table refers to a weakness in the proof of the UC Theorem that is not directly related to any specific concept of UC. It addresses a deficiency in the definition of an adversary.

- The *time-complexity* notion used in Plain UC [11, Section 3.2].
- The concept of *balanced environment* [11, Section 4.2], proposed to prevent situations where the adversary is not allocated sufficient execution time.

Table 3. Additional issues, limitations, and weaknesses described in the article. They complete the main problems listed in Table 1.

Notions	Issues	Limitations	Weaknesses
Extended Session		(L.3)	
Subroutine Respecting			
UC Operator	(I.5) (I.6)		
Predicates to Designate Receiver	(I.7)	(L.4)	
Time Complexity			(W.6) (W.7)
Balanced Environment			(W.8)
Other			(W.5)

To address these additional problems, we implement the following changes in Updated UC:

- We remove the ability to use predicates to designate receivers. This simplifies our presentation, and we have not identified any protocols proven UC-secure that rely on them.
- Regarding the issues with the time complexity notion in Plain UC, which are not central to our article, we adopt a simpler notion, leaving a more general treatment for future work.
- We do not provide a direct equivalent to the balanced environment concept from Plain UC. However, our definition of a PPT adversary includes guarantees that ensure sufficient computation time.

3 Preliminary Notations

The set of binary words is denoted by $\{0,1\}^*$ and the empty word by ϵ . In both Plain UC and Updated UC, we need to represent pairs and additional symbols – such as $\diamond$ or $\perp$ – on the tapes of Turing Machines, which can normally only contain binary words. We achieve this by implicitly using an encoding to represent them within $\{0,1\}^*$.

4 Overview of Plain UC

In this section, we recall some fundamental concepts of Plain UC, which will largely be utilized in Updated UC as well.

We made slight adjustments to the presentation, particularly in the definition of protocol execution. In addition, two more significant changes are made compared to Plain UC. First, as stated in the technical details (Section 2.9), we do not allow ITMs to designate recipients using predicates. Second, we use an alternative notion of time complexity along with a slightly modified notion of parametrized systems.

Because they are not necessary for understanding the problems of Plain UC discussed later, we omit the original time complexity definitions from the main body. However, they are recalled and discussed in Appendix F.5.

Finally, note that some notions – significantly revised in Updated UC – are not described here. Instead, they will be introduced in the sections that address the corresponding issues in Plain UC. These include concepts such as subroutine respecting, subroutine exposing, and compliance.

ITM, External Write, Protocol. The machines evoked in introduction are Interactive Turing Machines (ITM) [11, (Definition 4)]. They are special Turing machines that can send messages to each other through an operation called *external-write*, described below. We generally denote the code of an ITM using lowercase Greek letters.

An ITM μ contains multiples tapes, including:

- A read-only *identity tape* containing its code μ and an identity $\text{id} \in \{0, 1\}^*$. This pair (μ, id) is known as its *extended identity*.
- An *activation tape* containing 0 or 1, used during execution to designate which ITM can perform computations – only one ITM at a time.
- A read-only, read-once *randomness tape*, initialized with random bits.
- An *outgoing message tape* used to (1) send messages via the external-write operation, and (2) indicate the execution result (see the paragraph “Extended System and Execution of an Extended System” page 18).
- Three externally-writable tapes (*i.e.* writable only by other ITMs through the external-write operation) for receiving messages sent by other ITMs, with one tape for each type of message: an *input*, a *subroutine-output* (often abbreviated as *output*), and a *backdoor* tape.

Before executing an external-write instruction, an ITM must specify all information regarding the message to be sent. This is achieved by writing a tuple $(\text{forced-write}, \text{extid}_{\text{rec}}, \text{tape}, \text{reveal-ext-id}, \text{extid}_{\text{send}}, \text{m})$ on the outgoing message tape, where:

- $\text{extid}_{\text{rec}}$ is the extended identity of the recipient.
- $\text{tape} \in \{\text{input}, \text{subroutine-output}, \text{backdoor}\}$ is the destination tape.
- $\text{m} \in \{0, 1\}^*$ is the message to be sent, commonly referred to as input, output, or backdoor, depending on the chosen destination tape.
- $\text{reveal-ext-id} \in \{0, 1\}$ is a flag indicating whether the sender’s external identity is sent with m .
- $\text{forced-write} \in \{0, 1\}$ is a flag indicating whether the recipient ITI should be created if it does not exist when the external-write is applied.

We denote by `SetExtWrites` the set of such tuples. The complete behavior of this instruction is described in paragraph “Extended System and Execution of an Extended System”, page 18

We often use the term protocol to designate an ITM or a parametrized system (defined in paragraph “Parametrized Systems and Execution of a Parametrized System” page 20).

Configuration, ITI. A *configuration* [11, (Definition 5)] of an ITM consists of its current tape, the contents of all its tapes, and the location of the head on each tape. A configuration is active if the activation tape is set to 1, otherwise it is inactive.

An instance [11, (Definition 5)] of an ITM (ITI) μ consists of the content of its identity tape, whose is its extended identity $M = (\mu, \text{id})$. By abusing notation, we often speak of “ITI” instead of “extended identity”. For example, if an algorithm has an input which is an extended identity, we say that its input is an ITI. We generally denote ITIs using uppercase Latin letters.

Extended System and Execution of an Extended System. An extended system of ITMs [11, (section 3.1.2)] is a pair $(\mu, \mathcal{C})$, where μ is an ITM called the initial ITM and $\mathcal{C} : \{0, 1\}^* \rightarrow \{\text{allow}, \text{disallow}\} \cup \text{SetExtWrites}$ is a function called the control function, which role is described below.

To execute an extended system $(\mu, \mathcal{C})$ with an input $z \in \{0, 1\}^*$, the ITI $(\mu, 0)$, called the initial ITI, is first activated – *i.e.* 1 is written on its activation tape –, with z written on its input tape. During execution, there is no distinct ITIs with same extended identity and only the activated ITI can perform operations. An ITI will be deactivated in favor of another ITI in two occasions: when it makes an external-write or when it deactivates itself. We describe these two situations below and conclude the paragraph by defining the end and result of an execution.

EXTERNAL-WRITE. When activated, an ITI can send inputs, outputs, or backdoors to another ITI via the external-write operation. If the recipient ITI does not exist and **forced-write** is set, the recipient ITI is created in its initial state and receives the message in the corresponding input, output or backdoor tape. If the recipient ITI does not exist and **forced-write** is unset, the initial ITI $(\mu, 0)$ is instead activated, with no message. The value of flag **reveal-ext-id** is written with the message and, if it is set, the extended identity of the sender is also written.¹⁶ The effect of an external-write can be modified by the control function $\mathcal{C}$. When an external-write operation is made, $\mathcal{C}$ receives the history of all consecutive configurations of each activated ITIs and can decide to allow, disallow or replace the external-write operation with another. If the control function allows the external-write, nothing is changed. If it is disallowed, initial ITI $(\mu, 0)$ is activated next, with no message. If it is replaced by another external-write operation, the new external-write operation is applied.

¹⁶ The various elements written with the message are encoded in a way that ensures they are clearly separated. For example, it is not possible to perform an external-write with **reveal-ext-id** unset such that the recipient would interpret it as one with **reveal-ext-id** set.

SELF-DEACTIVATION, REJECT AND HALT. An activated ITI can deactivate itself. When this is done, the next activated ITI is the initial ITI $(\mu, 0)$, with no message. A *reject* [11, (Section 3.1.2)] is a specific case of deactivation, where the state of the ITI as well as all its non read-once tables are returned in their state prior to the last activation. Moreover, each ITI can enter in *halt* state. Upon entering the halt state, an ITI deactivates and will reject any subsequent messages.

END OF EXECUTION. The execution ends when the initial ITI halts. The result of the execution is the first bit of the outgoing message tape. We sometimes say “the environment returns b ” to indicate the termination and result of the execution.

Extended Execution. By an *extended execution*, we mean the execution of an extended system.

Execution Prefix. An execution prefix of an extended execution is a complete description of the extended execution from its start up to a given point in time.

Protocol Execution. The most important example of extended execution is the protocol execution [11, (Section 4.1)].

Let $\alpha, \mathcal{A}, \mathcal{E}$ three ITMs. We will refer to any ITI different of $(\mathcal{E}, 0)$ and $(\mathcal{A}, \diamond)$ as a protocol ITI.¹⁷ The execution of the protocol α with adversary $\mathcal{A}$ and environment $\mathcal{E}$ is the execution of the extended system $(\mathcal{E}, \mathcal{C}_{\text{EXEC}}^{\alpha, \mathcal{A}, \mathcal{E}})$, the control function ensuring some restrictions:

- Unless explicitly stated otherwise, the identity of any recipient of an external-write operation must be structured as a pair $(s, p) \in \{0, 1\}^* \times \{0, 1\}^*$ where s is called the *session identifier* (sid) and p the *party identifier* (pid).
- $(\mathcal{E}, 0)$ always set **forced-write** and can only send inputs. When $(\mathcal{E}, 0)$ sends input to an ITI with identity $\diamond$, the recipient ITI is replaced by $(\mathcal{A}, \diamond)$. When $(\mathcal{E}, 0)$ sends an input to an ITI with identity distinct of $\diamond$, the recipient’s code is replaced by α . When $\mathcal{E}$ sends an input to an ITI with identity distinct of $\diamond$, the identity must be structured as a couple (s, p) and it must consistently use the same sid s , usually denoted as s_{test} and called the test sid. Finally, we require that $(\mathcal{E}, 0)$ always sets **reveal-ext-id**, but it has impersonation power: in all its external-write operations, $\mathcal{E}$ can use a sender ITI other than $(\mathcal{E}, 0)$.¹⁸
- $(\mathcal{A}, \diamond)$ always unsets **forced-write**, **reveal-ext-id** and can only send backdoors.¹⁹ If it sends a backdoor to an ITI with identity 0, the recipient is replaced by $(\mathcal{E}, 0)$.

¹⁷ Note that in plain UC, the identity of adversary is normally $(s_{\text{test}}, \diamond)$ and not $\diamond$. This is a cosmetic change.

¹⁸ This simplifies the original formulation where the environment could set or unset this flag. It is essentially equivalent because the impersonation power allows the environment to conceal its extended identity.

¹⁹ [25, Appendix J.1] explains why **reveal-ext-id** needs to be unset.

- Protocol ITIs always set `reveal-ext-id` and `forced-write`.^{20 21} When a protocol ITI sends a backdoor, the recipient ITI is replaced by $(\mathcal{A}, \diamond)$. When a protocol ITI with code α and `sid` $\mathbf{s}_{\text{test}}$ (the test `sid`) sends an output to an ITI M that (1) does not have code α and `sid` $\mathbf{s}_{\text{test}}$, and (2) does not exist in the system, the message is written to the output tape of $(\mathcal{E}, 0)$, along with the target extended identity and the sender’s identity – but not the sender’s code.²²

Protocol Execution Prefix. A α -execution prefix is a prefix of the execution of α . A protocol execution prefix is a α -execution prefix for some protocol α .

Parametrized Systems and Execution of a Parametrized System. As mentioned at the beginning of this section, the definition of parametrized system we use is slightly different from that of Plain UC [11, (Section 3.2)].

The UC-emulation property is asymptotic, both in terms of the size of messages and with respect to a security parameter $k \in \mathbb{N}$. The notion of parametrized system is a generalization of the notion of an ITM that takes into account this security parameter. Essentially, it is a sequence of ITMs indexed by the security parameter.

More formally, a parametrized system is a sequence of ITMs $\phi = (\phi_k)_{k \in \mathbb{N}}$ such that the code of each ϕ_k can be written in polynomial time in k . More precisely, there exists a Turing machine and a polynomial P such that, given k , it returns the code ϕ_k in fewer than $P(k)$ operations. Parametrized systems will generally be written using bold lowercase Greek letters.

The execution of a parametrized system ϕ is defined as the execution of a protocol, with slight differences:

- All the inputs to $(\mathcal{E}, 0)$ must be pairs (k, z) , for $k \in \mathbb{N}$ and $z \in \{0, 1\}^*$. k is called the security parameter of the execution.
- The adversary has a new read-only tape containing the security parameter of the execution.²³
- When the security parameter of the execution is k , the rules of execution of ϕ_k are applied.

We denote by $\mathcal{C}_{\text{EXEC}}^{\phi, \mathcal{A}, \mathcal{E}}$ the control function and by $\text{EXEC}_{\phi, \mathcal{A}, \mathcal{E}}(k, z)$ the binary random-variable representing the result of the execution of ϕ with adversary $\mathcal{A}$ and environment $\mathcal{E}$ on input (k, z) .

²⁰ In the original formulation, `forced-write` could be unset for inputs and outputs. However, since we removed the ability to use predicates to designate the receiver, we do not consider this option relevant.

²¹ In the original formulation, `forced-write` needs to be unset for backdoor and thus the adversary cannot be invoked by a protocol ITI. This might be because executions are made with balanced environments [11, Section 4.2], where the adversary is invoked before any protocol sends a backdoor. As we do not restrict to such environment, we take in account the possibility for a protocol ITI to invoke the adversary.

²² Note that in this situation, the sender identity takes the form $(\mathbf{s}_{\text{test}}, \mathbf{p})$ for some $\mathbf{p} \in \{0, 1\}^*$.

²³ This differs from the usual formulation of Plain UC, and we believe this variant is more practical.

Subroutine, Extended Session. Let's consider M and M' two ITIs in an execution prefix. M' is a *subroutine* [11, (Section 3.1.2)] of M if M has passed input to M' and M' has not rejected it, or if M' has passed output to M and M has not rejected it.

The *extended session* [11, (Section 3.1.2)] of a protocol can be viewed as the set of ITIs that are executed by an instance of the protocol. More formally, considering μ an ITM, a sid $s \in \{0,1\}^*$ and some prefix of execution θ of a protocol, the extended session $\text{Ext}(\mu, s, \theta)$ is a set constructed inductively by inspecting the ITIs by order of invocation: each newly invoked ITI M is added to the extended session if (a) M has code μ and sid s (such an ITI is called a main party); or (b) M is a subroutine of an ITI that is already in the extended session; or (c) M is invoked by an ITI that is a member of the extended session, but not a main party. Note that determining whether an ITI belongs to an extended session only requires examining the first message received by the ITI. This is because ITIs are processed in the “order of invocation” and that we focus on “newly invoked ITI”. Therefore, it is equivalent to replace item (b) by: “it is invoked with an input by a main party and does not reject the input”.

The ITIs with code μ and sid s are called the main parties of $\text{Ext}(\mu, s, \theta)$ and form a set $\text{MainParty}(\mu, s)$. The members of the extended session that are not main parties are called *sub-parties* and form a set $\text{ExtSub}(\mu, s, \theta)$.

We will often omit the prefix θ from $\text{Ext}(\mu, s, \theta)$ when clear from context. In the context of an execution, we speak of an extended session of μ to mean any extended session $\text{Ext}(\mu, s)$ for some sid $s \in \{0,1\}^*$.

Finally, note that in the execution of a protocol μ with test sid s_{test} , the extended session $\text{Ext}(\mu, s_{\text{test}})$ is referred to as the extended test session.

Probabilistic Polynomial Time ITM. We introduce here the notions of PPT (Probabilistic Polynomial Time) adversary, environment, and protocol – formally defined in Appendix C.2.

These definitions differ from the notion of PPT ITM given in Plain UC [11, (Section 3.2)]. They are partially inspired by the concept of a polynomial time machine from [12, (Section 2.1)] and focused on the context of parametrized system execution.

An adversary is PPT if there exists a polynomial P such that at any moment in any possible execution, its time of execution is bounded by $\sum_i P(|w_i| + k)$ where $(w_i)_i$ is the sequence of external-writes it received and k the security parameter written on its dedicated tape. Thus, the adversary can always perform at least $n_M P(k)$ operations, where n_M is the number of messages it received.

An environment is PPT if there exists a polynomial P such that at any moment in any possible execution, its execution time is bounded by $P(k + |z|)$, where $(k, z) \in \mathbb{N} \cup \{0,1\}^*$ is its input, given at its first activation.

A protocol α is PPT if for any PPT environment $\mathcal{E}$ and PPT adversary $\mathcal{A}$, there exists a polynomial R such that any execution of α with environment $\mathcal{E}$, adversary $\mathcal{A}$ and input $(k, z) \in \mathbb{N} \cup \{0,1\}^*$ has an execution time bounded by $R(k + |z|)$.

UC-emulation. The definition of UC-emulation relation we give is similar to that in Plain UC, but it incorporates our variants of parametrized system and PPT notions for adversary and environment. Additionally, we do not consider the notion of balanced environment [11, (Section 4.2)].²⁴

To be defined, the UC-emulation relation requires three preliminary notions.

Let ξ a predicate on extended identities. An environment is said ξ -*identity-bounded* [11, (Section 4.2)] if it only impersonates extended identities M for which $\xi(M)$ is true.

A binary probability distribution ensemble [11, (Section 4.2)] is an infinite set $X = \{X(k, z)\}_{k \in \mathbb{N}, z \in \{0,1\}^*}$ of binary probability distributions, indexed by $\mathbb{N} \times \{0,1\}^*$.

Two binary probability distribution ensembles X and Y are *indistinguishable* [11, (Definition 8)] – written $X \approx Y$ – if for any $c, d \in \mathbb{N}$, there exists $k_0 \in \mathbb{N}$ such that for all $k > k_0$ and all $z \in \bigcup_{\kappa \leq k^d} \{0,1\}^\kappa$, we have: $|\Pr[X(k, z) = 1] - \Pr[Y(k, z) = 1]| < k^{-c}$. They are said to be *perfectly indistinguishable* if we always have $\Pr[X(k, z) = 1] = \Pr[Y(k, z) = 1]$. The execution of a parametrized system ρ naturally gives a binary probability distribution ensemble $(\text{EXEC}_{\rho, \mathcal{A}, \mathcal{E}}(k, z))_{k \in \mathbb{N}, z \in \{0,1\}^*}$ that is used in the definition of UC-emulation.

Definition 1 (UC-emulation [11, (Definition 9)]). We consider π, ϕ two PPT parametrized systems and ξ a predicate on extended identities. We say that π UC-emulates ϕ with respect to ξ -identity-bounded environments (or, π ξ -UC-emulates ϕ) if for any PPT adversary $\mathcal{A}$, there exists a PPT adversary $\mathcal{S}$ such that for any PPT, ξ -identity-bounded environment $\mathcal{E}$, we have $\text{EXEC}_{\pi, \mathcal{A}, \mathcal{E}} \approx \text{EXEC}_{\phi, \mathcal{S}, \mathcal{E}}$. If ξ is the predicate which is true for all ITIs, then π is said to UC-emulate ϕ . The UC-emulation is said to be perfect if the two probability distributions ensembles are perfectly indistinguishable. If this relation is only verified for a fixed adversary $\mathcal{A}$, we say that π ξ -UC-emulates ϕ with respect to $\mathcal{A}$.

In the description of our counter-examples, we will consider UC-emulation for ITMs π and ϕ instead of parametrized systems. When we state that π UC-emulates ϕ , it should be understood as π UC-emulates ϕ for $\pi_k = \pi$ and $\phi_k = \phi$ for each $k \in \mathbb{N}$.

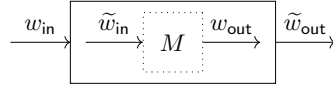
Dummy Adversary. In plain UC, an adversary $\mathcal{D}$, called the dummy adversary [11, Section 4.3.1], is of special importance as it can be shown that UC-emulation with respect to $\mathcal{D}$ implies UC-emulation [11, (Claim 11)] (i.e. UC-emulation with respect to any PPT adversary).

Specifically, the dummy adversary proceeds as follows. When activated with an input “ $(m, (\mu, \text{id}))$ ” from the environment $(\mathcal{E}, 0)$, where m is a message, and (μ, id) an extended identity, $(\mathcal{D}, \diamond)$ sends backdoor “ m ” to ITI (μ, id) . When activated with a backdoor “ m ” sent by an ITI (μ, id) , $(\mathcal{D}, \diamond)$ sends backdoor “ $(m, (\mu, \text{id}))$ ” to the environment $(\mathcal{E}, 0)$.

²⁴ This is justified by a weakness described in Appendix F.6.

$\mathcal{D}$ is a PPT adversary, as its execution time for each activation is linear in the size of the received message.

Universal Composition Operator (UC Operator). The UC operator $\rho^{\phi \rightarrow \pi}$ is defined in [11, (Section 6.1)] and relies on the notion of *structured protocol* [11, (Section 5.1)]. A structured protocol is an ITM whose functioning can be divided into two parts: a body, which is an ITI, and a shell that simulates the body and can modify the external-writes it receives or intends to send. In the following figure, a structured ITI N is shown with a body M . We observe that N can modify both the external-writes that M receives and those that M intends to send. The shell is responsible for simulating the body and applying these modifications.



In a nutshell, the ITIs of an extended session $\text{Ext}(\rho^{\phi \rightarrow \pi}, s)$ are structured ITIs whose bodies correspond to ITIs from the extended session $\text{Ext}(\rho, s)$ or from extended sessions of π . The shells of these ITIs are used to replace codes in certain sent and received external-write operations, in such a way that the execution of $\rho^{\phi \rightarrow \pi}$ corresponds to an execution of ρ in messages to sub-parties with code π are replaced by messages to ITIs with the same identity but code ϕ .

We now provide a more detailed explanation. To simplify it, we make the following three assumptions. First, no execution of ρ invokes a main party of π . Second, for any sids $s \neq s'$ and $\mu \in \{\phi, \pi\}$, $\text{ExtSub}(\mu, s) \cap \text{ExtSub}(\mu, s') = \emptyset$. Third, for any sids s, s' , $\text{Ext}(\phi, s) \cap \text{Ext}(\pi, s') = \emptyset$.

To describe the rules of replacement, we call an ITI *outside* if it does not belong to any extended session of π . The rules are as follows:

- *Replacements $\phi \rightarrow \pi$* : Inputs or outputs from an outside ITI to a main party of an extended session $\text{Ext}(\phi, s)$ are redirected to the main party of $\text{Ext}(\pi, s)$ with the same pid.
- *Replacements $\pi \rightarrow \phi$* : When a main party of an extended session $\text{Ext}(\pi, s)$ sends an output to an ITI M outside this extended session, the external-write is modified so that the sender code, originally π , is replaced by ϕ .

To apply these replacements, the shells store information through local variables and communicate with other shells. More precisely, any message sent by a structured protocol to another one can be seen as a couple $(m_{\text{body}}, m_{\text{shell}})$ where m_{body} is aimed to the body of the recipient (we call it a body message) and m_{shell} to its shell (we call it a shell message). The only aim of the shell messages is to communicate enough information to ensure good application of the replacements detailed above.

Finally, note that when applied to parametrized systems ρ , ϕ and π , $\rho^{\phi \rightarrow \pi}$ is itself a parametrized system with $(\rho^{\phi \rightarrow \pi})_k = \rho_k^{\phi_k \rightarrow \pi_k}$.

UC Theorem. The UC Theorem [11, (Theorem 22)] states that, under certain hypotheses on PPT parametrized systems ρ , ϕ , π and on a predicate ξ on extended identities, if π ξ -UC-emulates ϕ , then $\rho^{\phi \rightarrow \pi}$ UC-emulates ρ .

Concretely, they need to satisfy some conditions : (1) ϕ and π are both subroutine respecting [11, (Definition 19)] and subroutine exposing [11, (Definition 21)], and (2) ρ is (π, ϕ, ξ) -compliant [11, (Section 6.1)]. The subroutine respecting, subroutine exposing, and compliance notions are respectively linked to the problems of the UC Theorem described in sections 5.1, 5.3 and 5.4. They are detailed within those sections.

5 Problems Within The Universal Composition Theorem

This section provides a detailed account of the issues, limitations, and weaknesses in the UC theorem of Plain UC [11, (Theorem 22)] listed in Table 1.

While these points help in understanding the changes introduced by Updated UC, they are also of independent interest.

5.1 Problems Related to Extended Sessions and Subroutine Respecting Property

We begin by reviewing the concept of subroutine respecting protocols.

A protocol μ is said *subroutine respecting* [11, (Definition 19)] if, in any prefix of execution of any protocol and for any $\text{sid } s \in \{0, 1\}^*$, certain restrictions apply to communications with the extended session $\text{Ext}(\mu, s)$. For the problems we are addressing, we will focus on these four restrictions: (1) the main parties of $\text{Ext}(\mu, s)$ reject any output sent by an ITI outside $\text{Ext}(\mu, s)$; (2) the sub-parties of $\text{Ext}(\mu, s)$ reject any input or output sent by an ITI outside $\text{Ext}(\mu, s)$; (3) no member of $\text{Ext}(\mu, s)$ sends an input to an existing ITI that is not already a member of $\text{Ext}(\mu, s)$; and (4) no sub-party of $\text{Ext}(\mu, s)$ sends an output to an existing ITI that is not already a member of $\text{Ext}(\mu, s)$. Moreover, neither of the required conditions impose any restriction on received or sent backdoors.

(L.1) Extended Session Definition Excludes Too Many ITIs. The notions of extended session and of subroutine respecting protocol are overly restrictive.

Looking at the definition of extended session recalled in Section 4 page 21, we see that an ITI that is neither a main party, nor invoked by a member of the extended session, will never become a member of the extended session, even if it rejects the message that invoked it. This implies that a large class of protocols with sub-parties are not subroutine respecting, including the ideal protocol associated with an ideal functionality [11, (Section 5.3)].

We demonstrate this with a generic example of protocol with only one subroutine, illustrated in Figure 8. Considering two protocols π and π_{sub} and a $\text{pid } p_{\text{sub}}$ such that, for any $\text{sid } s \in \{0, 1\}^*$:

1. ITIs of the form $(\pi, (s, p))$ for $p \in \{0, 1\}^*$ send inputs only to $(\pi_{\text{sub}}, (s, p_{\text{sub}}))$.
2. $(\pi_{\text{sub}}, (s, p_{\text{sub}}))$ rejects any input or output message not sent by an ITI of the form $(\pi, (s, p))$ for $p \in \{0, 1\}^*$.
3. $(\pi_{\text{sub}}, (s, p_{\text{sub}}))$ sends no inputs and sends outputs only to ITIs of the form $(\pi, (s, p))$ for $p \in \{0, 1\}^*$.

4. There is a pair of sid and pid $(\tilde{s}, \tilde{p}) \in \{0, 1\}^*$ and a message $\tilde{m}$ such that, when invoked with input “ $\tilde{m}$ ” by any protocol, $(\pi, (\tilde{s}, \tilde{p}))$ sends an input to $(\pi_{\text{sub}}, (\tilde{s}, p_{\text{sub}}))$.

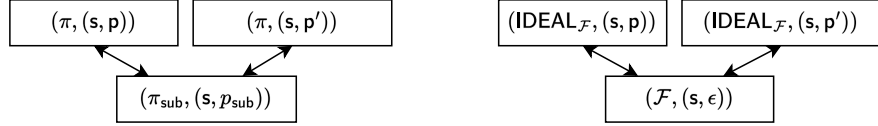


Fig. 8. Illustration of the generic example (left), with some ideal protocols as instances (right).

The protocol π should be a textbook example of subroutine respecting protocol, whose extended sessions $\text{Ext}(\pi, s)$ can only have sub-party $(\pi_{\text{sub}}, (s, p_{\text{sub}}))$. Nevertheless, this is proven to be false in next proposition.

Proposition 1. *π is not subroutine respecting.*

Proof. Considering an ITI $(\rho, (0, 0))$ that, at first activation, sends input “0” to $(\pi_{\text{sub}}, (\tilde{s}, p_{\text{sub}}))$ and, at second activation, sends input “ $\tilde{m}$ ” to $(\pi, (\tilde{s}, \tilde{p}))$. We next consider an environment $\mathcal{E}$ that, at first activation, sends input “0” to the protocol with identifier $(0, 0)$ and, at second activation, sends again input “0” to the protocol with identifier $(0, 0)$. We consider θ a prefix of execution of ρ with environment $\mathcal{E}$, some adversary and some input.

According to the definitions, we have $(\pi_{\text{sub}}, (\tilde{s}, p_{\text{sub}})) \notin \text{Ext}(\pi, \tilde{s}, \theta)$ because $(\pi_{\text{sub}}, (\tilde{s}, p_{\text{sub}}))$ is invoked by $(\rho, (0, 0))$ and $(\rho, (0, 0)) \notin \text{Ext}(\pi, \tilde{s}, \theta)$.

After its invocation, $(\pi_{\text{sub}}, (\tilde{s}, p_{\text{sub}}))$ rejects the message and the environment $(\mathcal{E}, 0)$ sends an input to $(\rho, (0, 0))$. In turn, $(\rho, (0, 0))$ invokes $(\pi, (\tilde{s}, \tilde{p}))$ with input “ $\tilde{m}$ ”, and $(\pi, (\tilde{s}, \tilde{p}))$ then sends an input to $(\pi_{\text{sub}}, (\tilde{s}, p_{\text{sub}}))$. This final input proves that π is not subroutine respecting, as a main party of $\text{Ext}(\pi, \tilde{s}, \theta)$ sent an input to an existing ITI that does not belong to $\text{Ext}(\pi, \tilde{s}, \theta)$, contradicting condition (3).

This counterexample covers numerous concrete situations. In particular, a direct verification shows that it includes ideal protocols $\text{IDEAL}_{\mathcal{F}}$ [11, (Section 5.3)] associated with an ideal functionality $\mathcal{F}$ that does not invoke other ITMs, as illustrated in Figure 8. This includes for example the ideal functionalities for PAKE [22,17,19], signature [9,1] and Oblivious Transfer [5].

To address this issue, Updated UC uses a new definition of extended session (Section 6.1).

(I.1) Subroutine Respecting Property and Backdoors.

We recall that in the execution of a protocol, the adversary cannot invoke ITIs. Nevertheless, once an ITI has been invoked it becomes accessible to the

adversary – even if it rejected all received messages. Additionally, in the full definition of the subroutine respecting notion [11, (Definition 19)], no condition applies to backdoor messages; the restrictions concern only input and output messages. These two facts lead to the following counterexample to the UC theorem.

We create an ITM π such that, for any $s, p \in \{0, 1\}^*$, $(\pi, (s, p))$ rejects any output. At reception of its first input, $(\pi, (s, p))$ initializes a variable `ReceivedInput`, then deactivates and rejects any subsequent input or output it receives. At reception of a backdoor message, if `ReceivedInput` is initialized, $(\pi, (s, p))$ rejects it. Else, if `ReceivedInput` is uninitialized, $(\pi, (s, p))$ sends an output to an ITI with code $\mu \neq \pi$.

It can be directly shown that π is subroutine respecting: it has no subroutines and rejects all output messages. Moreover, in the execution of π with any adversary and environment, all main parties of the extended test session are invoked through an input from the environment; they therefore all initialize a variable `ReceivedInput` and reject any backdoor message. As a result, it can be demonstrated that π UC-emulates the protocol ϕ that rejects any incoming message.

However, we can easily construct a protocol ρ such that $\rho^{\phi \rightarrow \pi}$ does not UC-emulate ρ . This is achieved by exploiting the fact that ρ can invoke a main party $(\phi, (s, p))$ using an output message.

Although we believe this issue does not affect real-world protocols, it highlights the need to refine the notion of subroutine respecting protocols. This refinement is introduced in Section 6.5.

(I.2) The Case of “Dynamic Sub-parties”. We introduce two concepts explain this issue. An ITI M is said to be *awake* in an execution prefix if it has been invoked and has not rejected all received messages. Additionally, we say that an ITI M is a *dynamic sub-party* of a protocol ϕ if there is a sid s and two execution prefixes $\theta_{in}, \theta_{out}$ such that $M \in \text{Ext}(\phi, s, \theta_{in})$, but $M \notin \text{Ext}(\phi, s, \theta_{out})$ and M is awake in θ_{out} .

Dynamic sub-parties do not align well with Plain UC because, during the execution of ϕ with an environment and adversary, the execution rules indicate that if M exists, it will always be a sub-party of the extended test session. Consequently, these executions do not consider scenarios where M is awake but outside the extended session. In particular, some protocols with dynamic sub-parties can lead to counterexamples of the UC Theorem.

For example, consider the protocol ϕ defined in Figure 9. We assert that it is a subroutine-respecting protocol with a dynamic sub-party $(\alpha, (0, 0))$. Specifically:

- $(\alpha, (0, 0))$ is a sub-party of $\text{Ext}(\phi, 0)$ if it is invoked with an input by $(\phi, (0, 0))$.
- $(\alpha, (0, 0))$ is awake and not a sub-party of $\text{Ext}(\phi, 0)$ if it is invoked with an input by another ITI.

To see why ϕ is subroutine respecting, note that if $(\alpha, (0, 0))$ is not invoked by an input from $(\phi, (0, 0))$, it sends an input to $(\phi, (0, 0))$. This input can be interpreted as a “warning” to inform $(\phi, (0, 0))$ that $(\alpha, (0, 0))$ is awake and does not belong to $\text{Ext}(\phi, 0)$. Once $(\phi, (0, 0))$ is “warned”, it refrains from sending

any input to $(\alpha, (0, 0))$, thereby maintaining the conditions of the subroutine-respecting property.

In Appendix D.2, we show that there exists a protocol π that UC-emulates a slight modification of ϕ , and a protocol ρ such that all the conditions of the UC theorem are satisfied. However, $\rho^{\phi \rightarrow \pi}$ does not UC-emulate ρ , thereby contradicting the UC theorem.

The core reason for the counterexample lies in the difference in the observable behavior²⁵ of ϕ when executed alone versus as a subroutine within another protocol. In any execution of ϕ with test sid 0, $(\phi, (0, 0))$ behaves essentially like a protocol that sends an output to $(\rho, (0, 0))$ upon receiving an input—regardless of whether it creates the subroutine $(\alpha, (0, 0))$. In this context, the creation of the subroutine has no observable effect.

However, this changes when ϕ is used as a subroutine by a protocol. A new observable behavior arises. Suppose, for example, that $(\rho, (0, 0))$ first sends an input to $(\phi, (0, 0))$, and then sends an output to $(\alpha, (0, 0))$. The result of the second input to $(\alpha, (0, 0))$ then depends on the first bit of randomness used by ϕ :

- If it is 1, then $(\alpha, (0, 0))$ has already been invoked by $(\phi, (0, 0))$ and rejects the input.
- If it is 0, then $(\alpha, (0, 0))$ is first invoked by a protocol other than $(\phi, (0, 0))$. It then sends an input to $(\phi, (0, 0))$, which subsequently sends an output to $(\rho, (0, 0))$.

To address this, Updated UC’s variant of the subroutine-respecting property (Section 6.1) excludes protocols with dynamic sub-parties. This is achieved through the introduction of the Panoptic Session concept, described in Section 6.3.

(I.3) Subroutine Respecting Property and Universal Composition Operator. This issue is particularly subtle, and we will only provide a brief explanation of the mechanism it exploits, with a full counterexample of the UC theorem presented in Appendix D.3.

When considering the UC theorem for protocols ρ, ϕ, π , an important hypothesis is that π is subroutine respecting. This property gives restrictions to the ITIs with code π and its sub-parties in any protocol execution.

In demonstrating the UC theorem, it’s crucial to verify these conditions within the execution of $\rho^{\phi \rightarrow \pi}$. However, such executions use structured protocols to simulate special kind of executions of ρ , where instances of ϕ are substituted with instances of π . In these executions, we can demonstrate that the simulated extended sessions of π may encounter situations that do not arise in any real extended sessions of π during any protocol execution. By definition, the subroutine respecting property of π does not cover such cases, and this absence of constraints can be exploited to construct a counterexample to the UC theorem.

While we did not find a concrete protocol with this issue, it is crucial to address it to ensure the correctness of the UC Theorem.

²⁵ *i.e.*, observable by the environment.

ρ is a protocol distinct of α and ϕ . Moreover, all external writes are made with flags reveal-ext-id and forced-write set.	
(ϕ, id)	(α, id)
if $\text{id} \neq (0, 0)$ then halt	if $\text{id} \neq (0, 0)$ then halt
<u>At reception of an input m from ITI M :</u>	<u>At reception of input m from ITI M :</u>
if it is first activation then	if it is first activation then
if $M \neq (\alpha, (0, 0))$	if $M = (\phi, (0, 0))$ then
$\wedge$ first bit of randomness is 1 then	send output “0” to $(\phi, (0, 0))$
send input “0” to $(\alpha, (0, 0))$	else
else	send input “0” to $(\phi, (0, 0))$
send output “0” to $(\rho, (0, 0))$	else halt
send output “0” to $(\rho, (0, 0))$	<u>At reception of backdoor : halt</u>
<u>At reception of output m from ITI M :</u>	<u>At reception of output : halt</u>
if $M = (\alpha, (0, 0))$ then	
send output “0” to $(\rho, (0, 0))$	
<u>At reception of backdoor : halt</u>	

Fig. 9. Algorithms of ϕ with dynamic sub-party α , for Issue (I.2).

In Updated UC, the notion of sub-party respecting (Section 6.5) addresses this issue by requiring that its restrictions hold not only for any protocol execution but also for any extended execution.²⁶

5.2 Problems Related to the Universal Composition Operator

In this section, we present an issue and a weakness linked to the UC operator [11, (Section 6.1, item 2)].

The issue highlights the necessity of replacing the UC operator, while the weakness underscores the value of having a modular formulation of the UC theorem that does not rely on a specific UC Operator implementation. These considerations are further discussed for Updated UC in Section 6.7.

(I.4) An Issue in Handling Outputs Forwarded to the Environment.

In this section, we examine a subtle mechanism required by the UC operator that was omitted from the overview for simplicity (see Section 4, page 23). We show that the UC operator in Plain UC does not implement this mechanism correctly, which we exploit to construct a counterexample to the UC theorem.

²⁶ Note that in the proof of the UC theorem, the exact set of execution prefixes required for analysis is the set of “relevant prefixes”, defined and examined in Appendix C.3. To obtain a simpler definition, we require the sub-party respecting conditions to hold for all prefixes of any extended execution, while relegating the specific treatment of relevant prefixes to the appendix.

Recall that, in the execution of a protocol ρ with environment $\mathcal{E}$ and some adversary, if a main party $(\rho, (\mathbf{s}_{\text{test}}, \mathbf{p}))$ of the extended test session sends an output to an ITI M that does not exist in the execution, the output is instead sent to $(\mathcal{E}, 0)$. Additionally, the identity $(\mathbf{s}_{\text{test}}, \mathbf{p})$ and the extended identity of M are also forwarded to $(\mathcal{E}, 0)$. Recall further that the ITIs of an extended session $\text{Ext}(\rho^{\phi \rightarrow \pi}, \mathbf{s})$ are structured ITIs whose bodies correspond either to ITIs of $\text{Ext}(\rho, \mathbf{s})$ or of extended sessions of π .

As a result, when the body of a structured ITI attempts to send such an output to an ITI M , it is crucial for the shell to determine whether a structured ITI S with body M exists in the execution. If such a structured ITI S exists, the message must be directed to S . Otherwise, the recipient must remain unmodified, as the output will be forwarded to $(\mathcal{E}, 0)$ along with the identity of the intended recipient. In this latter case, using S as the recipient instead of M would allow the environment to distinguish between executions of $\rho^{\phi \rightarrow \pi}$ and ρ .

However, the definition of $\rho^{\phi \rightarrow \pi}$ does not address this situation correctly. In an execution of $\rho^{\phi \rightarrow \pi}$ with test sid $\mathbf{s}_{\text{test}}$, when the body of a main party of $\text{Ext}(\rho^{\phi \rightarrow \pi}, \mathbf{s}_{\text{test}})$ attempts to send an output to an ITI $(\alpha, (\mathbf{s}', \mathbf{p}'))$, the procedure described in [11, (Section 6.1, item 2)] is as follows:

- If $(\alpha, \mathbf{s}') \neq (\rho, \mathbf{s}_{\text{test}})$, then $(\rho^{\phi \rightarrow \pi}, (\mathbf{s}_{\text{test}}, \mathbf{p}))$ sends the output to $(\alpha, (\mathbf{s}', \mathbf{p}'))$.
- Otherwise, if $(\alpha, \mathbf{s}') = (\rho, \mathbf{s}_{\text{test}})$, then the output is sent to $(\rho^{\phi \rightarrow \pi}, (\mathbf{s}_{\text{test}}, \mathbf{p}'))$
 - *i.e.*, the structured ITI with body $(\rho, (\mathbf{s}_{\text{test}}, \mathbf{p}'))$.

Consequently, $(\rho^{\phi \rightarrow \pi}, (\mathbf{s}_{\text{test}}, \mathbf{p}))$ behaves as if, whenever $(\alpha, \mathbf{s}') \neq (\rho, \mathbf{s}_{\text{test}})$, no structured ITI with body $(\alpha, (\mathbf{s}', \mathbf{p}'))$ exists. This assumption is incorrect and leads to counterexamples to the UC theorem, as detailed in Appendix D.4.

To correctly select the recipient in this situation, $(\rho^{\phi \rightarrow \pi}, (\mathbf{s}_{\text{test}}, \mathbf{p}))$ must be able to determine whether a structured ITI with body M is present in the execution. To enable this, the UC operator proposed in Updated UC uses a centralized approach: it employs a special ITI that tracks exchanged messages and maintains global awareness of instantiated structured protocols.

(W.1) Proof of UC theorem and structured ITIs.

In the proof of the UC theorem [11, (Theorem22)] for protocols ρ , ϕ , and π , the objective is to demonstrate the existence of a PPT adversary $\mathcal{S}$ such that the following equation [11, (Equation 2, Section 6.1)] holds for any PPT environment $\mathcal{E}$: $\text{EXEC}_{\rho^{\phi \rightarrow \pi}, \mathcal{D}, \mathcal{E}} \approx \text{EXEC}_{\rho, \mathcal{S}, \mathcal{E}}$.

In the left-hand execution, the non-directly rejected inputs that the environment $(\mathcal{E}, 0)$ can send to the dummy adversary $(\mathcal{D}, \diamond)$ are of the form (m, M) , where M is an ITI of an extended session of $\rho^{\phi \rightarrow \pi}$. As explained in Section 4 (page 23), M is thus a structured ITI *whose body is* a member of an extended session of ρ or π . However, the proof appears to assume that M *is* a member of an extended session of ρ or π — as seen, for instance, in Item 1.(c).(i) of [11, (Figure 10)]. This constitutes a missing step in the proof.

Such an oversight may be attributable to the proof's complexity, underscoring the importance of dividing the proof into smaller, more manageable components.

In Updated UC, we propose a formulation of the UC theorem that does not depend on the specific construction of the UC operator. This has the dual

advantage of making the proof easier to analyze and of facilitating the use of alternative constructions.

5.3 Problems Related to the Subroutine Exposing Property

This section presents a limitation and a weakness related to the *subroutine exposing property* ([11, (Definition 21)]), which is required for ϕ and π to apply the UC theorem.

To ease the reading, we recall the definition of a subroutine exposing protocol as provided in the technical overview (Section 2.4). For a protocol α , the subroutine exposing property requires that for each sid s , one of the ITIs in the extended session $\text{Ext}(\alpha, s)$ can inform – upon request – the adversary whether an ITI M belongs to the extended session $\text{Ext}(\alpha, s)$ at the time of the request. We refer to this ITI as the exposing ITI of the extended session $\text{Ext}(\alpha, s)$, or simply the exposing ITI when the context is clear. It is also required that the extended identity of the exposing ITI for the extended session $\text{Ext}(\alpha, s)$ can be determined using only α and s .²⁷

(L.2) Adding the Subroutine Exposing Property to a Protocol Does Not Preserve UC-Emulation.

To facilitate reading, we begin this section by restating the explanation provided in the technical overview (Section 2.4).

There is a straightforward mechanism, described in [11, (Section 6.1)], that transforms any protocol ϕ into a subroutine exposing protocol ϕ_{SE} . In summary, ϕ_{SE} introduces, in any extended session $\text{Ext}(\phi_{\text{SE}}, s)$, a middle-man $M_{\phi, s}$ that is aware of all ITIs belonging to the extended session and can therefore act as an exposing ITI, telling the adversary whether any ITI N belongs to $\text{Ext}(\phi_{\text{SE}}, s)$.

To our knowledge, most protocols in the UC framework are not subroutine exposing. Consequently, applying this transformation is essential to grant them that property and ensure they benefit from the UC theorem’s security guarantees.

For the transformation to be sound, it must preserve UC-emulation: whenever π UC-emulates ϕ , we should also have that π_{SE} UC-emulates ϕ_{SE} . However, this preservation is not guaranteed. We demonstrate this with a simple, intentionally constructed counterexample.

Let π and ϕ be the protocols described in Figure 10. Since neither π nor ϕ produces any output or interacts with the adversary, we know that π UC-emulates ϕ . Furthermore, both protocols are subroutine respecting. As a result, the only missing property for applying the UC theorem is subroutine exposing. Nevertheless, π_{SE} does not UC-emulate ϕ_{SE} . To see this, observe that $(\alpha, (0, 0))$ belongs to $\text{Ext}(\pi, 0)$ if and only if the environment invoked $(\pi, (0, 0))$ with input “0”. This implies that an adversary of π_{SE} can use the subroutine-exposing ITI to determine whether the environment invoked the protocol with identifier $(0, 0)$ using input “0”. In contrast, an adversary of ϕ_{SE} lacks such a mechanism.

²⁷ The exposing ITI is sometimes called a “directory ITI” in Plain UC.

This problem can arise in real-world protocols, where the existence of an ITI depends on an action taken by the environment. An example is given in the technical overview (Section 2.4).

The consequences of this limitation are twofold: first, it demonstrates that the subroutine exposing property must be explicitly accounted for in protocols analyzed under Plain UC, as the generic transformation does not preserve UC-emulation. Second, it shows that some realistic examples of UC-emulations cannot be leveraged within the UC theorem.

In Updated UC, we adopt a variant of subroutine exposing (Section 6.4) that eliminates the need for such a transformation and enables the analysis of more protocols, including the example from the technical overview (Section 2.4).

All external writes are made with flags reveal-ext-id and forced-write set.	
(π, id)	(α, id)
if $\text{id} \neq (0, 0)$ then halt	halt
At reception of an output or a backdoor : halt	
At reception of an input m from an ITI M :	(ϕ, id)
if it is first activation $\wedge m = 0$ then send input “0” to $(\alpha, (0, 0))$	halt
else halt	

Fig. 10. Algorithms of π, α and ϕ for Limitation (L.2).

(W.2) Determining the sids of Existing Sessions of ϕ . In the proof of UC theorem [11, (Theorem 22)] for protocols ρ, ϕ , and π ,²⁸ the author constructs a PPT adversary $\mathcal{S}$ [11, (Figure 10)] of ρ that behaves differently when receiving a backdoor from an ITI M , depending on whether there exists a sid s such that $M \in \text{Ext}(\phi, s)$.

$\mathcal{S}$ relies on the subroutine exposing property of ϕ to determine if an ITI belongs to an extended session of ϕ during the execution of ρ (see [11, (2.(a) of Figure 10)]). Since using the exposing ITI of an extended session $\text{Ext}(\phi, s)$ generally requires knowing s , $\mathcal{S}$ must know the sids of the existing extended sessions of ϕ in the current execution of ρ . However, no mechanism is provided to identify them. This is critical because the number of possible sids could be exponential, meaning that the adversary would not be PPT if required to test all possible sids.

For instance, consider a protocol ρ that, upon activation, samples a random $s \in \{0, 1\}^k$ and then invokes $(\phi, (s, 0))$: the number of possible sids is exponential in the security parameter k .

²⁸ In discussing this weakness, we simplify the explanation by omitting the concept of top-level sessions [11, (Section 6.1)], which is not essential for understanding and will not be required for the rest of the document.

We believe than this weakness cannot be solved in Plain UC without additional properties. A direct solution could be to only consider protocols ρ that informs the adversary of these sids. Nevertheless, we think this would overly restrict the applicability of the UC theorem.

In Updated UC, this weakness does not exist due to our use of a variant of the subroutine exposing property, described in Section 6.4.

5.4 Problem Related to the Compliance Property

This section presents a weakness related to the *compliance* property ([11, (Section 6.1)]), which is required to apply the UC theorem. It highlights that this notion is not restrictive enough.

Compliance is the sole requirement concerning the protocol ρ in the UC Theorem, aside from being PPT. Simply put, for protocols ϕ and π , and ξ a predicate on extended identities, ρ is (ϕ, π, ξ) -compliant [11, (Section 6.1)] if, in any execution²⁹ of ρ : (1) some ITIs satisfy ξ ; and (2) if two external-write made by protocol ITIs have target codes π and ϕ , they have distinct target sids.

(W.3) Identifying Whether π is the Result of a Replacement.

In the proof of the UC theorem [11, (Theorem22)] for protocols ρ , ϕ , and π ,³⁰ the author constructs a PPT adversary $\mathcal{S}$ [11, (Figure 10)] for ρ that can behave in two ways upon receiving an input (m, M) from the environment $(\mathcal{E}, 0)$: either as the dummy adversary or as a copy of an adversary $\mathcal{S}_\pi$ of ϕ such that $\text{EXEC}_{\pi, \mathcal{D}, \mathcal{E}} \approx \text{EXEC}_{\phi, \mathcal{S}_\pi, \mathcal{E}}$.

The compliance notion does not prohibit ρ from invoking ITMs with code π ; we consider such a protocol ρ . In this setting, an ITI M with body $(\pi, (s, p))$ appearing in an extended session of $\rho^{\phi \rightarrow \pi}$ can be classified into two types:

- Type 1: M was invoked as the result of a replacement $\phi \rightarrow \pi$,
- Type 2: M is not the result of a replacement.

This is illustrated in Figure 11, where M_1 with body $(\pi, (s_1, p))$ is of type 1, and M_2 with body $(\pi, (s_2, p))$ is of type 2.

Using the notations from this figure, observe that $(\mathcal{S}, \diamond)$ should act differently when $(\mathcal{E}, 0)$ sends an input “ $(m, (\pi, (s_1, p)))$ ” versus an input “ $(m, (\pi, (s_2, p)))$ ” to $(\mathcal{S}, \diamond)$. Indeed:

- When $(\mathcal{E}, 0)$ sends “ $(m, (\pi, (s_1, p)))$ ”, $\mathcal{S}$ should act as a copy of $\mathcal{S}_\pi$, because $(\pi, (s_1, p))$ exists in the execution of ρ but is replaced by $(\phi, (s_1, p))$ in the execution of $\rho^{\pi \rightarrow \phi}$ – namely, M_1 .
- When $(\mathcal{E}, 0)$ sends “ $(m, (\pi, (s_2, p)))$ ”, $\mathcal{S}$ should act as the dummy adversary, as this ITI also appears with the same body in the execution of $\rho^{\pi \rightarrow \phi}$ – namely, M_2 .

²⁹ Note that our simplifications of protocol execution (Section 4, page 19) reduce the complexity of this notion, particularly because: (1) $(\mathcal{E}, 0)$ always sets `reveal-ext-id`, and (2) all protocol ITIs set flag `forced-write`.

³⁰ Same as note 28 on page 31.

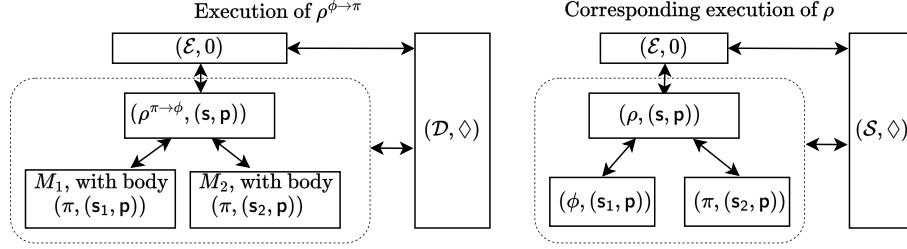


Fig. 11. Illustration for Weakness (W.3).

However, the algorithm's instruction [11, (Figure 10), item 1.(c).i] shows that $\mathcal{S}$ does not check whether $(\pi, (s, p))$ is of type 1 or 2, and always acts as a copy of $\mathcal{S}_\pi$.

To resolve this issue, $\mathcal{S}$ would need to distinguish the type of such ITIs. Yet we see no way for $\mathcal{S}$ to make this distinction based solely on the information available during execution.

To address this concern, the compliance update introduced in Section 6.6 ensures that no ITI in any extended session of $\rho^{\phi \rightarrow \pi}$ is of type 2.

(W.4) Compliance Property and Universal Composition Operator.

In Issue (I.3) (page 27), we have shown that the conditions required by the subroutine respecting notion should be verified not only in protocol executions but also in more general forms of execution.

A similar problem arises with the conditions of the compliance notion. We classify this as a weakness rather than an issue, as we have not provided a counterexample that exploits it.

Given the similarity of the two problems, one might logically attempt to address it in a similar manner—by requiring the conditions of the compliance property to hold not only for any protocol execution but also for any extended execution.

However, the conditions required by the subroutine respecting and compliance properties differ significantly, and directly applying this solution would be overly restrictive – making it nearly impossible for a protocol to be compliant.

To update the compliance notion without being overly restrictive, we need a more precise characterization of the set of extended execution that should be considered. To that end, Updated UC introduces a new kind of execution, called replacement execution (see Section 6.2).

6 Updated UC: An Update of Plain UC

This section introduces Updated UC, an update to the Plain UC framework that revises several core definitions to address the issues identified in earlier sections.

Each modification in Updated UC will be motivated by specific problems in Plain UC described in the preceding sections.

For clarity of exposition, the universal composition theorem is presented separately in Section 7.

Readers primarily interested in using Updated UC without reviewing its underlying motivations may refer to the concise usage guide in Appendix B, which summarizes the updated definitions and the revised UC theorem.

6.1 A New Notion of Extended Session

As shown in Limitation (L.1) (Section 5.1), the fact that an ITI rejects a message should not prevent it from belonging to an extended session. In Updated UC, we modify the definition of extended session to account for this.

First, we introduce the notion of a sleeping ITI: an ITI is considered sleeping if it does not exist, or if it has always rejected every message it has received. Note that this implies that, upon receiving a message, a sleeping ITI reacts as if it were handling its first message.

Then, when constructing the extended session of a protocol, we no longer focus solely on messages received by newly invoked ITIs, as in the original definition (i.e., ITIs that did not exist before receiving the message). Instead, we inspect each sleeping ITI that receives a message.

Definition 2 (Sleep). *An ITI M is sleeping in a prefix of execution θ of an extended system if one of the following holds:*

- M does not exist in θ .
- M has rejected every message it received – including the most recent one. That is, if the prefix ends with a message being delivered to M , we use the full configuration of M (as recorded in θ) to determine whether it rejects the message.

An ITI M is awake if it is not sleeping.

Definition 3 (Main parties and extended session). *Let α be an ITM and $s \in \{0, 1\}^*$ a sid. The main parties of $\text{Ext}(\phi, s)$ are defined as $\text{MainParty}(\alpha, s) = \{(\alpha, (s, p)) : p \in \{0, 1\}^*\}$.*

Given a prefix θ of extended execution, the extended session $\text{Ext}(\alpha, s, \theta)$ is defined as a set of ITIs initialized³¹ to $\text{MainParty}(\alpha, s)$, and constructed inductively by inspecting each input or output received by a sleeping ITI.

When a sleeping ITI M receives an input or output with sender an ITI N :

- *If the identifier of M is not of the form (s', p') for some $s', p' \in \{0, 1\}^*$, nothing is done. In the next items, we assume this condition holds.*
- *If $M \in \text{MainParty}(\alpha, s)$, then M already belongs to $\text{Ext}(\alpha, s, \theta)$.*
- *If it is an input and $N \in \text{Ext}(\alpha, s, \theta)$, then M is added to $\text{Ext}(\alpha, s, \theta)$.*
- *If it is an output and $N \in \text{Ext}(\alpha, s, \theta) \setminus \text{MainParty}(\alpha, s)$, then M is added to $\text{Ext}(\alpha, s, \theta)$.*

³¹ In the original definition, $\text{MainParty}(\alpha, s)$ is generally not a subset of $\text{Ext}(\alpha, s)$. It is a convenient convention.

– Otherwise, nothing is done.

We define $\text{ExtSub}(\alpha, s, \theta) = \text{Ext}(\alpha, s, \theta) \setminus \text{MainParty}(\alpha, s)$, and call the ITIs in this set the sub-parties.

When the context is clear, we omit θ and simply write $\text{Ext}(\alpha, s)$ and $\text{ExtSub}(\alpha, s)$.

6.2 Replacement Execution: A (Not Entirely) New Notion of Execution

Updated UC introduces a new notion of execution that depends on two protocols ϕ, π and a number $\ell \in \mathbb{N}$. Simply put, the (ϕ, ℓ, π) -execution of a protocol ρ corresponds to executing ρ while replacing the first ℓ calls to ϕ by calls to π , and informing the adversary of the sids of the replaced extended sessions.

Although variants of such executions already appear in the proof of the UC theorem [11, (Theorem 22)] within Plain UC, they were not used to define any protocol property. However, ignoring such executions leads to problems such as Issue (I.3). and Weakness (W.4). In Updated UC, the replacement executions will be used to update the compliance notion (Section 6.6) and define the notion of UC Operator (Section 6.7). Moreover, the proof of the UC Theorem made in Appendix C will extensively use them.

Replacement Execution of an ITM.

Definition 4 ((ϕ, ℓ, π) -execution). Let ρ, ϕ , and π be three ITMs, $\ell \in \mathbb{N} \cup \{\infty\}$, and let $\mathcal{A}, \mathcal{E}$ be two ITMs.

The (ϕ, ℓ, π) -execution of ρ with environment $\mathcal{E}$ and adversary $\mathcal{A}$ is defined as the execution of the extended system $(\mathcal{E}, \mathcal{C}_{(\phi, \ell, \pi)\text{-EXEC}}^{\rho, \mathcal{A}, \mathcal{E}})$, where the control function $\mathcal{C}_{(\phi, \ell, \pi)\text{-EXEC}}^{\rho, \mathcal{A}, \mathcal{E}}$ is defined similarly to $\mathcal{C}_{\text{EXEC}}^{\rho, \mathcal{A}, \mathcal{E}}$, with some differences listed below:

1. *Tracking the sids of replaced extended sessions:* The execution maintains a set $\text{ReplacedSession} \subseteq \{0, 1\}^*$ (initialized to $\emptyset$) that tracks the sids of replaced sessions – as detailed in next items. Its size is bounded by ℓ .
In the following items, we will denote an ITI as “outside” if it is neither the environment $(\mathcal{E}, 0)$ nor the adversary $(\mathcal{A}, \diamond)$, and if it does not belong to:
 - $\text{Ext}(\phi, s)$ for any $s \in \{0, 1\}^*$,
 - $\text{Ext}(\pi, s)$ for any $s \in \text{ReplacedSession}$.
2. *Replacement $\phi \rightarrow \pi$:* When an input or output is sent from an outside ITI to an ITI $(\phi, (s, p))$ for some $s, p \in \{0, 1\}^*$:
 - If $s \in \text{ReplacedSession}$ or if $|\text{ReplacedSession}| < \ell$, the message is instead sent to $(\pi, (s, p))$ and s is added to ReplacedSession (if not already present).
 - Otherwise, the message is not redirected and the execution continues.
3. *Replacement $\pi \rightarrow \phi$:* When an output is sent from an ITI $(\pi, (s, p))$ for some pid $p \in \{0, 1\}^*$ and sid $s \in \text{ReplacedSession}$, to an ITI $N \notin \text{Ext}(\pi, s)$, the sender ITI $(\pi, (s, p))$ is replaced by $(\phi, (s, p))$.
4. *Adversary learns ReplacedSession :* Whenever a message m is sent to the adversary, the control function replaces it with the pair $(\text{ReplacedSessions}, m)$. This ensures the adversary always knows the current value of ReplacedSessions .

We denote by $(\phi, \ell, \pi)\text{-EXEC}_{\rho, \mathcal{A}, \mathcal{E}}(z)$ the random variable representing the result of executing $(\mathcal{E}, \mathcal{C}_{(\phi, \ell, \pi)\text{-EXEC}}^{\rho, \mathcal{A}, \mathcal{E}})$ on input z .

Note that replacement execution is a generalization of the standard notion of protocol execution. Indeed, for any protocol ρ , a protocol execution of ρ corresponds to a $(\phi, 0, \pi)$ -execution of ρ . The only difference is that, in the replacement execution, any message received by the adversary is a pair whose first element is the set $\emptyset$.

Replacement Execution of a Parametrized System.

Let $\ell \in \mathbb{N} \cup \{\infty\}$, and let ρ, ϕ , and π be three parametrized systems, and $\mathcal{A}, \mathcal{E}$ two ITMs. The (ϕ, ℓ, π) -execution of ρ with adversary $\mathcal{A}$ and environment $\mathcal{E}$ is defined analogously, with the following differences:

- All inputs to $\mathcal{E}$ must be pairs (k, z) with $k \in \mathbb{N}$ and $z \in \{0, 1\}^*$. The value k is the security parameter.
- The adversary has read-only access to the security parameter.
- When the security parameter is k , the execution proceeds as a (ϕ_k, ℓ, π_k) -execution of ρ_k .

We denote the corresponding control function by $\mathcal{C}_{(\phi, \ell, \pi)\text{-EXEC}}^{\rho, \mathcal{A}, \mathcal{E}}$ and the result of the execution by $(\phi, \ell, \pi)\text{-EXEC}_{\rho, \mathcal{A}, \mathcal{E}}(k, z)$.

PPT Triplet of Parametrized Systems.

In Section 4, we defined the notion of a PPT protocol in the context of standard protocol execution. We now define an analogous notion for replacement executions.

A triplet (ρ, ϕ, π) of parametrized systems is said to be PPT if, for every PPT environment $\mathcal{E}$ and PPT adversary $\mathcal{A}$, there exists a polynomial R such that for any $\ell \in \mathbb{N}$ and any input $(k, z) \in \mathbb{N} \times \{0, 1\}^*$, any (ϕ, ℓ, π) -execution of ρ with environment $\mathcal{E}$ and adversary $\mathcal{A}$ completes in time bounded by $R(k + |z|)$.

Replacement Execution Prefixes.

Just as we defined prefixes of protocol executions (Section 4, page 20), we now define prefixes for replacement executions.

A (ρ, ϕ, ℓ, π) -execution prefix is a prefix of an (ϕ, ℓ, π) -execution of ρ . We refer to any such prefix as a *replacement execution prefix*.

6.3 Panoptic Session

To define our variants of the subroutine respecting and subroutine exposing properties, we introduce a new construction whose use will be central in avoiding several issues identified in the first part of the paper—primarily Issue (I.2), Limitation (L.2), and Weakness (W.2).

Definition 5 (Panoptic session). *Let α be an ITM, and let s be a sid. We define the panoptic session of α with sid s , as*

$$\text{Pan}(\alpha, s) = \bigcup_{\theta} \text{Ext}(\alpha, s, \theta),$$

where the union is taken over all α -execution prefixes with test sid $\mathbf{s}$.

We define the set of panoptic sub-parties as

$$\text{PanSub}(\alpha, \mathbf{s}) = \text{Pan}(\alpha, \mathbf{s}) \setminus \text{MainParty}(\alpha, \mathbf{s}).$$

Additionally, for an ITM α , we define

$$\text{Pan}(\alpha) = \{\beta \text{ ITM} : \exists \text{id}, \mathbf{s} \in \{0, 1\}^*, (\beta, \text{id}) \in \text{Pan}(\alpha, \mathbf{s})\}.$$

We note that computing a panoptic session is no more difficult than computing the extended sessions already required by Plain UC (for example, to verify the subroutine respecting property), as it is simply a union over such sessions.

We will also use this notion to define a restriction on the parametrized systems π and ϕ considered by the UC theorem. This restriction is reasonable, since in practical scenarios, ϕ is typically an ideal protocol that does not share sub-parties with π .

Definition 6 (Disjunction). *Two protocols π and ϕ are said to be disjoint if, for any sids $\mathbf{s}, \mathbf{s}' \in \{0, 1\}^*$, we have $\text{Pan}(\pi, \mathbf{s}) \cap \text{Pan}(\phi, \mathbf{s}') = \emptyset$. Two parametrized systems ϕ and π are said to be disjoint if, for all $k \in \mathbb{N}$, ϕ_k and π_k are disjoint.*

6.4 Sid-Exposing

To address the two issues discussed in Section 5.3 regarding the subroutine exposing property, we introduce a new concept called *sid-exposing*.

As a prerequisite for a protocol α to be sid-exposing, we require that all its panoptic sessions are disjoint, as formally defined below.

Definition 7 (Separation). *A protocol α is said to be separated if for any two distinct sids $\mathbf{s} \neq \mathbf{s}'$, $\text{Pan}(\alpha, \mathbf{s}) \cap \text{Pan}(\alpha, \mathbf{s}') = \emptyset$. A parametrized system α is separated if each α_k is separated for all $k \in \mathbb{N}$.*

Assuming that α is separated, we observe that any ITI M belongs to at most one set $\text{Pan}(\alpha, \mathbf{s})$ for some unique $\mathbf{s} \in \{0, 1\}^*$, or to none of them.

The sid-exposing property asks that we can efficiently determine, given an ITI M , the corresponding $\mathbf{s}$, if it exists.

Definition 8 (Sid-Exposing). *Let α be a separated protocol and $\mathbf{s}$ a sid. We say that α is sid-exposing if there exists a PPT Turing machine (TM) Expose^α , called the exposing TM of α , which takes as input an ITI M and returns the unique sid $\mathbf{s}$ such that $M \in \text{Pan}(\alpha, \mathbf{s})$, if such a sid exists, and $\perp$ otherwise.*

A separated parametrized system α is sid-exposing if there exists a PPT Turing machine PSEExpose^α whose inputs are pairs $(k, M) \in \mathbb{N} \times \text{SetITI}$, and such that $\text{PSEExpose}^\alpha(k, \cdot)$ is an exposing TM of α_k for each $k \in \mathbb{N}$.

We believe that this property is generally easy to verify, and we are not aware of any protocol in the literature that would fail to be sid-exposing.

This notion presents three main advantages over the subroutine exposing notion, effectively addressing the problems detailed in Section 5.3.

First, the exposing TM Expose^α is not associated with a specific sid. In contrast, in the subroutine exposing property, there is a distinct exposing ITI for each sid. This resolves Weakness (W.2).

Second, the notion is *non-interactive*: the adversary can simulate Expose^α locally without interacting with the protocol. This makes it more practical and avoids the need for the transformation discussed in (L.2).

Finally, this property reveals strictly less information to the adversary. Subroutine exposing allows the adversary to test whether an ITM M exists in the *current execution*, whereas sid-exposing only reveals whether M exists in *some execution* (possibly not the current one). A positive consequence is that the UC theorem in Updated UC can apply to concrete protocols such as π^* and ϕ^* discussed in the technical details (Section 2.4).

6.5 Sub-party Respecting Protocol

As shown in Section 5.1, the subroutine respecting definition lacks a condition about backdoors (I.1), should not account only for protocol executions (I.3), and allows dynamic sub-parties (I.2).

In this section, we introduce a more robust concept called “sub-party respecting” to address these three issues. To achieve this, the sub-party respecting notion incorporates the following key changes compared to the original subroutine respecting definition:

- It prevents Issue (I.1) by adding a condition about the reception of backdoors by sleeping ITIs.
- To address Issue (I.2), we replace extended sessions with panoptic sessions and add a requirement regarding the reception of outputs by sleeping ITIs. Together, these measures prevent protocols with dynamic sub-parties to be sub-party respecting.
- To address Issue (I.3), the conditions are evaluated on any extended execution, not just any protocol execution.³²

Definition 9 (Sub-party respecting). *A protocol α is said to be sub-party respecting if, for any prefix θ of any extended execution, and for each sid s , the following conditions hold:*

- spr.1 All sleeping ITIs $N \in \text{Pan}(\alpha, s)$ reject any backdoor.*
- spr.2 All sleeping ITIs $N \in \text{PanSub}(\alpha, s)$ reject any output whose sender extended identity is an ITI $M \in \text{MainParty}(\alpha, s)$.*

³² It would have been sufficient to evaluate the conditions for the “relevant prefixes”, defined and examined in Appendix C.3. However, we chose to evaluate the conditions for a broader set of prefixes to achieve a simpler overall definition, relegating the specific definition and examination of the relevant prefixes to the appendix.

- spr.3 All ITIs of $\text{PanSub}(\alpha, s)$ reject any input whose sender extended identity³³ is an ITI $M \notin \text{Pan}(\alpha, s)$.*
- spr.4 All ITIs of $\text{Pan}(\alpha, s)$ reject any output whose sender extended identity is an ITI $M \notin \text{Pan}(\alpha, s)$.*
- spr.5 No ITI of $\text{PanSub}(\alpha, s)$ sends an output to an ITI $M \notin \text{Pan}(\alpha, s)$.*
- spr.6 No ITI of $\text{Pan}(\alpha, s)$ sends an input to an ITI $M \notin \text{Pan}(\alpha, s)$.*

A parametrized system α is sub-party respecting if α_k is sub-party respecting for each $k \in \mathbb{N}$.

Note that it may seem abstract to require these conditions for any prefix of any extended execution. A more concrete and informal interpretation is that they must hold for any possible internal state and any possible message (including those forged with impersonated sender identities).

6.6 Compliance

In Plain UC, compliance [11, (Section 6.1)] is the only property that constrains the protocol ρ when applying the UC theorem to protocols ρ , π , and ϕ ³⁴. The same principle applies in Updated UC, which employs a slight variant of this notion, as specified in Definition 10.

Our variant introduces three changes. First, its conditions are not evaluated on any protocol execution, but rather on some replacement executions – this is motivated by Issue (I.3), Section 5.1. Second, to address Weakness (W.3) (also in Section 5.1), we forbid the presence of ITIs of Type 2, using the terminology introduced in that weakness. Third, we require that for each security parameter k , we have $\rho_k \notin \text{Pan}(\phi_k) \cup \text{Pan}(\pi_k)$. This is a reasonable assumption, since ρ typically represents a “larger protocol” that uses π or ϕ as subroutines.

Definition 10 (Compliance). Let π , ϕ , and ρ be protocols such that π and ϕ are separated and disjoint, and let ξ be a predicate on extended identities. The protocol ρ is said to be (π, ϕ, ξ) -compliant if, for every $\ell \in \mathbb{N}$ and every (ρ, ϕ, ℓ, π) -execution prefix θ , the following hold:

1. $\rho \notin \text{Pan}(\phi) \cup \text{Pan}(\pi)$.
2. There is no sid s such that an ITI $M \notin \text{Ext}(\pi, s, \theta)$ makes an external-write³⁵ targeting a recipient $N \in \text{MainParty}(\pi, s)$.
3. The extended identities of all ITIs M in θ that send inputs to ITIs with code ϕ satisfy the predicate ξ .³⁶

³³ Here, we specifically mean the sender’s extended identity as indicated at the reception of the message. It is important to note that there is no guarantee the message was actually sent by an ITI with this extended identity, as impersonations are possible.

³⁴ Except time-execution constraints.

³⁵ Here, we specifically refer to the external-write instruction issued by M , before any potential modification by the control function.

³⁶ Same as footnote 35.

If $\xi = \text{true}$, we simply say that ρ is (π, ϕ) -compliant.

Let π , ϕ , and ρ be parametrized systems. ρ is said to be (π, ϕ, ξ) -compliant if, for every $k \in \mathbb{N}$, the protocol ρ_k is (π_k, ϕ_k, ξ) -compliant.

6.7 Universal Composition Operator and Extension of UC-emulation

To address the concerns related to the universal composition operator (UC operator) in Plain UC (Section 5.2), we propose a general definition of the UC operator. This clarifies the concept and allows us to state the UC Theorem independently of any particular UC operator implementation, making it more general and modular. Since this definition relies on an extension of the UC-emulation relation, we first introduce this extension and then present the definition.

Additionally, we provide in Appendix E a new construction of UC operator that adheres to our definition and mitigates the issues identified with the UC operator of Plain UC. We will conclude this section by explaining the underlying idea behind its construction.

Extension of UC-emulation Relation. Using the definitions of Section 6.1, we observe that for any parametrized systems π, ϕ, ρ , ITMs $\mathcal{E}, \mathcal{A}$, and $k \in \mathbb{N}, z \in \{0, 1\}^*$, we can construct two binary probability distributions ensembles:

$$(\text{EXEC}_{\rho, \mathcal{A}, \mathcal{E}}(k, z))_{k \in \mathbb{N}, z \in \{0, 1\}^*} \quad \text{and} \quad ((\phi, \infty, \pi)\text{-EXEC}_{\rho, \mathcal{A}, \mathcal{E}}(k, z))_{k \in \mathbb{N}, z \in \{0, 1\}^*}.$$

We want to use them to define a generalization of UC-emulation, adapted to (ϕ, ∞, π) -executions. We want this extension to retain desirable properties of UC-emulation, notably transitivity, and the equivalence between UC-emulation and UC-emulation with respect to the dummy adversary [11, (Claim 11)].

To this end, we define $\mathcal{D}^{\text{DROP}}$, an adaptation of the dummy adversary $\mathcal{D}$ for (ϕ, ∞, π) -executions. When $\mathcal{D}^{\text{DROP}}$ receives a message $(\text{ReplacedSession}, m)$, it behaves exactly like $\mathcal{D}$ on the message m . $\mathcal{D}^{\text{DROP}}$ is said to be “dropping”, as defined below:

Definition 11 (Dropping Adversary). An adversary $\mathcal{A}$ is said to be “dropping” if, upon receiving a message of the form $(\text{ReplacedSession}, m)$ — with ReplacedSession a set of sids and m a message — it only uses m , thereby “dropping” ReplacedSession .

In our extension of UC-emulation, we restrict attention to dropping adversaries in (ϕ, ∞, π) -executions. Otherwise, a generalization of [11, (Claim 11)] would fail.

Definition 12 (Extension of UC-emulation). Let π, ϕ, ρ, ν be four PPT parametrized systems, and let ξ be a predicate on extended identities. Suppose (ρ, ϕ, π) is a PPT triplet.

We say that (ρ, ϕ, π) UC-emulates ν with respect to ξ -identity-bounded environments (or: (ρ, ϕ, π) ξ -UC-emulates ν) if for any dropping PPT adversary

$\mathcal{A}$, there exists a PPT adversary $\mathcal{S}$ such that for any PPT ξ -identity-bounded environment $\mathcal{E}$, we have (ϕ, ∞, π) -EXEC $_{\rho, \mathcal{A}, \mathcal{E}} \approx$ EXEC $_{\mathbf{v}, \mathcal{S}, \mathcal{E}}$.

We say that $\mathbf{v}$ UC-emulates (ρ, ϕ, π) with respect to ξ -identity-bounded environments (or: $\mathbf{v}$ ξ -UC-emulates (ρ, ϕ, π)) if for any PPT adversary $\mathcal{A}$, there exists a dropping PPT adversary $\mathcal{S}$ such that for any PPT ξ -identity-bounded environment $\mathcal{E}$, we have EXEC $_{\mathbf{v}, \mathcal{A}, \mathcal{E}} \approx (\phi, \infty, \pi)$ -EXEC $_{\rho, \mathcal{S}, \mathcal{E}}$.

The UC-emulation is said to be perfect if the two distribution ensembles are perfectly indistinguishable. If the relation only holds for a specific adversary $\mathcal{A}$, we say that UC-emulation is with respect to $\mathcal{A}$.

UC-emulation for a fixed predicate ξ is transitive. For example, if α ξ -UC-emulates (ρ, ϕ, π) and (ρ, ϕ, π) ξ -UC-emulates $\mathbf{v}$, then α ξ -UC-emulates $\mathbf{v}$.

We also generalize [11, (Claim 11)] to this setting:

Proposition 2 (UC-emulation with respect to $\mathcal{D}^{\text{DROP}}$ is equivalent to UC-emulation). *Let $\pi, \phi, \rho, \mathbf{v}$ be four PPT parametrized systems and ξ a predicate on extended identities. Suppose (ρ, ϕ, π) is a PPT triplet. Then, (ρ, ϕ, π) ξ -UC-emulates $\mathbf{v}$ if and only if (ρ, ϕ, π) ξ -UC-emulates $\mathbf{v}$ with respect to $\mathcal{D}^{\text{DROP}}$.*

Proof. Direct adaptation of the proof of [11, (Claim 11)].

Definition of UC Operator. We now define what we mean by a UC operator. Note that the conditions imposed on ρ , ϕ , and π correspond to those required in our UC theorem.

Definition 13 (UC Operator). *Let PPT-Param be the set of PPT parametrized systems. Let $S \subset \text{PPT-Param}^3$ and UC-OP : $S \rightarrow \text{PPT-Param}$ a function. We say that UC-OP is a S-Universal Composition Operator (or S-UC Operator) if, for any $(\rho, \phi, \pi) \in S$ such that:*

- π, ϕ are separated, sub-party respecting, sid-exposing, and disjoint,
- ρ is (π, ϕ) -compliant,
- (ρ, ϕ, π) is a PPT triplet,

Then, UC-OP (ρ, ϕ, π) perfectly UC-emulates (ρ, ϕ, π) .

Description of our UC Operator Implementation. As described in Issue (I.4) (page 28), the UC operator used in Plain UC suffers from flaws that lead to counterexamples to the UC theorem.³⁷ In Appendix E, we describe a revised UC operator that meets our definition. Here, we summarize the core concept.

In every extended session Ext $(\rho^{\phi \rightarrow \pi}, s_{\text{test}})$, we introduce an ITI Coord (s_{test}) , referred to as the coordinator. This ITI acts as an active middle-man within the session, observing nearly all messages and modifying them as needed to ensure correct replacements. In particular, it records enough information to avoid the problem described in Issue (I.4).

³⁷ An additional issue is discussed in Appendix F.1.

7 Universal Composition Theorem

This section states the UC theorem of Updated UC. It is both more general and more modular than the UC Theorem of Plain UC [11, (Theorem 22)], as it does not rely on a particular UC Operator implementation. Any choice of a UC operator (Definition 13) – including the one provided in Appendix E – immediately yields a concrete instantiation of the theorem, analogous to the UC Theorem in Plain UC.

A complete proof is provided in Appendix C. While its overall structure follows that of the original proof in Plain UC, it builds on the changes introduced in Updated UC to address the issues identified in this work, and is more detailed. In particular the proofs of Lemmas 2 and 3 represent significant new contributions. In the proof of the UC Theorem of Plain UC, the equivalents of these lemmas^{38,39} are asserted through the inspection of the involved ITMs. However, detailed demonstrations are not provided, which we believe substantially contributes to the counterexamples we identified.

Theorem 1 (Universal Composition Theorem). *Let π , ϕ , and ρ be three PPT parametrized systems, and let ξ be a predicate on extended identities. Assume that (ρ, ϕ, π) is a PPT triplet, and that ϕ and π are sub-party respecting, separated, disjoint, and sid-exposing. Further assume that ρ is (π, ϕ, ξ) -compliant and that π ξ -UC-emulates ϕ . Then, (ρ, ϕ, π) UC-emulates ρ .*

Corollary 1 (Universal Composition Theorem with a UC Operator). *Let $S \subset \text{PPT-Param}^3$ and let UC-OP be an S -UC operator. Let π , ϕ , and ρ be parametrized systems such that $(\rho, \phi, \pi) \in S$, and let ξ be a predicate on extended identities. Under the same assumptions as Theorem 1, UC-OP (ρ, ϕ, π) UC-emulates ρ .*

Proof. Immediate from transitivity of UC-emulation and Proposition 2.

In typical use cases of Updated UC, the user aims to show that a protocol π UC-emulates another protocol ϕ , and wishes to preserve this security under composition. To that end, it suffices to verify that π and ϕ are PPT, disjoint, separated, sid-exposing, and sub-party respecting. This ensures that the UC theorem applies to most natural choices of a PPT protocol ρ , while the compliance and PPT-triplet requirements act as sanity conditions to exclude problematic edge cases.

References

1. Backes, M., Hofheinz, D.: How to break and repair a universally composable signature functionality. In: Zhang, K., Zheng, Y. (eds.) ISC 2004: 7th International

³⁸ Lemma 2 corresponds to Equality [11, (Equation (7), page 68)].

³⁹ Lemma 3 corresponds to the equivalence of inequalities [11, (Equation (4), page 67)] and [11, (Equation (6), page 68)].

- Conference on Information Security. Lecture Notes in Computer Science, vol. 3225, pp. 61–72. Springer Berlin Heidelberg, Germany, Palo Alto, CA, USA (Sep 27–29, 2004). https://doi.org/10.1007/978-3-540-30144-8_6
2. Badertscher, C., Canetti, R., Hesse, J., Tackmann, B., Zikas, V.: Universal composition with global subroutines: Capturing global setup within plain UC. In: Pass, R., Pietrzak, K. (eds.) TCC 2020: 18th Theory of Cryptography Conference, Part III. Lecture Notes in Computer Science, vol. 12552, pp. 1–30. Springer, Cham, Switzerland, Durham, NC, USA (Nov 16–19, 2020). https://doi.org/10.1007/978-3-030-64381-2_1
 3. Badertscher, C., Maurer, U., Tschudi, D., Zikas, V.: Bitcoin as a transaction ledger: A composable treatment. *Journal of Cryptology* **37**(2), 18 (Apr 2024). <https://doi.org/10.1007/s00145-024-09493-7>
 4. Barbosa, M., Gellert, K., Hesse, J., Jarecki, S.: Bare PAKE: Universally composable key exchange from just passwords. In: Reyzin, L., Stebila, D. (eds.) *Advances in Cryptology – CRYPTO 2024*, Part II. Lecture Notes in Computer Science, vol. 14921, pp. 183–217. Springer, Cham, Switzerland, Santa Barbara, CA, USA (Aug 18–22, 2024). https://doi.org/10.1007/978-3-031-68379-4_6
 5. Blazy, O., Chevalier, C.: Generic construction of UC-secure oblivious transfer. In: Malkin, T., Kolesnikov, V., Lewko, A.B., Polychronakis, M. (eds.) *ACNS 15: 13th International Conference on Applied Cryptography and Network Security*. Lecture Notes in Computer Science, vol. 9092, pp. 65–86. Springer, Cham, Switzerland, New York, NY, USA (Jun 2–5, 2015). https://doi.org/10.1007/978-3-319-28166-7_4
 6. Camenisch, J., Drijvers, M., Tackmann, B.: Multi-protocol UC and its use for building modular and efficient protocols. *Cryptology ePrint Archive*, Report 2019/065 (2019), <https://eprint.iacr.org/2019/065>
 7. Camenisch, J., Krenn, S., Küsters, R., Rausch, D.: iUC: Flexible universal compositability made simple. In: Galbraith, S.D., Moriai, S. (eds.) *Advances in Cryptology – ASIACRYPT 2019*, Part III. Lecture Notes in Computer Science, vol. 11923, pp. 191–221. Springer, Cham, Switzerland, Kobe, Japan (Dec 8–12, 2019). https://doi.org/10.1007/978-3-030-34618-8_7
 8. Canetti, R.: Universally composable security: A new paradigm for cryptographic protocols. In: 42nd Annual Symposium on Foundations of Computer Science. pp. 136–145. IEEE Computer Society Press, Las Vegas, NV, USA (Oct 14–17, 2001). <https://doi.org/10.1109/SFCS.2001.959888>
 9. Canetti, R.: Universally composable signatures, certification and authentication. *Cryptology ePrint Archive*, Report 2003/239 (2003), <https://eprint.iacr.org/2003/239>
 10. Canetti, R.: Universally composable security: A new paradigm for cryptographic protocols. *Cryptology ePrint Archive*, Report 2000/067, version 20181231:043505 (2018), <https://eprint.iacr.org/archive/2000/067/20181231:043505>
 11. Canetti, R.: Universally composable security: A new paradigm for cryptographic protocols. *Cryptology ePrint Archive*, Report 2000/067. Last version : 20200212:021048 (2020), <https://eprint.iacr.org/archive/2000/067/20200212:021048>
 12. Canetti, R., Cohen, A., Lindell, Y.: A simpler variant of universally composable security for standard multiparty computation. In: Gennaro, R., Robshaw, M.J.B. (eds.) *Advances in Cryptology – CRYPTO 2015*, Part II. Lecture Notes in Computer Science, vol. 9216, pp. 3–22. Springer Berlin Heidelberg, Germany, Santa Barbara, CA, USA (Aug 16–20, 2015). https://doi.org/10.1007/978-3-662-48000-7_1

13. Canetti, R., Dodis, Y., Pass, R., Walfish, S.: Universally composable security with global setup. In: Vadhan, S.P. (ed.) TCC 2007: 4th Theory of Cryptography Conference. Lecture Notes in Computer Science, vol. 4392, pp. 61–85. Springer Berlin Heidelberg, Germany, Amsterdam, The Netherlands (Feb 21–24, 2007). https://doi.org/10.1007/978-3-540-70936-7_4
14. Canetti, R., Jain, P., Swanberg, M., Varia, M.: Universally composable end-to-end secure messaging. In: Dodis, Y., Shrimpton, T. (eds.) Advances in Cryptology – CRYPTO 2022, Part II. Lecture Notes in Computer Science, vol. 13508, pp. 3–33. Springer, Cham, Switzerland, Santa Barbara, CA, USA (Aug 15–18, 2022). https://doi.org/10.1007/978-3-031-15979-4_1
15. Canetti, R., Rabin, T.: Universal composition with joint state. In: Boneh, D. (ed.) Advances in Cryptology – CRYPTO 2003. Lecture Notes in Computer Science, vol. 2729, pp. 265–281. Springer Berlin Heidelberg, Germany, Santa Barbara, CA, USA (Aug 17–21, 2003). https://doi.org/10.1007/978-3-540-45146-4_16
16. Chen, M., Dey, P., Ganesh, C., Mukherjee, P., Sarkar, P., Sasmal, S.: Universally composable non-interactive zero-knowledge from sigma protocols via a new straight-line compiler (2025). <https://doi.org/10.1007/978-3-031-91820-9>
17. Cremers, C., Naor, M., Paz, S., Ronen, E.: CHIP and CRISP: Protecting all parties against compromise through identity-binding PAKEs. In: Dodis, Y., Shrimpton, T. (eds.) Advances in Cryptology – CRYPTO 2022, Part II. Lecture Notes in Computer Science, vol. 13508, pp. 668–698. Springer, Cham, Switzerland, Santa Barbara, CA, USA (Aug 15–18, 2022). https://doi.org/10.1007/978-3-031-15979-4_23
18. Davies, G.T., Faller, S.H., Gellert, K., Handirk, T., Hesse, J., Horváth, M., Jager, T.: Security analysis of the WhatsApp end-to-end encrypted backup protocol. In: Handschuh, H., Lysyanskaya, A. (eds.) Advances in Cryptology – CRYPTO 2023, Part IV. Lecture Notes in Computer Science, vol. 14084, pp. 330–361. Springer, Cham, Switzerland, Santa Barbara, CA, USA (Aug 20–24, 2023). https://doi.org/10.1007/978-3-031-38551-3_11
19. Hesse, J.: Separating symmetric and asymmetric password-authenticated key exchange. In: Galdi, C., Kolesnikov, V. (eds.) SCN 20: 12th International Conference on Security in Communication Networks. Lecture Notes in Computer Science, vol. 12238, pp. 579–599. Springer, Cham, Switzerland, Amalfi, Italy (Sep 14–16, 2020). https://doi.org/10.1007/978-3-030-57990-6_29
20. Hofheinz, D., Shoup, V.: GNUC: A new universal composability framework. *Journal of Cryptology* **28**(3), 423–508 (Jul 2015). <https://doi.org/10.1007/s00145-013-9160-y>
21. Hofheinz, D., Unruh, D., Müller-Quade, J.: Polynomial runtime and composability. *Cryptology ePrint Archive*, Report 2009/023 (2009), <https://eprint.iacr.org/2009/023>
22. Jarecki, S., Krawczyk, H., Xu, J.: OPAQUE: An asymmetric PAKE protocol secure against pre-computation attacks. In: Nielsen, J.B., Rijmen, V. (eds.) Advances in Cryptology – EUROCRYPT 2018, Part III. Lecture Notes in Computer Science, vol. 10822, pp. 456–486. Springer, Cham, Switzerland, Tel Aviv, Israel (Apr 29 – May 3, 2018). https://doi.org/10.1007/978-3-319-78372-7_15
23. Küsters, R., Tuengerthal, M., Rausch, D.: The IITM model: A simple and expressive model for universal composability. *Journal of Cryptology* **33**(4), 1461–1584 (Oct 2020). <https://doi.org/10.1007/s00145-020-09352-1>
24. Maurer, U.: Constructive cryptography – a new paradigm for security definitions and proofs. In: Mödersheim, S., Palamidessi, C. (eds.) *Theory of Security and Applications*. pp. 33–56. Springer Berlin Heidelberg, Berlin, Heidelberg (2012)

25. Rausch, D., Küsters, R., Chevalier, C.: Embedding the UC model into the IITM model. In: Dunkelman, O., Dziembowski, S. (eds.) *Advances in Cryptology – EU-ROCRYPT 2022, Part II*. Lecture Notes in Computer Science, vol. 13276, pp. 242–272. Springer, Cham, Switzerland, Trondheim, Norway (May 30 – Jun 3, 2022). https://doi.org/10.1007/978-3-031-07085-3_9

A Quick Guide to Appendices

In addition to this guide, the appendices contain five sections that can be read in any order. They remain independent except for shared conventions and notations:

- Notations and conventions necessary for Appendices C and E are detailed in Section C.1.
- For Appendices D and F, relevant notations and conventions are provided in Section D.1.

Let’s briefly explain each appendix:

- Appendix B provides a short and practical guide for users of UC interested in applying the updated definitions and UC Theorem.
- Appendix C gives a detailed proof of the UC Theorem. A detailed description of its subsections is given at the beginning of the appendix.
- Appendix D contains complete counterexamples and proofs for Issues (I.2), (I.3), and (I.4) discussed in the paper.
- Appendix E details a construction of a UC operator that satisfies Definition 13.
- Appendix F presents further discussion on subtleties of Plain UC and additional issues, limitations, and weaknesses. In particular, it explains the rationale for avoiding predicates to designate receivers, using an alternative time-complexity notion, and omitting the balanced environment concept. A more detailed presentation is given at the beginning of the appendix.

Finally, the complete table of contents appears at the end of the document.

B User-guide to updated UC

This short section is intended for users who wish to apply the updated UC definitions to prove the security of a protocol π .

Let $\pi = (\pi_k)_{k \in \mathbb{N}}$ be the protocol under analysis, and let $\phi = (\phi_k)_{k \in \mathbb{N}}$ be the corresponding ideal protocol. The goal is to show that π ξ -UC-emulates ϕ for some predicate ξ on extended identities, and to ensure that this security is preserved under composition.

B.1 Proving UC-emulation

When demonstrating that π UC-emulates ϕ , or when checking that an already established UC-emulation from Plain UC remains valid under Updated UC, the user must account for the following changes:

- Our notion of a parametrized system has been revised (see Section 4, page 20).
- The definition of protocol executions differs slightly from Plain UC; these differences are indicated in the footnotes to the “Protocol Execution” paragraph (Section 4, page 19). In particular, protocol ITIs (distinct from the environment and adversary) now always set the flags **forced-write** and **reveal-ext-id**, and thus may invoke the adversary; likewise, the environment always sets **reveal-ext-id** – while retaining the power to replace its extended identity by impersonation.
- We no longer require environments to be balanced [11, Section 4.2] in the definition of UC-emulation.
- We adopt a simpler notion of time complexity (informally introduced in Section 4 and formally defined in Section C.2), since time complexity is not the main focus here and because the Plain UC notion has known issues (see Appendix F.5). Although more general notions could be integrated into Updated UC, we leave this for future work.

B.2 Ensuring Preservation of UC-Emulation Under Composition by Reasonable Protocols

In this part, we detail the conditions that ϕ and π must satisfy to ensure that, for every reasonable PPT protocol ρ , if ϕ ξ -UC-emulates π , then $\rho^{\phi \rightarrow \pi}$ also ξ -UC-emulates ϕ .

The user must first verify that ϕ and π are PPT (Definition 16). Next, for each security parameter $k \in \mathbb{N}$ and each session identifier $s \in \{0, 1\}^*$, they compute the panoptic sessions $\text{Pan}(\pi_k, s)$ and $\text{Pan}(\phi_k, s)$, as per Definition 5. These play the same role in Updated UC as extended sessions played in Plain UC.

As an example, consider the ideal protocol $\text{IDEAL}_{\mathcal{F}}$ [11, (Section 5.3)] for an ideal functionality $\mathcal{F}$ that does not invoke other ITMs (Figure 12). For any sid $s \in \{0, 1\}^*$, we have:

$$\text{Pan}(\text{IDEAL}_{\mathcal{F}}, s) = \{(\text{IDEAL}_{\mathcal{F}}, (s, p)) : p \in \{0, 1\}^*\} \cup \{(\mathcal{F}, (s, \epsilon))\}.$$

The user then checks that, for all $k \in \mathbb{N}$, the following hold:

$$\begin{aligned} \forall s \neq s' \in \{0, 1\}^*, \text{Pan}(\phi_k, s) \cap \text{Pan}(\phi_k, s') &= \emptyset, \\ \forall s \neq s' \in \{0, 1\}^*, \text{Pan}(\phi_k, s) \cap \text{Pan}(\pi_k, s') &= \emptyset, \\ \forall s, \tilde{s} \in \{0, 1\}^*, \text{Pan}(\phi_k, s) \cap \text{Pan}(\phi_k, \tilde{s}) &= \emptyset, \end{aligned}$$

to ensure that π and ϕ are separated (Definition 7) and disjoint (Definition 6).

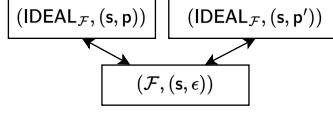


Fig. 12. Illustration of an ideal protocol whose ideal functionality does not invoke other ITMs.

Next, they verify that π and ϕ are sid-exposing (Definition 8), which amounts to showing that the function

$$M \mapsto \begin{cases} s & \text{if there exist a sid } s \text{ such that } M \in \text{Pan}(\phi_k, s), \\ \perp & \text{otherwise} \end{cases}$$

is efficiently computable.

Finally, one must ensure that ϕ and π are sub-party respecting (Definition 6.5).

B.3 Ensuring Preservation of UC-Emulation Under Composition by a Specific Protocol ρ

To guarantee that $\rho^{\phi \rightarrow \pi}$ ξ -UC-emulates a specific PPT protocol $\rho = (\rho_k)_{k \in \mathbb{N}}$, the user must additionally check that:

- (ρ, ϕ, π) forms a triplet of PPT parametrized systems (Definition 17);
- ρ is (π, ϕ, ξ) -compliant (Definition 10).

Both of these rely on the notion of replacement execution (Section 6.2).

C Complete Proof of UC Theorem of Updated UC (Theorem 1)

The aim of this appendix is to provide the complete proof of Theorem 1. The initial appendices introduce necessary notations and conventions (C.1), present more formal definitions of time-complexity (C.2), and establish preliminary results regarding relevant execution prefixes (C.3). The final appendices (C.4, C.5, C.6, C.7 and C.8) are fully dedicated to proving the theorem.

As discussed in Section 7, this proof follows the steps of the original UC theorem for plain UC, leveraging the adjustments in Updated UC to address previously identified issues.

Consequently, most steps in this proof will resemble those of [11, (Theorem 22)]. However, we emphasize that, besides the changes introduced by our new definitions, we add significant new contributions.

First, our proof is more modular. It's not only independent of a specific choice of UC-operator but also structured in a modular way. The theorem is proved

using a central lemma (Appendix C.4), which itself relies on two additional lemmas (C.5), respectively proven in Appendices C.7 and C.8. This breakdown of the proof simplifies its analysis.

Secondly, the proofs of Lemmas 2 and 3 represent significant new contributions. In the proof of [11, (Theorem 22)], the equivalents of these lemmas^{40,41} are asserted through the inspection of the involved ITMs. However, detailed demonstrations are not provided, which we believe substantially contributes to the counterexamples we identified concerning the plain UC theorem.

Third, we made some necessary changes beyond those required by the new definitions of Updated UC. Notably, we implemented a significant modification of the simulator, justified by the weaknesses (W.5) described in Appendix F.3.

C.1 Additional Notations and Conventions

Additional Notations for ITIs.

Let SetITI denote the set of all ITIs.

For an ITI M , we denote its code by $\text{code}(M)$ and its identifier by $\text{id}(M)$. If the identifier is of the form $(s, p) \in \{0, 1\}^* \times \{0, 1\}^*$, we write $\text{sid}(M)$ for its sid and $\text{pid}(M)$ for its pid.

For any external-write w , we denote its message by $\text{mess}(w)$ and its destination tape by $\text{tape}(w)$.

Additional Conventions and Instructions for Algorithms.

LOCAL FUNCTIONS. Each local function has access to the local variables of the main function and may modify them.

SENDING MESSAGES. The instructions **apply** and **send** both indicate that the ITI performs an external-write instruction w . Specifically, **apply** appears as “**apply** w ”, while **send** is used in the form “**send** ‘ m ’ to tape t of ITI N with flags (...)”. If the context permits, flags may be omitted. When the tape t is known, the shorthand notations **sendinput**, **sendoutput**, or **sendbackdoor** are used.

DEACTIVATION, REJECT, AND HALT. The instruction **deactivate** deactivates the ITI, **reject** causes it to reject, and **halt** places it in a halt state. Upon entering the halt state, the ITI deactivates and rejects any subsequent messages.

PARSING. The instruction “**checkorchalt** m is (...)” checks whether the message m matches the specified pattern. If it does, execution continues; otherwise, the ITI halts. This instruction is typically used to parse m as a tuple and halt if parsing fails. The variant **checkorreject** performs the same check, but rejects instead of halting.

SIMULATING. The instruction “**initialize** a simulation of U ” means that the ITI M creates all necessary internal state to simulate the ITI U . After each step of

⁴⁰ Lemma 2 corresponds to Equality [11, (Equation (7), page 68)].

⁴¹ Lemma 3 corresponds to the equivalence of inequalities [11, (Equation (4), page 67)] and [11, (Equation (6), page 68)].

simulated execution, M saves U 's updated state and tape contents so that the simulation may resume on future activations. The simulation may involve:

- Simulating the application of external-write instructions to U via “**simulate**”.
- Simulating the deactivation or rejection of U via “**simulate**”.
- Reacting to external-write instructions or deactivations produced by the simulation of U .

ENDING EXECUTION. In environment algorithms, for $b \in \{0, 1\}$, the instruction “**return** b ” causes the environment $\mathcal{E}$ to write b to its outgoing message tape and halt. This marks the termination of the execution with result b . We may refer to this result as the “result of $(\mathcal{E}, 0)$ ”.

DEFAULT BEHAVIOR. If an ITM's algorithm does not specify behavior for certain events (e.g., it defines reactions to inputs but not to outputs or backdoors), the ITM is assumed to reject by default.

Use of Flags. Unless explicitly stated otherwise, the following conventions apply:

- Any message exchanged between protocol ITIs (that is, ITIs other than the adversary or the environment) has both forced-write and reveal-ext-id set.
- Any backdoor sent by the adversary has both forced-write and reveal-ext-id unset.
- Any input sent by the environment has both forced-write and reveal-ext-id set.

Use of Autoreferences. For clarity, we permit the code of an ITI to reference its own code, even though this is formally disallowed.

This convention is especially useful when analyzing scenarios involving two ITIs (ψ, id) and (α, id') that exchange messages in both directions:

- (ψ, id) sends a message to (α, id') ,
- and (α, id') responds by sending a message to (ψ, id) .

This mutual reference implies that the code of ψ includes that of α , which itself includes ψ , forming an undefined circular dependency.

To resolve this, we use the identity tape feature introduced in [11, (Section 3.1.1)]. The protocol ψ is modified as follows:

- All occurrences of ψ in its own code are replaced by a special sequence **seq**.
- At runtime, the ITI replaces **seq** with the code read from its identity tape.

C.2 Detailed Definitions of Time Complexity

Definition 14 (PPT environment). Let $\mathcal{E}$ an ITM. For $(k, z) \in \mathbb{N} \times \{0, 1\}^*$, $B \in \mathbb{N}^*$ and a finite sequence of external-write operations $W = (w_i)_{i=1}^B$ with recipient $(\mathcal{E}, 0)$, we define the (k, z, W) -execution of $\mathcal{E}$ as follows:

- It begins by executing $(\mathcal{E}, 0)$ with input (k, z) .
- If $\mathcal{E}$ halts, the (k, z, W) -execution ends.
- If $\mathcal{E}$ deactivates (due to self-deactivation or to an external-write operation), the next external operation of $(w_i)_{i=1}^B$ is given to $\mathcal{E}$. If there is no next external operation, the (k, z, W) -execution ends.

We say that $\mathcal{E}$ is a PPT environment if there exists a polynomial P such that, for any $(k, z) \in \mathbb{N} \times \{0, 1\}^*$, $B \in \mathbb{N}$ and any finite sequence of external-write operations $W = (w_i)_{i=1}^B$ with recipient $(\mathcal{E}, 0)$, the (k, z, W) -execution of $\mathcal{E}$ has an execution time bounded by $P(k + |z|)$.

Definition 15 (PPT adversary). Let $\mathcal{A}$ an ITM. For $k \in \mathbb{N}, B \in \mathbb{N}^*$ and a finite sequence of external-write operations $W = (w_i)_{i=1}^B$ with recipient $(\mathcal{A}, \diamond)$, we define the (k, W) -execution of $\mathcal{A}$ as follows:

- It begins by executing $(\mathcal{A}, \diamond)$ with external-write operation w_1 and writing k in the security parameter tape of $\mathcal{A}$.
- When $\mathcal{A}$ deactivates (due to self-deactivation or to an external-write operation), the next external operation of $(w_i)_{i=1}^B$ is given to $\mathcal{E}$. If there is no next external operation, the (k, W) -execution ends.

We say that $\mathcal{A}$ is a PPT adversary if there exists a polynomial P such that, for any $k \in \mathbb{N}$, $B \in \mathbb{N}^*$ and any finite sequence of external-write operations $W = (w_i)_{i=1}^B$ with recipient $(\mathcal{A}, \diamond)$, the (k, W) -execution of $\mathcal{A}$ has an execution time bounded by $\sum_{i=1}^B Q(k + |w_i|)$.

Definition 16 (PPT protocol). A parametrized system ρ is a PPT protocol if for any PPT environment $\mathcal{E}$ and PPT adversary $\mathcal{A}$, there is a polynomial $R = R(\mathcal{E}, \mathcal{A})$ such that for any $(k, z) \in \mathbb{N} \times \{0, 1\}^*$: any execution of ρ with adversary $\mathcal{A}$, environment $\mathcal{E}$ and input (k, z) has an execution time bounded by $R(k + |z|)$.

Definition 17 (Triplet of PPT parametrized systems). (ρ, ϕ, π) is a triplet of PPT parametrized systems if for any PPT environment $\mathcal{E}$ and PPT adversary $\mathcal{A}$, there is a polynomial $R = R(\mathcal{E}, \mathcal{A})$ such that for any $(k, z) \in \mathbb{N} \times \{0, 1\}^*$ and $l \in \mathbb{N}$: any (ϕ, l, π) -execution of ρ with adversary $\mathcal{A}$, environment $\mathcal{E}$ and input (k, z) has an execution time bounded by $R(k + |z|)$.

C.3 Relevant Prefixes

In the definition of sub-party respecting protocols (Definition 9), we formulate several conditions that must hold for every prefix of any extended execution.

However, when proving the UC Theorem (Theorem 7), only a small subset of these prefixes is actually involved. We refer to them as “relevant”, and we define them precisely in Definition 18.

We next highlight three useful properties of relevant prefixes that play a crucial role in the proof of the UC Theorem: Propositions 3, 4, and 5.

More precisely:

- Proposition 3 essentially states that, in a α -relevant prefix θ and under appropriate hypotheses, any awake ITI $M \in \text{Pan}(\alpha, s)$ belongs to $\text{Ext}(\alpha, s, \theta)$. This result may be seen as a weak form of inclusion of the panoptic session in the extended session.

- Proposition 4 explains that, in a α -relevant prefix θ and under appropriate hypotheses, the external-writes exchanged between ITIs in $\text{Pan}(\alpha, s)$ are not subject to the additional rules of replacement execution (*i.e.*, replacements of the form $\alpha \rightarrow \beta$ or $\beta \rightarrow \alpha$). In other words, when we restrict our attention to the ITIs in $\text{Pan}(\alpha, s)$, their behavior appears as in a standard protocol execution, even if the prefix belongs to a replacement execution.
- Proposition 5 states that, in a α -relevant prefix θ and under the right assumptions, we have $\text{Ext}(\alpha, s, \theta) \subset \text{Pan}(\alpha, s)$.

Definition 18 (Relevant execution prefix). *Let α a protocol. An α -relevant execution prefix refer to either:*

- *A ρ -execution prefix for $\rho \notin \text{Pan}(\alpha)$.*
- *A $(\rho, \alpha, \ell, \beta)$ -execution prefix or $(\rho, \beta, \ell, \alpha)$ -execution prefix for a separated ITM β disjoint with α , $\ell \in \mathbb{N}$ and $\rho \notin \text{Pan}(\alpha) \cup \text{Pan}(\beta)$.*

Proposition 3 (Awake ITIs of the Panoptic and Extended Sessions). *Let α a separated and sub-party respecting protocol. For any α -relevant execution prefix θ and any sid $s \in \{0, 1\}^*$, there is no ITI $M \in \text{Pan}(\alpha, s)$ which is awake on θ but does not belong to $\text{Ext}(\alpha, s, \theta)$.*

Proof. Suppose, for contradiction, that there exists an awake ITI in θ that belongs to $\text{Pan}(\alpha, s)$ but not to $\text{Ext}(\alpha, s, \theta)$. Let M be the first such ITI, and let w be the external-write operation whose sender identity is N and which wakes M .

By definition of panoptic and extended sessions,

$$\text{MainParty}(\alpha, s) \subset \text{Pan}(\alpha, s) \cap \text{Ext}(\alpha, s, \theta),$$

so $M \in \text{PanSub}(\alpha, s)$. We now prove three facts about w :

- fact.1 The message of w is not a backdoor.
- fact.2 If the message of w is an input, then $N \in \text{Pan}(\alpha, s)$.
- fact.3 If the message of w is an output, then $N \in \text{PanSub}(\alpha, s)$.

By Condition spr.1 of sub-party respecting (Definition 9, page 38), M would reject any backdoor, so fact fact.1 holds. Moreover:

- If w is an input, then by Condition spr.3 and the fact that M did not reject, we have $N \in \text{Pan}(\alpha, s)$.
- If w is an output, then by Condition spr.4 and the fact that M did not reject, we have $N \in \text{Pan}(\alpha, s)$. Moreover, using Condition spr.2, we obtain $N \in \text{PanSub}(\alpha, s)$.

Although the sender identity indicated in w is N , it is possible that N was not the actual originator of the external-write. Indeed, w may have been modified by the control function.

We now divide the analysis into separate cases, depending on how w could have been generated.

Case 1: N actually issued the external-write.

Assume that w was genuinely sent by N . Let θ' be the sub-prefix of θ that ends just before N sends the message. By the minimality of M , no ITI

$U \in \text{Pan}(\alpha, s)$ is awake in θ' without also belonging to $\text{Ext}(\alpha, s, \theta')$. Since $N \in \text{Pan}(\alpha, s)$ and is awake in θ' , we must have

$$N \in \text{Ext}(\alpha, s, \theta') \subset \text{Ext}(\alpha, s, \theta).$$

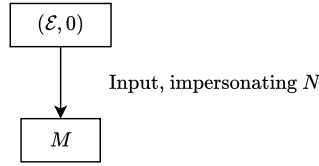
Now, recalling that the message of w is not a backdoor (Fact **fact.1**), we distinguish two subcases:

- If the message is an input, then the fact that $N \in \text{Ext}(\alpha, s, \theta)$ implies, by the definition of extended sessions, that $M \in \text{Ext}(\alpha, s, \theta)$.
- If the message is an output, then from Fact **fact.3**, we know that $N \in \text{ExtSub}(\alpha, s, \theta)$, and hence, again by the definition of extended sessions, $M \in \text{Ext}(\alpha, s, \theta)$.

In both cases, this contradicts the assumption that $M \notin \text{Ext}(\alpha, s, \theta)$. This completes the argument for this case.

Case 2: The environment impersonated N .

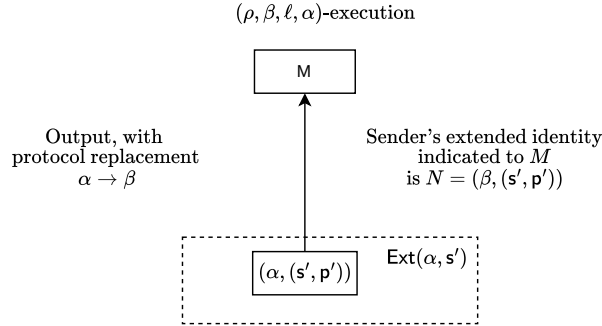
We now consider the possibility that w was issued by the environment while impersonating N . This situation is illustrated below:



However, in any α -relevant prefix, we have $\rho \notin \text{Pan}(\alpha)$, where ρ is the executed protocol. As a result, $M \in \text{PanSub}(\alpha)$ cannot be a main party of ρ , and thus the environment cannot send an input to M . This contradicts the assumption that M was awakened by w .

Case 3: N appears due to a replacement $\alpha \rightarrow \beta$.

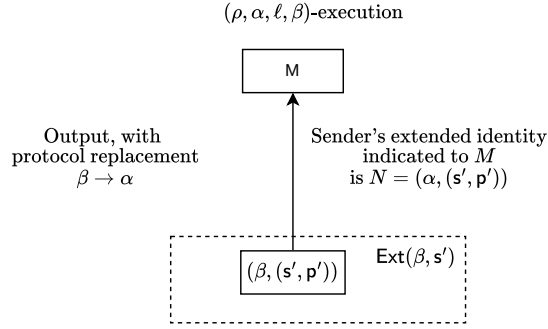
We now consider the case where θ is a $(\rho, \beta, \ell, \alpha)$ -execution prefix, for some ITM β that is separated and disjoint from α . The sender identity N might then appear due to a replacement $\alpha \rightarrow \beta$. This situation is depicted below:



Such a replacement would imply that $N \in \text{MainParty}(\beta, s')$ for some s' . However, from Fact fact.3, we know that $N \in \text{PanSub}(\alpha, s)$, which contradicts the disjointness of α and β . Hence, this case cannot occur.

Case 4: N appears due to a replacement $\beta \rightarrow \alpha$.

Finally, suppose that θ is a $(\rho, \alpha, \ell, \beta)$ -execution prefix, where β is an ITM separated and disjoint from α . The sender identity N might appear as a result of a replacement $\beta \rightarrow \alpha$. This case is illustrated here:



In this situation, we would have $N \in \text{MainParty}(\alpha, s')$ for some s' . However, from Fact fact.3, we know that $N \in \text{PanSub}(\alpha, s)$, which contradicts the separation property of α . This case is thus also impossible.

In all possible cases, we reach a contradiction. Therefore, no such ITI M can exist, which completes the proof.

Proposition 4 (No Replacements between ITIs of the Panoptic Session). *Let θ be a $(\rho, \alpha, \ell, \beta)$ -execution prefix or a $(\rho, \beta, \ell, \alpha)$ -execution prefix, for a separated ITM β disjoint from α , an integer $\ell \in \mathbb{N}$, and a protocol ρ such that $\rho \notin \text{Pan}(\alpha) \cup \text{Pan}(\beta)$.*

Then, in the prefix θ , for any session identifier $s \in \{0, 1\}^$ and any $U, V \in \text{Pan}(\alpha, s)$, the following holds:*

- *No external-write issued by U with indicated receiver V is modified by the control function due to a protocol replacement $\alpha \rightarrow \beta$ or $\beta \rightarrow \alpha$.*
- *No external-write received by V with indicated sender U has been modified by the control function due to a protocol replacement $\alpha \rightarrow \beta$ or $\beta \rightarrow \alpha$.*

Proof. Fix an arbitrary session identifier $s \in \{0, 1\}^*$. We assume that θ is a $(\rho, \alpha, \ell, \beta)$ -execution prefix, where β is an ITM that is separated and disjoint from α , $\ell \in \mathbb{N}^*$, and $\rho \notin \text{Pan}(\alpha) \cup \text{Pan}(\beta)$. The proof for the case where θ is a $(\rho, \beta, \ell, \alpha)$ -execution prefix is analogous.

Proof of the first item.

Assume, for contradiction, that there exist $U, V \in \text{Pan}(\alpha, s)$ and an external-write issued by U to V that is modified by the control function due to a replacement $\alpha \rightarrow \beta$ or $\beta \rightarrow \alpha$.

Since θ is a $(\rho, \alpha, \ell, \beta)$ -execution prefix, such a modification can only occur if there exists a session identifier s' such that:

1. $U \notin \text{Ext}(\alpha, s', \theta)$,
2. the message is an input,
3. and $V \in \text{MainParty}(\alpha, s')$.

However, since $V \in \text{MainParty}(\alpha, s')$ and $V \in \text{Pan}(\alpha, s)$, and because α is separated, this implies $s = s'$.

Now, since $U \in \text{Pan}(\alpha, s)$ and U is awake, Proposition 3 implies that $U \in \text{Ext}(\alpha, s, \theta)$. But this contradicts the assumption that $U \notin \text{Ext}(\alpha, s', \theta)$ (with $s' = s$).

Proof of the second item.

Assume, again for contradiction, that there exist $U, V \in \text{Pan}(\alpha, s)$ and an external-write received by V , with sender indicated as U , which has been modified by the control function due to a replacement $\alpha \rightarrow \beta$ or $\beta \rightarrow \alpha$.

As before, since θ is a $(\rho, \alpha, \ell, \beta)$ -execution prefix, this type of modification can only occur if there exists some session identifier s' such that:

1. $U \in \text{MainParty}(\beta, s')$,
2. the message is an output,
3. and $V \notin \text{Ext}(\alpha, s', \theta)$.

However, this leads to a contradiction because we assumed $U \in \text{Pan}(\alpha, s)$, and α and β are disjoint. Hence, U cannot simultaneously belong to $\text{MainParty}(\beta, s')$.

In both cases, we derive contradictions. Therefore, no such replacement can occur between ITIs in $\text{Pan}(\alpha, s)$.

Proposition 5 (Extended Session is Included in the Panoptic Session).

Let α be a separated, sid-exposing, and sub-party respecting protocol. Then, for any α -relevant execution prefix θ and any $s \in \{0, 1\}^$: $\text{Ext}(\alpha, s, \theta) \subset \text{Pan}(\alpha, s)$.*

Proof. Fix an α -relevant execution prefix θ . Assume that θ is a $(\rho, \alpha, \ell, \beta)$ -execution prefix, where β is an ITM that is separated and disjoint from α , $\ell \in \mathbb{N}^*$, and $\rho \notin \text{Pan}(\alpha) \cup \text{Pan}(\beta)$. The proof is analogous in other cases.

The main difficulty of this proof lies in the mismatch between θ , which is a replacement execution prefix of some protocol $\rho \neq \alpha$, and the definition of $\text{Pan}(\alpha, s)$, which considers only α -execution prefixes (i.e., executions of α without replacements).

To overcome this, we proceed in two steps: we first show that θ is "close enough" to an α -execution prefix, then use this to prove the inclusion.

Step 1: Construction and analysis of $\mathcal{E}_{\text{sim}}$.

We define an environment $\mathcal{E}_{\text{sim}}$ (given in Figure 14) whose role is to simulate the entire execution of θ —including $(\mathcal{E}, 0)$ and $(\mathcal{A}, \diamond)$ —except for the ITIs belonging to $\text{Pan}(\alpha, s)$.⁴²

Let θ_{sim} be the prefix of the execution of α with environment $\mathcal{E}_{\text{sim}}$, dummy adversary $\mathcal{D}$, and test sid s . We claim that θ_{sim} "simulates" θ .

⁴² This is achieved using Expose^α , hence the requirement that α be sid-exposing.

This setup is illustrated in Figure 13. Each ITI in θ is either a real ITI in $\text{Ext}(\alpha, s, \theta_{\text{sim}})$ or simulated by $(\mathcal{E}_{\text{sim}}, 0)$. By construction, $\mathcal{E}_{\text{sim}}$ only simulates ITIs that are not in $\text{Pan}(\alpha, s)$.

Ideally, this simulation would be perfect: every message exchanged between ITIs in θ would correspond to a real or simulated message in θ_{sim} . However, this is not quite the case. There are two primary discrepancies:

- The dummy adversary $\mathcal{D}$ in θ_{sim} has no counterpart in θ .
- Not all interactions in θ are simulated exactly in θ_{sim} . For instance, in θ , an ITI $M \notin \text{Pan}(\alpha, s)$ might send an input to some $N \in \text{PanSub}(\alpha, s)$, whereas in θ_{sim} , the simulation of M simply deactivates instead.

Despite these differences, the simulation is “almost perfect”. The dummy adversary $\mathcal{D}$ does not pose a problem: it merely relays communications between ITIs of the test session and the simulated adversary $(\mathcal{A}, \diamond)$, ensuring faithful simulation.

The second issue is more subtle. Differences only arise in five specific scenarios, which we analyze below:

- **Input from outside the panoptic session:** If $M \notin \text{Pan}(\alpha, s)$ sends an input to $N \in \text{PanSub}(\alpha, s)$ in θ , then the simulation in θ_{sim} deactivates M instead of sending the input. This is harmless because N would reject the input anyway due to Condition **spr.3** of sub-party respecting.
- **Output to the panoptic session:** If $M \notin \text{Pan}(\alpha, s)$ sends an output to $N \in \text{Pan}(\alpha, s)$ in θ , the simulation again deactivates M . This causes no issue because, by Condition **spr.4**, N would reject the message.
- **Unspecified simulation of outputs to outside:** The definition of $\mathcal{E}_{\text{sim}}$ does not specify how to simulate outputs from $M \in \text{PanSub}(\alpha, s)$ to $N \notin \text{Pan}(\alpha, s)$. This case, however, cannot occur due to Condition **spr.5**.
- **Unspecified simulation of inputs to outside:** Likewise, the simulation does not specify how to handle inputs from $M \in \text{Pan}(\alpha, s)$ to $N \notin \text{Pan}(\alpha, s)$, which is also ruled out by Condition **spr.5**.
- **Control-function-induced sender/receiver changes:** If a message exchanged between M and N with $M, N \in \text{Ext}(\alpha, s, \theta)$ were modified by the control function due to a protocol replacement $\alpha \rightarrow \beta$ or $\beta \rightarrow \alpha$, such a modification would not occur in θ_{sim} , as θ_{sim} is an execution without replacement—thus breaking the simulation. However, this situation cannot arise thanks to Proposition 4 and the fact that $\text{Ext}(\alpha, s, \theta_{\text{sim}}) \subset \text{Pan}(\alpha, s)$.

In summary, the only differences between θ and θ_{sim} are:

- bad.1 If $M \notin \text{Pan}(\alpha, s)$ sends an input to $N \in \text{PanSub}(\alpha, s)$, N rejects it in θ , and M is deactivated in θ_{sim} .
- bad.2 If $M \notin \text{Pan}(\alpha, s)$ sends an output to $N \in \text{Pan}(\alpha, s)$, N rejects it in θ , and M is deactivated in θ_{sim} .

Finally, since θ_{sim} is an α -execution prefix, we have by definition:

$$\text{Ext}(\alpha, s, \theta_{\text{sim}}) \subset \text{Pan}(\alpha, s).$$

Step 2: Proof of the proposition.

Let $u = |\text{ExtSub}(\alpha, \mathbf{s}, \theta)|$. For each $0 \leq n \leq u$, define the property:

$$\mathcal{P}(n) : \text{ For any sub-prefix } \theta' \text{ of } \theta \text{ such that } |\text{Ext}(\alpha, \mathbf{s}, \theta')| = n : \\ \text{Ext}(\alpha, \mathbf{s}, \theta') \subset \text{Pan}(\alpha, \mathbf{s}) .$$

Our goal is to prove $\mathcal{P}(u)$ by induction on n .

Base case: $\mathcal{P}(0)$.

If $|\text{ExtSub}(\alpha, \mathbf{s}, \theta')| = 0$, then by definition,

$$\text{Ext}(\alpha, \mathbf{s}, \theta') = \text{MainParty}(\alpha, \mathbf{s}) \subset \text{Pan}(\alpha, \mathbf{s}).$$

Inductive step: Assume $\mathcal{P}(n-1)$ holds for $0 < n \leq u$, and let us prove $\mathcal{P}(n)$.

Suppose $|\text{ExtSub}(\alpha, \mathbf{s}, \theta)| = n$. Let N be the last ITI added to $\text{ExtSub}(\alpha, \mathbf{s}, \theta)$. Let $\hat{\theta} \subset \theta$ be a sub-prefix such that

$$\text{ExtSub}(\alpha, \mathbf{s}, \hat{\theta}) = \text{ExtSub}(\alpha, \mathbf{s}, \theta) \setminus \{N\}.$$

By definition of Ext , in θ , N was sleeping and became active upon receiving a message (input or output) from some ITI $M \in \text{Ext}(\alpha, \mathbf{s}, \theta)$.

By the inductive hypothesis:

- If $M \in \text{MainParty}(\alpha, \mathbf{s})$, then clearly $M \in \text{Pan}(\alpha, \mathbf{s})$.
- If $M \in \text{ExtSub}(\alpha, \mathbf{s}, \theta)$, then $M \in \text{ExtSub}(\alpha, \mathbf{s}, \hat{\theta})$, and since $\mathcal{P}(n-1)$ holds, we have $M \in \text{Pan}(\alpha, \mathbf{s})$.

Hence, the message that activated N was sent by an ITI $M \in \text{Pan}(\alpha, \mathbf{s})$. Therefore, it cannot fall under the exceptional cases **bad.1** or **bad.2**, since these require $M \notin \text{Pan}(\alpha, \mathbf{s})$. This means the message corresponds to a real message in θ_{sim} , and thus both M and N lie in $\text{Ext}(\alpha, \mathbf{s}, \theta_{\text{sim}})$.

Since $\text{Ext}(\alpha, \mathbf{s}, \theta_{\text{sim}}) \subset \text{Pan}(\alpha, \mathbf{s})$, we conclude that $N \in \text{Pan}(\alpha, \mathbf{s})$.

This completes the inductive step, and hence the proof.

C.4 Proof of the Theorem Relying on the Use of a Central Lemma

This section and the following sections (C.5, C.6, C.7, and C.8) are entirely dedicated to proving the UC Theorem of Updated UC.

More precisely:

- This section restates and proves the theorem with the help of a central lemma (Lemma 1).
- Section C.5 proves the central lemma using two auxiliary lemmas (Lemmas 2 and 3).
- Section C.6 defines an environment $\mathcal{E}_\phi$ and a polynomial Q utilized to prove the two auxiliary lemmas.
- Section C.7 proves the first auxiliary lemma.
- Section C.8 proves the second auxiliary lemma.

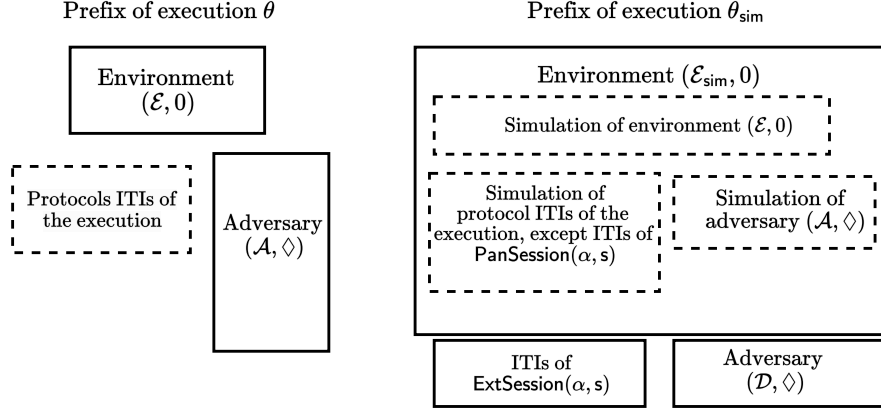


Fig. 13. Illustration of θ and θ_{sim} .

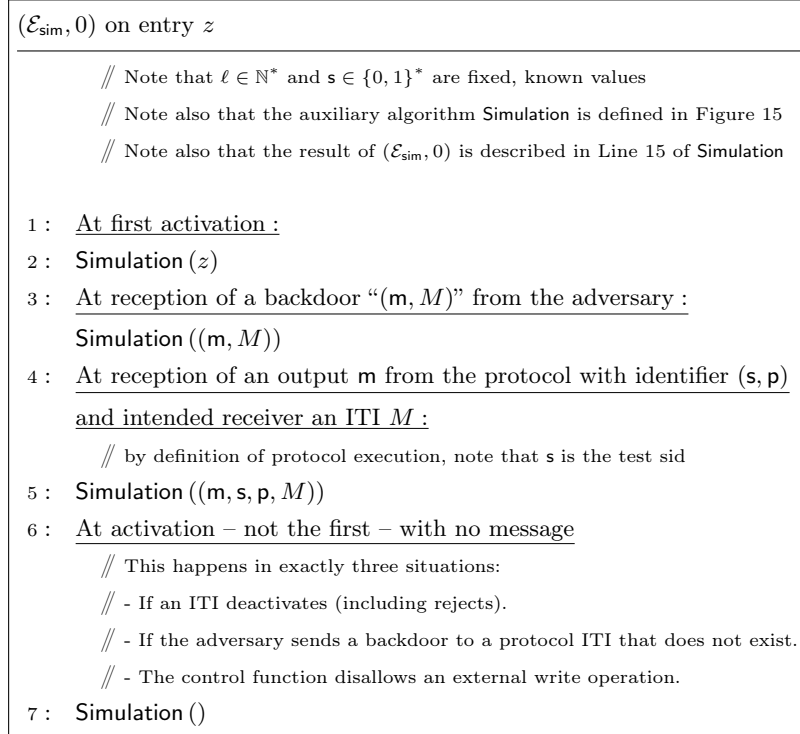


Fig. 14. Main algorithm used in the definition of $\mathcal{E}_{\text{sim}}$, to prove Proposition 5. It relies on the algorithm **Simulation** defined in Figure 15.

```

Simulation(input)
1 :    // Note that  $\ell \in \mathbb{N}^*$  and  $\mathbf{s} \in \{0, 1\}^*$  are fixed, known values
    if input can be parsed as  $z \in \{0, 1\}^*$  then
2 :     initialize the simulation of the  $(\alpha, \ell, \beta)$ -execution of  $\rho$ 
        with environment  $\mathcal{E}$  and adversary  $\mathcal{A}$  on input  $z$ 
3 :     Continue the simulation until one of the exceptions after line 14 occurs
4 : else if input can be parsed as a pair  $(\mathbf{m}, M) \in \{0, 1\}^* \times \text{SetITI}$  then
5 :     simulate the sending of backdoor  $\mathbf{m}$  to the simulation of
         $(\mathcal{A}, \diamond)$  as if it came from  $M$ .
6 :     Continue the simulation until one of the exceptions after line 14 occurs
7 : else if input can be parsed as  $(\mathbf{m}, \mathbf{s}, \mathbf{p}, M) \in \{0, 1\}^* \times \{0, 1\}^* \times \text{SetITI}$  then
8 :     simulate the sending of output  $\mathbf{m}$  to the simulation of  $M$  as if
        it came from  $(\alpha, (\mathbf{s}, \mathbf{p}))$ 
9 :     Continue the simulation until one of the exceptions after line 14 occurs
10 : else if Simulation has been called with no input then
11 :     simulate the activation of  $(\mathcal{E}, 0)$  with no message and continue
12 :         the simulation until one of the exceptions after line 14 occurs
13 : else halt
14 : List of exceptions:
15 : exception : simulation of  $(\mathcal{E}, 0)$  returns a bit  $b \in \{0, 1\}$  then return  $b$ 
16 : exception : an input or output “ $\mathbf{m}$ ” is sent by a simulated protocol ITI  $M$ 
        to a protocol ITI  $N$  that is a main party of  $(\alpha, \mathbf{s})$  then
        if it is an input then
17 :         send input “ $\mathbf{m}$ ” to the protocol with same sid and pid than  $N$ 
18 :         if it is an output then
19 :             simulate the deactivation of  $M$  and continue the simulation
20 :                 until one of the exceptions after line 14 occurs
21 : exception : an input or output “ $\mathbf{m}$ ” is sent by a simulated protocol ITI  $M$ 
        to a protocol ITI  $N$  with code  $\neq \alpha$ 
        such that  $\text{Expose}^\alpha(N) = \mathbf{s}$  then
22 :     simulate the deactivation of  $M$  and continue the simulation
23 :         until one of the exceptions after line 14 occurs
24 : exception : the simulation of  $(\mathcal{A}, \diamond)$  intends to send backdoor  $m$ 
25 :         to a protocol ITI  $N$  such that  $\text{Expose}^\alpha(N) = \mathbf{s}$  then
26 :         send input “ $(m, N)$ ” to the adversary

```

Fig. 15. Auxiliary algorithm used in the definition of $\mathcal{E}_{\text{sim}}$, to prove Proposition 5.

We now restate the theorem and begin the proof.

Restatement of Theorem 1 (Universal Composition Theorem)

Let π , ϕ and ρ three PPT parametrized systems and ξ a predicate on extended identities. Assume that (ρ, ϕ, π) is a PPT triplet of parametrized systems, and that ϕ, π are sub-party respecting, separated, disjoint and sid-exposing. Furthermore, suppose that ρ is (π, ϕ, ξ) -compliant and that π ξ -UC-emulates ϕ . Then, (ρ, ϕ, π) UC-emulates ρ .

Using UC-emulation with respect to the dropping dummy adversary $\mathcal{D}^{\text{DROP}}$, we are looking for a PPT adversary $\mathcal{S}_\rho$ such that, for any PPT environment $\mathcal{E}$:

$$(\phi, \infty, \pi)\text{-EXEC}_{\rho, \mathcal{D}^{\text{DROP}}, \mathcal{E}} \approx \text{EXEC}_{\rho, \mathcal{S}_\rho, \mathcal{E}}. \quad (1)$$

The fact that π ξ -UC-emulates ϕ guarantees that there exists a PPT adversary $\mathcal{S}_\phi$ of ϕ , such that for any ξ -identity-bounded, PPT environment $\mathcal{E}$:

$$\text{EXEC}_{\pi, \mathcal{D}, \mathcal{E}} \approx \text{EXEC}_{\phi, \mathcal{S}_\phi, \mathcal{E}}. \quad (2)$$

We construct a PPT adversary $\mathcal{S}_\rho$ out of $\mathcal{S}_\phi$ and then show it satisfies Equation 1.

Definition of the Adversary $\mathcal{S}_\rho$. The complete code of the adversary $\mathcal{S}_\rho$ is provided in Figure 16, and its behavior is described in this paragraph.

The adversary $\mathcal{S}_\rho$ simulates multiple variants of $\mathcal{S}_\phi$, each slightly modified. For each $s \in \{0, 1\}^*$, the adversary $\mathcal{S}_{\phi, s}$ behaves identically to $\mathcal{S}_\phi$, with one exception: when $\mathcal{S}_\phi$ attempts to send a backdoor to an ITI $M \notin \text{Pan}(\phi, s)$, $\mathcal{S}_{\phi, s}$ deactivates instead of sending the message. This modification is introduced to avoid undesirable side effects.⁴³ The full code of $\mathcal{S}_{\phi, s}$ is given in Figure 17.

The behavior of $\mathcal{S}_\rho$, represented in Figure 18, can be summarized as follows:

- Upon receiving an input (m, M) from the environment, $\mathcal{S}_\rho$ uses the sid-exposing property of both π and ϕ in order to compute $\text{positionM}_\pi = \text{PSExpose}^\pi(k, M)$, and $\text{positionM}_\phi = \text{PSExpose}^\phi(k, M)$. It then proceeds as follows:
 - If $\text{positionM}_\pi \in \{0, 1\}^*$, then $\mathcal{S}_\rho$ uses the simulation of $\mathcal{S}_{\phi, \text{positionM}}$ to process the input (m, M) .
 - Else, if $\text{positionM}_\phi \in \{0, 1\}^*$, then $\mathcal{S}_\rho$ rejects.⁴⁴
 - Otherwise, if $\text{positionM}_\phi = \perp$, then $\mathcal{S}_\rho$ acts as the dummy adversary and sends backdoor m to M .

⁴³ In the proof of the UC Theorem of Plain UC [11, (Theorem 22)], the simulator directly simulates copies of $\mathcal{S}_\phi$. However, our proof requires this adjustment. Although we do not describe a specific vulnerability in plain UC, we believe this change is also necessary in that context.

⁴⁴ This case does not appear in the simulator used in the proof of the UC Theorem of Plain UC [11, (Theorem 22)]. Its omission is the source of Weakness (W.5), as described in Section F.3, for readers familiar with the proof of the plain UC Theorem.

```

( $\mathcal{S}_\rho, \Diamond$ )
// ( $\mathcal{S}_\rho, \Diamond$ ) will simulate multiple modified adversaries ( $\mathcal{S}_{\phi,s}, \Diamond$ )
// whose code is given in Figure 17
1 : At first activation, ( $\mathcal{S}_\rho, \Diamond$ ) reads the security parameter  $k$ 
   indicated in its dedicated tape, then acts as indicated
2 : At reception of an external-write  $w$  that is a backdoor  $m$  from an ITI  $M$  :
3 :  $\text{positionM}_\phi \in \{\perp\} \cup \{0, 1\}^* \leftarrow \text{PSExpose}^\phi(k, M)$ 
4 : if  $\text{positionM}_\phi \in \{0, 1\}^*$  then  $\text{Simulate}(w, \text{positionM}_\phi)$ 
5 : if  $\text{positionM}_\phi = \perp$  then send backdoor “( $m, M$ )” to the environment
6 : At reception of an external-write  $w$  that is an input (from the environment) :
7 : checkorchalt the message can be parsed as a pair  $(m, M)$  with
    $m$  a message and  $M$  an ITI
8 :  $\text{positionM}_\pi \in \{\perp\} \cup \{0, 1\}^* \leftarrow \text{PSExpose}^\pi(k, M)$ 
9 :  $\text{positionM}_\phi \in \{\perp\} \cup \{0, 1\}^* \leftarrow \text{PSExpose}^\phi(k, M)$ 
10 : if  $\text{positionM}_\pi \in \{0, 1\}^*$  then  $\text{Simulate}(w, \text{positionM})$ 
11 : else if  $\text{positionM}_\phi \in \{0, 1\}^*$  then reject
12 : else then send backdoor “ $m$ ” to  $M$ 

Simulate( $w, s \in \{0, 1\}^*$ )
1 : checkorreject  $w$  is an external-write
2 : if ( $\mathcal{S}_{\phi,s}, \Diamond$ ) has never been simulated then
3 :   initialize its simulation (with security parameter  $k$ )
4 :  $\tilde{w} \leftarrow w$  with recipient replaced by ( $\mathcal{S}_{\phi,s}, \Diamond$ )
5 : simulate applying  $\tilde{w}$  to the simulation of ( $\mathcal{S}_{\phi,s}, \Diamond$ )
6 : Then simulation of ( $\mathcal{S}_{\phi,s}, \Diamond$ ) continues until one of the following events occurs :
   if the simulation of ( $\mathcal{S}_{\phi,s}, \Diamond$ ) rejects then reject
7 : if the simulation of ( $\mathcal{S}_{\phi,s}, \Diamond$ ) deactivates (but does not reject) then deactivate
8 : if the simulation of ( $\mathcal{S}_{\phi,s}, \Diamond$ ) intends to send a backdoor then
9 :   send this backdoor

```

Fig. 16. Code of adversary $\mathcal{S}_\rho$. It uses the adversary $\mathcal{S}_{\phi,s}$ defined in Figure 17.

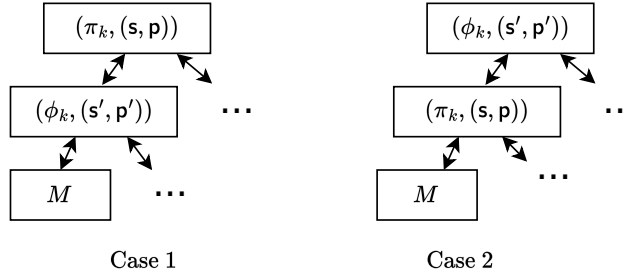
$(\mathcal{S}_{\phi,s}, \diamond)$
$\parallel (\mathcal{S}_{\phi,s}, \diamond)$ will simulate the adversary $(\mathcal{S}_{\phi}, \diamond)$ 1 : <u>At first activation, with reception of an external-write w</u> 2 : $(\mathcal{S}_{\phi,s}, \diamond)$ reads the security parameter k indicated in its dedicated tape. 3 : initialize a simulation of the adversary $(\mathcal{S}_{\phi}, \diamond)$ with security parameter k 4 : SimulateOperation (w) 5 : <u>At reception of an external-write w :</u> 6 : SimulateOperation (w) SimulateOperation (w) 1 : checkorhalt w is an external-write 2 : $\tilde{w} \leftarrow w$ with recipient replaced by $(\mathcal{S}_{\phi}, \diamond)$ 3 : simulate applying $\tilde{w}$ to the simulation of $(\mathcal{S}_{\phi}, \diamond)$ 4 : <u>Then simulation of $(\mathcal{S}_{\phi}, \diamond)$ continues until one of the following events occurs :</u> 5 : if the simulation of $(\mathcal{S}_{\phi,s}, \diamond)$ rejects then reject 6 : if the simulation of $(\mathcal{S}_{\phi,s}, \diamond)$ deactivates (but does not reject) then deactivate 7 : if the simulation of $(\mathcal{S}_{\phi}, \diamond)$ intends to send a backdoor to the environment then 8 : send this backdoor to the environment 9 : if the simulation of $(\mathcal{S}_{\phi}, \diamond)$ intends to send a backdoor to an ITI N then 10 : $\text{positionN} \in \{\perp\} \cup \{0, 1\}^* \leftarrow \text{PSExpose}^{\phi}(k, N)$ 11 : if $\text{positionN} \neq s$ then deactivate 12 : else send this backdoor to N

Fig. 17. Code of the adversary $\mathcal{S}_{\phi,s}$ for $s \in \{0, 1\}^*$.

- Upon receiving a backdoor m from some ITI M , $\mathcal{S}_\rho$ uses the sid-exposing property of ϕ to compute $\text{positionM}_\phi = \text{PSExpose}^\phi(k, M)$. It then proceeds as follows:
 - If $\text{positionM}_\phi \in \{0, 1\}^*$, then $\mathcal{S}_\rho$ uses the simulation of $\mathcal{S}_{\phi, \text{positionM}}$ to process the backdoor m received from M .
 - Otherwise, $\mathcal{S}_\rho$ acts as the dummy adversary and sends the backdoor (m, M) to the environment.

The adversary $\mathcal{S}_\rho$ is PPT because $\mathcal{S}_\phi$ is PPT, and both Expose^π and Expose^ϕ are PPT Turing machines.

We end this description by noting that disjointness property is essential to avoid ambiguous situations. For instance, if an ITI M lies in $\text{Pan}(\pi_k, s) \cap \text{Pan}(\phi_k, s')$ – as may be detected via the sid-exposing property of π and ϕ – then it would be impossible to decide whether we are in the first or second case below, each requiring distinct treatments.



Proof of Equation (1) by Contraposition, Using a Central Lemma To show that $\mathcal{S}_\rho$ satisfies Equation (1), we proceed by contraposition.

Suppose that there exists a PPT environment $\mathcal{E}_\rho$ such that Equation (1) is false. By definition, this means: $\exists c, d \in \mathbb{N}$ such that $\forall k_0 \in \mathbb{N}, \exists k > k_0, \exists z \in \bigcup_{0 < a \leq k^d} \{0, 1\}^a$:

$$\left| \Pr[(\phi, \infty, \pi)\text{-EXEC}_{\rho, \mathcal{D}^{\text{DROP}}, \mathcal{E}_\rho}(k, z) = 1] - \Pr[\text{EXEC}_{\rho, \mathcal{S}_\rho, \mathcal{E}_\rho}(k, z) = 1] \right| \geq k^{-c}. \quad (3)$$

We will use this environment $\mathcal{E}_\rho$ to construct a ξ -identity-bounded PPT environment $\mathcal{E}_\phi$ for which Equation (2) also fails.

More precisely, we will show that $\mathcal{E}_\phi$ satisfies the following central lemma:

Lemma 1 (Central Lemma for UC Theorem). *There is a polynomial Q increasing on $[0, \infty[$, with values in $]0, \infty[$, such that, for any $(k, z) \in \mathbb{N} \times$*

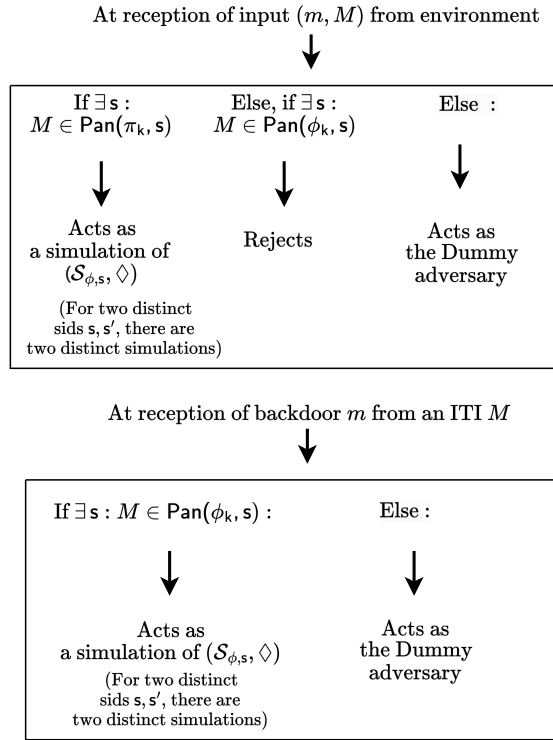


Fig. 18. Simplified representation of the adversary $(\mathcal{S}_\rho, \diamond)$ for security parameter k .

$\bigcup_{0 < a \leq k^d} \{0, 1\}^a$:

$$\begin{aligned} & \left| \Pr[\text{EXEC}_{\pi, \mathcal{D}, \mathcal{E}_\phi}(k, z) = 1] - \Pr[\text{EXEC}_{\phi, \mathcal{S}_\phi, \mathcal{E}_\phi}(k, z) = 1] \right| \\ &= \frac{1}{Q(k)} \left| \Pr[(\phi, \infty, \pi)\text{-EXEC}_{\rho, \mathcal{D}^{\text{DROP}}, \mathcal{E}_\rho}(k, z) = 1] - \Pr[\text{EXEC}_{\rho, \mathcal{S}_\rho, \mathcal{E}_\rho}(k, z) = 1] \right|. \end{aligned}$$

Using Lemma 1 and Equation (3), we can conclude that Equation (2) is false. Indeed, they imply that $\forall k_0, \exists k > k_0, \exists z \in \bigcup_{0 < a \leq k^d} \{0, 1\}^a$ such that:

$$\left| \Pr[\text{EXEC}_{\pi, \mathcal{D}, \mathcal{E}_\phi}(k, z) = 1] - \Pr[\text{EXEC}_{\phi, \mathcal{S}_\phi, \mathcal{E}_\phi}(k, z) = 1] \right| \geq \frac{k^{-c}}{Q(k)}.$$

Which completes the proof of UC Theorem.

As a consequence, to complete the proof of the theorem, it remains to construct the environment $\mathcal{E}_\phi$ and prove Lemma 1.

This will be carried out in the following steps:

- Section C.5: Proof of Lemma 1, using two auxiliary lemmas.
- Section C.6: Definition of the environment $\mathcal{E}_\phi$ and the polynomial Q .
- Section C.7: Proof of the first auxiliary lemma (Lemma 2).
- Section C.8: Proof of the second auxiliary lemma (Lemma 3).

C.5 Proof of the Central Lemma (Lemma 1) Using two Auxiliary Lemmas

In this section, we assume the existence of a PPT ξ -identity environment $\mathcal{E}_\phi$, along with a polynomial Q that is increasing on $[0, \infty[$ and takes values in $[2, \infty[$. We also assume that for each $\ell \in \mathbb{N}$, there exists an environment $\mathcal{E}_\phi^\ell$ such that the first operation of $\mathcal{E}_\phi$ on any input (k, z) is to uniformly sample an integer $\ell \in \{0, \dots, Q(k) - 1\}$ and then to act exactly as the environment $\mathcal{E}_\phi^\ell$ with input (k, z) until the end of the execution.

Furthermore, we assume that $\mathcal{E}_\phi$ and Q satisfy the following two auxiliary lemmas, which will be proven later.

Lemma 2 (First Auxiliary Lemma). *Let d the constant described in Section C.4, whose existence follows from the assumption that Equation (1) is false. For $(k, z) \in \mathbb{N} \times \bigcup_{0 < a \leq k^d} \{0, 1\}^a$ and $\ell \in \{0, \dots, Q(k) - 2\}$:*

$$\text{EXEC}_{\pi, \mathcal{D}, \mathcal{E}_\phi^\ell}(k, z) = \text{EXEC}_{\phi, \mathcal{S}_\phi, \mathcal{E}_\phi^{\ell+1}}(k, z).$$

Proof. See Appendix C.7.

Lemma 3 (Second Auxiliary Lemma). *Let d the constant described in Section C.4, whose existence follows from the assumption that Equation (1) is false. For $(k, z) \in \mathbb{N} \times \bigcup_{0 < a \leq k^d} \{0, 1\}^a$:*

$$\begin{aligned} \text{EXEC}_{\pi, \mathcal{D}, \mathcal{E}_\phi^{Q(k)-1}}(k, z) &= (\phi, \infty, \pi)\text{-EXEC}_{\rho, \mathcal{D}^{\text{DROP}}, \mathcal{E}_\rho}(k, z), \\ \text{EXEC}_{\phi, \mathcal{S}_\phi, \mathcal{E}_\phi^0}(k, z) &= \text{EXEC}_{\rho, \mathcal{S}_\rho, \mathcal{E}_\rho}(k, z). \end{aligned}$$

Proof. See Appendix C.8.

With these assumptions, we can prove the central Lemma.

Restatement of Lemma 1 (Central Lemma for UC Theorem) *There is a polynomial Q increasing on $[0, \infty[$, with values in $[2, \infty[$, such that, for any $(k, z) \in \mathbb{N} \times \bigcup_{0 < a \leq k^d} \{0, 1\}^a$:*

$$\begin{aligned} & \left| \Pr[\text{EXEC}_{\pi, \mathcal{D}, \mathcal{E}_\phi}(k, z) = 1] - \Pr[\text{EXEC}_{\phi, \mathcal{S}_\phi, \mathcal{E}_\phi}(k, z) = 1] \right| \\ &= \frac{1}{Q(k)} \left| \Pr[(\phi, \infty, \pi)\text{-EXEC}_{\rho, \mathcal{D}^{\text{DROP}}, \mathcal{E}_\rho}(k, z) = 1] - \Pr[\text{EXEC}_{\rho, \mathcal{S}_\rho, \mathcal{E}_\rho}(k, z) = 1] \right|. \end{aligned}$$

Proof. We set $(k, z) \in \mathbb{N} \times \bigcup_{0 < a \leq k^d} \{0, 1\}^a$. For $0 \leq \ell \leq Q(k) - 1$, we denote by E^ℓ the event “ $\mathcal{E}_\phi$ sampled ℓ ”.

We can see that for any $0 \leq \ell \leq Q(k) - 1$, $\Pr[E^\ell] = 1/Q(k)$. Moreover, we have:

$$\begin{aligned} \Pr[\text{EXEC}_{\pi, \mathcal{D}, \mathcal{E}_\phi}(k, z) = 1 \mid E^\ell] &= \Pr[\text{EXEC}_{\pi, \mathcal{D}, \mathcal{E}_\phi^\ell}(k, z) = 1], \\ \Pr[\text{EXEC}_{\phi, \mathcal{S}_\phi, \mathcal{E}_\phi}(k, z) = 1 \mid E^\ell] &= \Pr[\text{EXEC}_{\phi, \mathcal{S}_\phi, \mathcal{E}_\phi^\ell}(k, z) = 1]. \end{aligned} \quad (4)$$

We can thus compute:

$$\begin{aligned} & \left| \Pr[\text{EXEC}_{\pi, \mathcal{D}, \mathcal{E}_\phi}(k, z) = 1] - \Pr[\text{EXEC}_{\phi, \mathcal{S}_\phi, \mathcal{E}_\phi}(k, z) = 1] \right| \\ &= \frac{1}{Q(k)} \left| \sum_{\ell=0}^{Q(k)-1} \Pr[\text{EXEC}_{\pi, \mathcal{D}, \mathcal{E}_\phi}(k, z) = 1 \mid E^\ell] - \Pr[\text{EXEC}_{\phi, \mathcal{S}_\phi, \mathcal{E}_\phi}(k, z) = 1 \mid E^\ell] \right| \\ &\stackrel{\text{Equation (4)}}{=} \frac{1}{Q(k)} \left| \sum_{\ell=0}^{Q(k)-1} \Pr[\text{EXEC}_{\pi, \mathcal{D}, \mathcal{E}_\phi^\ell}(k, z) = 1] - \Pr[\text{EXEC}_{\phi, \mathcal{S}_\phi, \mathcal{E}_\phi^\ell}(k, z) = 1] \right| \\ &= \frac{1}{Q(k)} \left| \Pr[\text{EXEC}_{\pi, \mathcal{D}, \mathcal{E}_\phi^{Q(k)-1}}(k, z) = 1] - \Pr[\text{EXEC}_{\phi, \mathcal{S}_\phi, \mathcal{E}_\phi^0}(k, z) = 1] \right. \\ &\quad \left. + \sum_{\ell=0}^{Q(k)-2} \Pr[\text{EXEC}_{\pi, \mathcal{D}, \mathcal{E}_\phi^\ell}(k, z) = 1] - \Pr[\text{EXEC}_{\phi, \mathcal{S}_\phi, \mathcal{E}_\phi^{\ell+1}}(k, z) = 1] \right| \\ &\stackrel{\text{Lemma 2}}{=} \frac{1}{Q(k)} \left| \Pr[\text{EXEC}_{\pi, \mathcal{D}, \mathcal{E}_\phi^{Q(k)-1}}(k, z) = 1] - \Pr[\text{EXEC}_{\phi, \mathcal{S}_\phi, \mathcal{E}_\phi^0}(k, z) = 1] \right| \\ &\stackrel{\text{Lemma 3}}{=} \frac{1}{Q(k)} \left| \Pr[(\phi, \infty, \pi)\text{-EXEC}_{\rho, \mathcal{D}^{\text{DROP}}, \mathcal{E}_\rho}(k, z) = 1] - \Pr[\text{EXEC}_{\rho, \mathcal{S}_\rho, \mathcal{E}_\rho}(k, z) = 1] \right|. \end{aligned}$$

C.6 Definition of Environment $\mathcal{E}_\phi$ and Polynomial Q

In this section, we define the environment $\mathcal{E}_\phi$, which corresponds to the environment $\mathcal{E}_\pi$ used in the proof of [11, (Theorem 22)].

This definition requires introducing an auxiliary adversary, denoted $\tilde{\mathcal{S}}_\rho$, which is a variation of $\mathcal{S}_\rho$ designed specifically for (π, ℓ, ϕ) -executions of ρ .

Additionally, we introduce a polynomial Q , defined separately after presenting $\tilde{\mathcal{S}}_\rho$. It will be used in the construction of $\mathcal{E}_\phi$ to ensure it is a PPT ITM.

The final subsection of this section then formally defines and explains the construction of $\mathcal{E}_\phi$.

Throughout this section, we will sometimes refer to the ordering of extended sessions of ϕ_k according to their position in the execution. By definition of a (π, ℓ, ϕ) -execution, the first ℓ extended sessions of ϕ_k are replaced by extended sessions of π_k . We use the same ordering for panoptic sessions: for $u \geq 1$, the u -th panoptic session of ϕ_k corresponds to the u -th extended session of ϕ_k . Therefore, the first ℓ panoptic sessions of ϕ_k are replaced by the corresponding panoptic sessions of π_k .

Definition of Adversary $\tilde{\mathcal{S}}_\rho$. The algorithm for $\tilde{\mathcal{S}}_\rho$ is presented in Figure 19.

Its behavior is illustrated in Figure 20. To understand this diagram, recall that in any (π, ℓ, ϕ) -execution of ρ with security parameter $k \in \mathbb{N}$, each message received by the adversary is a pair $(\text{ReplacedSession}, m)$, where ReplacedSession denotes the set of sids corresponding to extended sessions of ϕ_k that have been replaced by sessions of π_k .

We can see that $\tilde{\mathcal{S}}_\rho$ is PPT, since $\mathcal{S}_\phi$, PSExpose^π , and PSExpose^ϕ are PPT.

Definition of Polynomial Q . Since (ρ, ϕ, π) is a PPT triplet, there exists a polynomial R such that for any $\ell \in \mathbb{N}$ and input $(k, z) \in \mathbb{N} \times \{0, 1\}^*$, every (ϕ, ℓ, π) -execution of ρ with adversary $\tilde{\mathcal{S}}_\rho$ and environment $\mathcal{E}_\rho$ on input (k, z) terminates in at most $R(k + |z|)$ steps. Without loss of generality, we may assume R is increasing on $[0, \infty[$ and takes values in $[2, \infty[$.

We define $Q(k) = R(k + k^d)$, where d is the constant from Equation (3) page 62. Consequently, for any $\ell \in \mathbb{N}$ and any $(k, z) \in \mathbb{N} \times \bigcup_{0 < a \leq k^d} \{0, 1\}^a$, the total number of computational steps (and therefore of extended sessions of ϕ_k and π_k) in a (ϕ, ℓ, π) -execution of ρ with adversary $\tilde{\mathcal{S}}_\rho$, environment $\mathcal{E}_\rho$ and input (k, z) is bounded by $Q(k)$.

Definition of Environment $\mathcal{E}_\phi$ The definition of $\mathcal{E}_\phi$ is given via two figures:

- Figure 22 presents the main function of $\mathcal{E}_\phi$, which relies on a subroutine **Simulation**.
- Figure 23 provides the definition of **Simulation**.

We now give an overview of the structure of $\mathcal{E}_\phi$.

Upon its first activation with input (k, z) , environment $(\mathcal{E}_\phi, 0)$ samples an integer $\ell \in \{0, \dots, Q(k) - 1\}$ and proceeds as $(\mathcal{E}_\phi^\ell, 0)$ on input (k, z) .

Roughly speaking, the goal of $(\mathcal{E}_\phi^\ell, 0)$ on input (k, z) is to simulate a (ϕ_k, ℓ, π_k) -execution of ρ_k with adversary $\tilde{\mathcal{S}}_\rho$ and environment $\mathcal{E}_\rho$, with the exception that it omits the ITIs from the $(\ell + 1)$ -th panoptic session of ϕ_k and the corresponding simulation of $\mathcal{S}_{\phi, s_\phi}$ (where s_ϕ denotes the sid of the $(\ell + 1)$ -th panoptic session).

$(\tilde{\mathcal{S}}_\rho, \diamond)$

```

//  $(\tilde{\mathcal{S}}_\rho, \diamond)$  will simulate multiple modified adversaries  $(\mathcal{S}_{\phi,s}, \diamond)$ 
// whose code is given in Figure 17
1 : At first activation,  $(\mathcal{S}_\rho, \diamond)$  reads the security parameter  $k$ 
    indicated in its dedicated tape, then acts as indicated
2 : At reception of an external-write  $w$  that is a backdoor from an ITI  $M$  :
3 : checkorreject the message is a pair  $(\text{ReplacedSession}, m)$  with
    ReplacedSession a subset of  $\{0, 1\}^*$ 
4 :  $w^{\text{WithoutSet}} \leftarrow w$  with message replaced by  $m$ 
5 :  $\text{positionM}_\phi \in \{\perp\} \cup \{0, 1\}^* \leftarrow \text{PSExpose}^\phi(k, M)$ 
6 : if  $\text{positionM}_\phi \in \{0, 1\}^*$  then Simulate  $(w^{\text{WithoutSet}}, \text{positionM}_\phi)$ 
7 : if  $\text{positionM}_\phi = \perp$  then send backdoor “ $(m, M)$ ” to the environment
8 : At reception of an external-write  $w$  that is an input (from the environment) :
9 : checkorreject the message is a pair  $(\text{ReplacedSession}, (m, M))$  with
    ReplacedSession a subset of  $\{0, 1\}^*$ 
10 :  $m$  a message and  $M$  an ITI
11 :  $w^{\text{WithoutSet}} \leftarrow w$  with message replaced by  $m$ 
12 :  $\text{positionM}_\pi \in \{\perp\} \cup \{0, 1\}^* \leftarrow \text{PSExpose}^\pi(k, M)$ 
13 :  $\text{positionM}_\phi \in \{\perp\} \cup \{0, 1\}^* \leftarrow \text{PSExpose}^\phi(k, M)$ 
14 : if  $\text{positionM}_\pi \in \{0, 1\}^*$  then
15 :     if  $\text{positionM}_\pi \notin \text{ReplacedSession}$  then Simulate  $(w^{\text{WithoutSet}}, \text{positionM})$ 
16 :     else send backdoor “ $m$ ” to  $M$ 
17 : else if  $\text{positionM}_\phi \in \{0, 1\}^*$  then reject
18 : else then send backdoor “ $m$ ” to  $M$ 

Simulate  $(w, s \in \{0, 1\}^*)$ 

1 : checkorreject  $w$  is an external-write
2 : if  $(\mathcal{S}_{\phi,s}, \diamond)$  has never been simulated then
3 :     initialize its simulation (with security parameter  $k$ )
4 :  $\tilde{w} \leftarrow w$  with recipient replaced by  $(\mathcal{S}_{\phi,s}, \diamond)$ 
5 : simulate applying  $\tilde{w}$  to the simulation of  $(\mathcal{S}_{\phi,s}, \diamond)$ 
6 : Then simulation of  $(\mathcal{S}_{\phi,s}, \diamond)$  continues until one of the following events occurs :
    if the simulation of  $(\mathcal{S}_{\phi,s}, \diamond)$  rejects then reject
7 : if the simulation of  $(\mathcal{S}_{\phi,s}, \diamond)$  deactivates (but does not reject) then deactivate
8 : if the simulation of  $(\mathcal{S}_{\phi,s}, \diamond)$  intends to send a backdoor then
9 :     send this backdoor

```

Fig. 19. Code of adversary $\tilde{\mathcal{S}}_\rho$. The highlighted line shows the precise difference between $\tilde{\mathcal{S}}_\rho$ and $\mathcal{S}_\rho$ (see Figure 16).

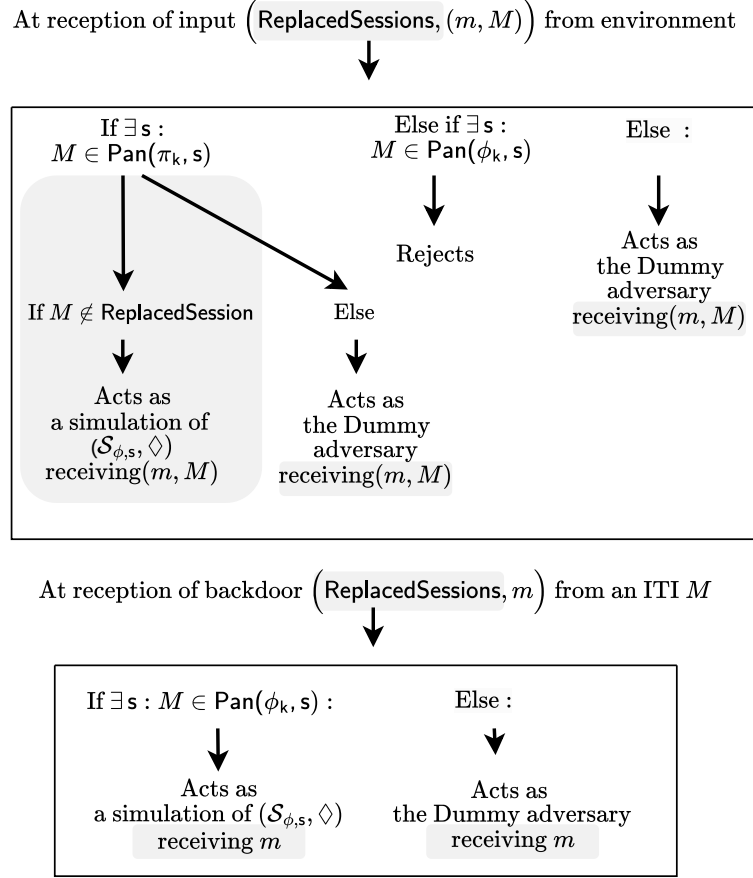


Fig. 20. Simplified representation of the adversary $(\tilde{S}_\rho, \diamond)$ for security parameter k . The gray area highlights the difference from S_ρ .

This is depicted in Figure 21. Note that, by definition, s_ϕ is the first extended session of ϕ_k not replaced in the (ϕ, ℓ, π) -execution.

More formally, on input (k, z) , $(\mathcal{E}_\phi^\ell, 0)$ simulates the execution described above, with the following exceptions:

- Inputs intended for main parties of $\text{Pan}(\phi_k, s_\phi)$ are not simulated, but instead sent directly by $\mathcal{E}_\phi$, impersonating the sender.
- If a simulated ITI N attempts to send an input to an ITI $M \in \text{PanSub}(\phi_k, s_\phi)$, the message is dropped and N deactivates.
- If a simulated ITI N attempts to send an output to an ITI $M \in \text{Pan}(\phi_k, s_\phi)$, the message is dropped and N deactivates.
- Messages from the protocol are interpreted as if coming from the $(\ell + 1)$ -th extended session of ϕ_k .
- Messages addressed to the simulation of $(\mathcal{S}_{\phi, s_\phi}, \diamond)$ are rerouted to the adversary.
- Messages from the adversary are interpreted as originating from the simulation of $(\mathcal{S}_{\phi, s_\phi}, \diamond)$.

To ensure that $\mathcal{E}_\phi$ runs in polynomial time, it maintains a counter that tracks the number of operations performed by the simulated ITIs. This counter does not account for the operations of $\mathcal{E}_\phi$ itself, but only for those performed by the ITIs being simulated, excluding any simulation overhead. If the counter exceeds $Q(k)$, the simulation halts and $\mathcal{E}_\phi$ returns 0.

Because, by construction, any (ϕ_k, ℓ, π_k) -execution of ρ_k with adversary $\tilde{\mathcal{S}}_\rho$ and input $(k, z) \in \bigcup_{0 \leq a \leq k^a} \{0, 1\}^a$ completes within $Q(k)$ steps, the counter does not affect the simulation in such executions.

Lemma 4. $\mathcal{E}_\phi$ is a ξ -identity bounded and PPT environment.

Proof. The counter mechanism ensures that $\mathcal{E}_\phi$ is PPT. Moreover, since ρ is (π, ϕ, ξ) -compliant, it satisfies Item 3 of Definition 10, and thus $\mathcal{E}_\phi$ is ξ -identity bounded.

C.7 Proof of the First Auxiliary Lemma

In the demonstration of the first auxiliary lemma, we will count the extended sessions of ϕ_k by their order of existence⁴⁵. By definition of the (π, ℓ, ϕ) -execution, the first ℓ extended sessions of ϕ_k will be replaced by extended sessions of π_k . We will also use this ordering for panoptic sessions: for $u \geq 1$, the u -th panoptic session of ϕ_k is defined as the panoptic session corresponding to the u -th extended session of ϕ_k . Consequently, the first ℓ panoptic sessions of ϕ_k will be replaced by panoptic sessions of π_k .

Restatement of Lemma 2 (First Auxiliary Lemma) *Let d be the constant described in Section C.4, whose existence follows from the assumption that Equation (1) is false.*

⁴⁵ Note that an extended session becomes none-empty at the moment one of its main party is invoked by an input or output.

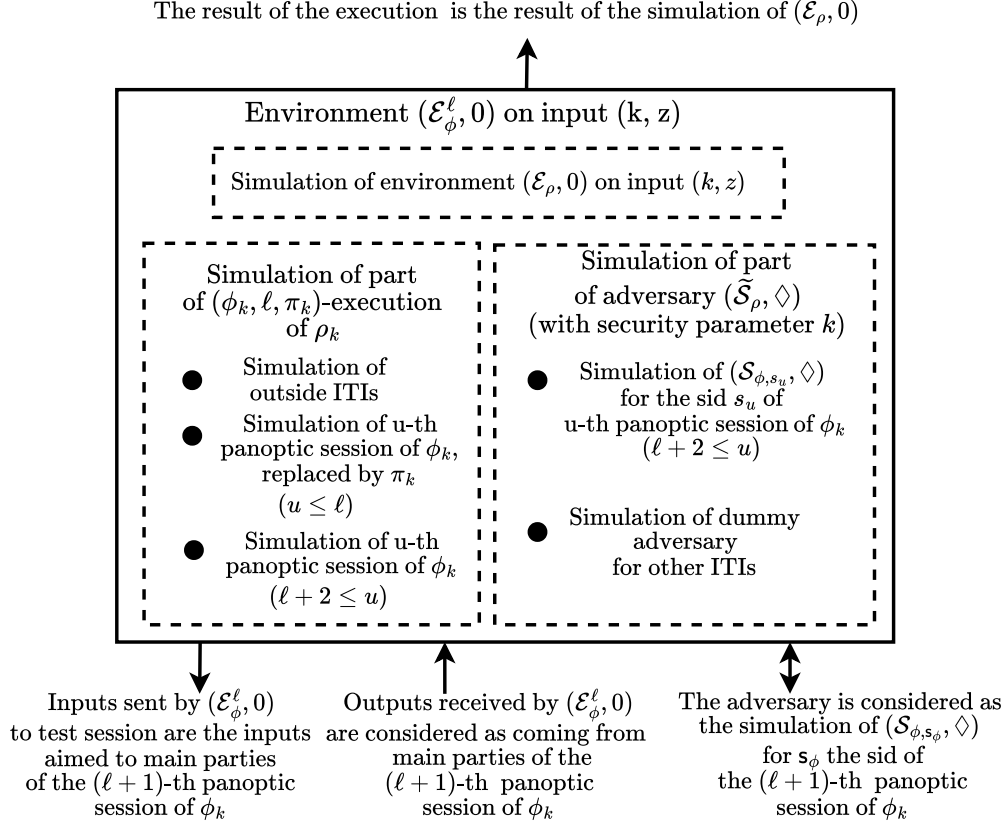


Fig. 21. Simplified representation of the environment $(\mathcal{E}_\phi^\ell, 0)$. We recall that s_ϕ is the sid of the $(\ell + 1)$ -th panoptic session of ϕ_k .

$(\mathcal{E}_\phi, 0)$ on entry (k, z)	
Local variables: s_ϕ initialized to a symbol $\square \notin \{\perp\} \cup \{0, 1\}^*$ // Note that the auxiliary function Simulation is defined in Figure 23	
1 :	<u>Mechanism used to ensure $\mathcal{E}_\phi$ is a PPT adversary :</u>
2 :	During the execution of $\mathcal{E}_\phi$ on any input (k, z) , a counter cpt tracks the total number of operations performed by all ITIs simulated by $\mathcal{E}_\phi$ (note that it does not count the operations of $\mathcal{E}_\phi$ itself, only those of the simulated ITIs) If, at any point, cpt $> Q(k)$, $\mathcal{E}_\phi$ stops and returns 0
3 :	// Note that the result of $(\mathcal{E}_\phi, 0)$ is described in Line 15 of Simulation
4 :	<u>At first activation :</u>
5 :	$(\mathcal{E}_\phi, 0)$ reads the security parameter k
6 :	$\ell \leftarrow \{0, \dots, Q(k) - 1\}$
7 :	Simulation $((\ell, k, z))$
8 :	<u>At reception of a backdoor “m” from the adversary :</u> // At this moment, s_ϕ will have been set by Simulation // As shown by Line 28 of Figure 23
9 :	Simulation (m)
10 :	<u>At reception of an output m from the protocol with identifier (s, p) and intended receiver an ITI M :</u> // by definition of protocol execution, note that s is the test sid
11 :	Simulation (m , s , p , M)
12 :	<u>At activation – not the first – with no message</u> // This happens in exactly three situations: // - If a (real) ITI deactivates (including rejects). // - If the (real) adversary sends a backdoor to a protocol ITI that does not exist. // - The (real) control function disallows an external write operation.
13 :	Simulation ()

Fig. 22. Main function of $\mathcal{E}_\phi$. For $\ell \in \{0, \dots, Q(k) - 1\}$, we denote by $\mathcal{E}_\phi^\ell$ the environment obtained by fixing the sample at Line 6 to ℓ .

Simulation(input)	
1 :	if input can be parsed as $(\ell, k, z) \in \mathbb{N} \times \mathbb{N} \times \{0, 1\}^*$ then
2 :	initialize the simulation of the (ϕ_k, ℓ, π_k) -execution of ρ_k with environment $\mathcal{E}$ and adversary $\tilde{\mathcal{S}}_\rho$ on input z
3 :	Continue the simulation until one of the exceptions after line 14 occurs
4 :	else if input can be parsed as a message $\mathbf{m} \in \{0, 1\}^*$ then
5 :	simulate the sending of backdoor $\mathbf{m}$ as if it came from the simulation of $(\mathcal{S}_{\pi, s_\phi}, \diamond)$ by $\tilde{\mathcal{S}}_\rho$
6 :	Continue the simulation until one of the exceptions after line 14 occurs
7 :	else if input can be parsed as $(\mathbf{m}, \mathbf{s}, \mathbf{p}, M) \in \{0, 1\}^* \times \{0, 1\}^* \times \{0, 1\}^* \times \text{SetITI}$ then
8 :	simulate the sending of output $\mathbf{m}$ to the simulation of M as if it came from $(\phi, (\mathbf{s}, \mathbf{p}))$
9 :	Continue the simulation until one of the exceptions after line 14 occurs
10 :	else if Simulation has been called with no input then
11 :	simulate the activation of $(\mathcal{E}, 0)$ with no message and continue the simulation
12 :	until one of the exceptions after line 14 occurs
13 :	else halt
14 :	List of exceptions:
15 :	exception : simulation of $(\mathcal{E}, 0)$ returns a bit $b \in \{0, 1\}$ then return b
16 :	exception : an input or output “ $\mathbf{m}$ ” is sent by a simulated protocol ITI M to a protocol ITI N that is a main party of the $(\ell + 1)$ -th extended session of ϕ_k then // An extended session is counted at the moment a main-party receives an input or an output
17 :	if $s_\phi = \square$ then $s_\phi \leftarrow \text{sid}(N)$
18 :	if it is an input then
19 :	send input “ $\mathbf{m}$ ” to the protocol with same sid and pid than N
20 :	if it is an output then
21 :	simulate the deactivation of M and continue the simulation
22 :	until one of the exceptions after line 14 occurs
23 :	exception : an input or output “ $\mathbf{m}$ ” is sent by a simulated protocol ITI M to a protocol ITI N with code $\neq \phi_k$ such that $\text{PSExpose}^\phi(k, N) = s_\phi$ then
24 :	simulate the deactivation of M and continue the simulation
25 :	until one of the exceptions after line 14 occurs
26 :	exception : the simulation of $\tilde{\mathcal{S}}_\rho$ intends to simulate an input
27 :	“ $\mathbf{m}$ ” to its simulation of $\mathcal{S}_{\phi, s_\phi}$ then
28 :	// Note that, this can only happen if $s_\phi \neq \square$
29 :	// As a consequence, s_ϕ is set before first activation of the adversary
30 :	send input “ $\mathbf{m}$ ” to the adversary

Fig. 23. Definition of the function Simulation used by $\mathcal{E}_\phi$.

For $(k, z) \in \mathbb{N} \times \bigcup_{0 < a \leq k^a} \{0, 1\}^a$ and $\ell \in \{0, \dots, Q(k) - 2\}$:

$$\text{EXEC}_{\pi, \mathcal{D}, \mathcal{E}_\phi^\ell}(k, z) = \text{EXEC}_{\phi, \mathcal{S}_\phi, \mathcal{E}_\phi^{\ell+1}}(k, z).$$

Proof. We will show that for $(k, z) \in \mathbb{N} \times \bigcup_{0 < a \leq k^a} \{0, 1\}^a$ and $\ell \in \{0, \dots, Q(k) - 1\}$:

$$\text{EXEC}_{\phi, \mathcal{S}_\phi, \mathcal{E}_\phi^\ell}(k, z) = (\phi, \ell, \pi)\text{-EXEC}_{\rho, \tilde{\mathcal{S}}_\rho, \mathcal{E}_\rho}(k, z), \quad (5)$$

$$\text{EXEC}_{\pi, \mathcal{D}, \mathcal{E}_\phi^\ell}(k, z) = (\phi, \ell + 1, \pi)\text{-EXEC}_{\rho, \tilde{\mathcal{S}}_\rho, \mathcal{E}_\rho}(k, z). \quad (6)$$

These two equalities together imply the lemma.

We proceed to prove the first equality; the second can be shown analogously.

Fix $(k, z) \in \mathbb{N} \times \bigcup_{0 < a \leq k^a} \{0, 1\}^a$ and $\ell \in \{0, \dots, Q(k) - 1\}$.

For clarity, we temporarily disregard the counter mechanism defined in Line 1 of Figure 22, and defer its discussion to page 80.

To simplify terminology throughout this proof, we introduce the following notation:

- The *R-execution* (“R” for Real) refers to the (ϕ_k, ℓ, π_k) -execution of ρ_k with environment $\mathcal{E}_\rho$ and adversary $\tilde{\mathcal{S}}_\rho$, on input (k, z) .
- The *S-execution* (“S” for Simulated) refers to the execution of ϕ_k with environment $\mathcal{E}_\phi^\ell$ and adversary $\mathcal{S}_\phi$, on input (k, z) .

Let $\mathbf{s}_\rho$ be the test session identifier in the R-execution, and $\mathbf{s}_\phi$ the test sid in the S-execution.

Our goal is to establish a tight correspondence between the two executions such that the outputs of the real environment $(\mathcal{E}_\rho, 0)$ in the R-execution and the simulated version of $(\mathcal{E}_\rho, 0)$ in the S-execution are identically distributed.

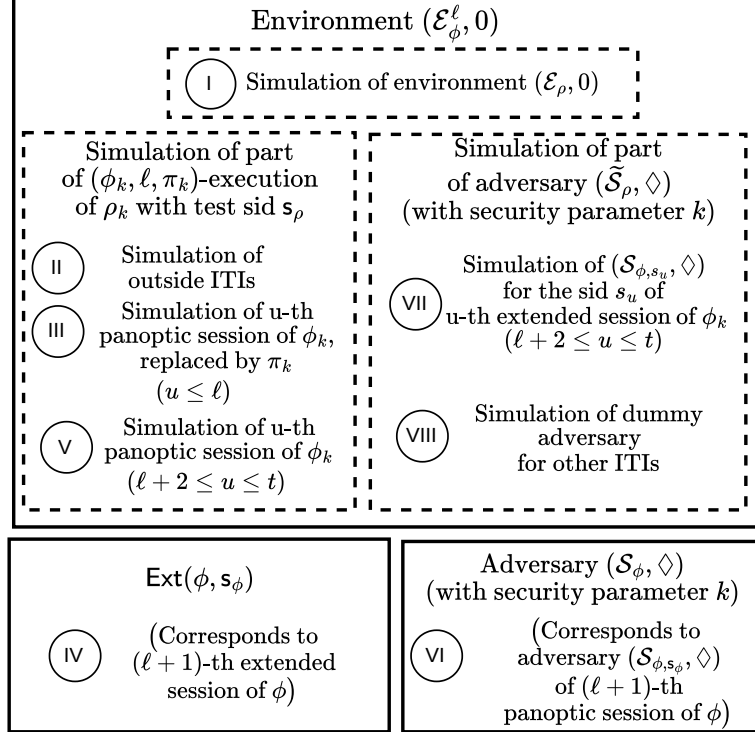
Both executions consist of a combination of real and simulated ITIs, grouped into eight classes ① through ⑧ for the R-execution and corresponding classes ① through ⑧ for the S-execution. These groupings are illustrated in Figure 24.

We adopt the following convention for the remainder of this proof: the group ④ (respectively, ④) consists precisely of the ITIs forming a panoptic session—specifically, the session $\text{Pan}(\phi_k, \mathbf{s}_\phi)$. Accordingly, when referring to the “extended session”, “main parties”, “sub-parties”, “panoptic session”, or “panoptic sub-parties” of ④ (respectively, ④), we mean the corresponding elements of the panoptic session represented by ④ (respectively, ④).

In an ideal setting, every message exchanged in the R-execution would have a direct counterpart in the S-execution. However, this is not the case. For example, in the R-execution, an ITI $M \in \textcircled{2}$ may send an input to a panoptic sub-party $N \in \textcircled{4}$. In the S-execution, however, the corresponding ITI $M \in \textcircled{II}$ is simulated by $(\mathcal{E}_\phi^\ell, 0)$, and N is a panoptic sub-party of ④. Due to the protocol execution rules, $(\mathcal{E}_\phi^\ell, 0)$ is not permitted to send inputs to ITIs in $\text{PanSub}(\phi_k, \mathbf{s}_\phi)$ —in particular, not to N .

To guarantee that such mismatches do not impact the outcome of the execution, we must verify that, in each of these cases, the R- and S-executions produce behavior that is indistinguishable from the point of view of $(\mathcal{E}_\rho, 0)$. In particular,

S-execution (an execution of ϕ):



R-execution (a (ϕ, l, π) -execution of ρ):

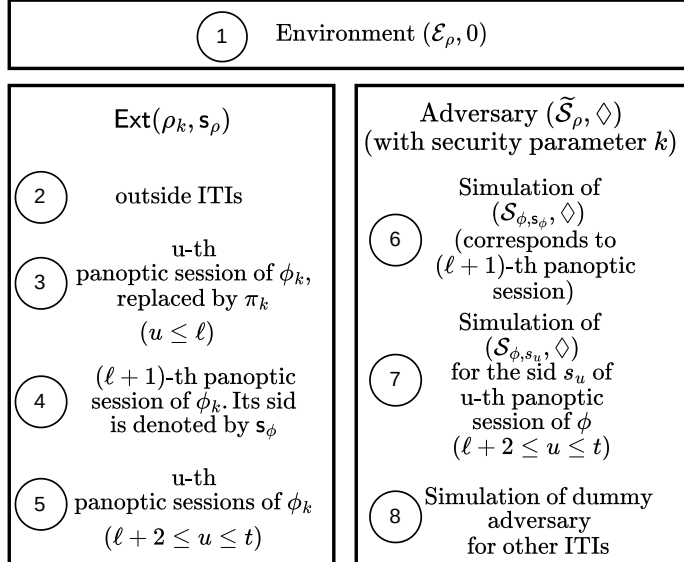


Fig. 24. The upper figure is a representation of a S-execution, with the groups **I** to **VIII**. The lower figure is a representation of a R-execution, with the groups **1** to **8**.

we will frequently rely on the observation that, from the point of view of $(\mathcal{E}_\rho, 0)$, there is no difference between an ITI M deactivating and M sending a message to an ITI N that rejects it.

Accordingly, we must carefully examine all situations in which the correspondence between the executions is imperfect, and ensure that none of these discrepancies affect the outcome as observed by $(\mathcal{E}_\rho, 0)$. To this end, we analyze how the correspondence is established for each communication of the form $\textcircled{i} \rightarrow \textcircled{j}$ in the R-execution, where $i, j \in \{1, \dots, 8\}$ and $i \neq j$.

We begin by noting that communications between ITIs belonging to any of the groups $\textcircled{1}$, $\textcircled{2}$, $\textcircled{3}$, $\textcircled{5}$, $\textcircled{7}$, and $\textcircled{8}$ can be simulated perfectly by the S -execution, since the corresponding ITIs in the S -execution ($\textcircled{I}$, $\textcircled{II}$, $\textcircled{III}$, $\textcircled{V}$, $\textcircled{VII}$, $\textcircled{VIII}$) are all simulated by $(\mathcal{E}_\phi^\ell, 0)$.

This leaves us with the following four cases, which require careful analysis:

- case 1: Messages *from* $\textcircled{4}$ to other groups versus messages from $\textcircled{IV}$.
- case 2: Messages *to* $\textcircled{4}$ versus messages to $\textcircled{IV}$.
- case 3: Messages *from* the simulated adversary ITI $\textcircled{6}$ to other groups versus those from $\textcircled{VI}$.
- case 4: Messages *to* $\textcircled{6}$ from other ITIs versus those to $\textcircled{VI}$.

Case 1: Correspondence for Messages Sent by $\textcircled{4}$ and $\textcircled{IV}$

We now examine the messages sent by ITIs in $\textcircled{4}$ (in the R-execution) and the corresponding behavior of ITIs in $\textcircled{IV}$ (in the S -execution). The only types of such communications are:

- case1.i Input to an ITI in $\textcircled{2}$, $\textcircled{3}$, or $\textcircled{5}$.
- case1.ii Output to an ITI in $\textcircled{2}$, $\textcircled{3}$, or $\textcircled{5}$.
- case1.iii Backdoor to the adversary (an ITI in $\textcircled{6}$, $\textcircled{7}$, or $\textcircled{8}$).
- case1.iv An ITI of $\textcircled{4}$ deactivates, causing the environment to be activated with no message.
- case1.v An ITI of $\textcircled{4}$ performs an external write that is disallowed by the control function, causing the environment to be activated with no message.

Argument for case1.i:

This type of communication cannot occur, due to the fact that ϕ_k is sub-party respecting. More specifically, this follows from Item spr.6 of Definition 9.

Argument for case1.ii:

In the R-execution, the sub-party respecting property of ϕ_k ensures that only main parties of $\text{Ext}(\phi_k, \mathbf{s}_\phi)$ are allowed to send such outputs (Item spr.5 of Definition 9).

In the S -execution, the corresponding outputs are redirected to the environment $(\mathcal{E}_\phi^\ell, 0)$, which then simulates their delivery to the intended ITIs in $\textcircled{II}$, $\textcircled{III}$, or $\textcircled{V}$. This reproduces the same outcome from the perspective of the receiving ITI.

Argument for case1.iii:

We will also use this analysis to address Item case2.iii. Specifically, we examine the following two types of backdoor communications:

- Backdoors sent by ④ to the adversary (that is, to ⑥, ⑦, or ⑧),
- Backdoors sent by the adversary (⑥, ⑦, or ⑧) to ④.

In the R-execution, $\tilde{S}_\rho$ internally simulates a collection of adversaries $(\mathcal{S}_{\phi, s}, \diamond)$ – one for each panoptic session $\text{Pan}(\phi_k, s)$ that is replaced by a panoptic session $\text{Pan}(\pi_k, s)$.

Communications between $\tilde{S}_\rho$ and these simulated adversaries are handled in such a way that each $(\mathcal{S}_{\phi, s}, \diamond)$ “believes” it is operating as a real adversary. This is achieved through the simulation of backdoors and inputs.

In the S-execution, the environment $(\mathcal{E}_\phi^\ell, \diamond)$ simulates the full behavior of $\tilde{S}_\rho$, with the sole exception of the specific adversary $(\mathcal{S}_{\phi, s_\phi}, \diamond)$, which is instead realized as the actual adversary $(\mathcal{S}_\phi, \diamond)$ of the S-execution.

There are four potential flaws in this simulation, which we now examine in detail:

- *Flaw 1:* In the S-execution, the adversary $(\mathcal{S}_\phi, \diamond)$ is used in place of $(\mathcal{S}_{\phi, s_\phi}, \diamond)$. $(\mathcal{S}_\phi, \diamond)$ is permitted to send backdoors to any ITI that exists in the execution, whereas $(\mathcal{S}_{\phi, s_\phi}, \diamond)$ restricts backdoor transmissions to ITIs within $\text{Pan}(\phi_k, s_\phi)$. Using Propositions 3 and 5, we can see that the behaviors of the two executions coincide.
- *Flaw 2:* In the S-execution, there is no mechanism to simulate backdoors from ④ to ⑦ or from ④ to ⑧. However, by the definition of $\tilde{S}_\rho$, any backdoor sent by an ITI $M \in \text{④}$ is directed to ⑥, the simulation of $\mathcal{S}_{\phi, s_\phi}$. As a result, there are no backdoors from ④ to ⑦ or from ④ to ⑧.
- *Flaw 3:* In the S-execution, there is no mechanism for simulating backdoors from ⑦ to ④. However, backdoors from ⑦ to ④ do not occur in the R-execution either. Indeed, for any $s \neq s_\phi$, the simulated adversary $(\mathcal{S}_{\phi, s}, \diamond)$ only sends backdoors to ITIs in $\text{Pan}(\phi_k, s)$. Since ϕ_k is separated, it follows that $\text{Pan}(\phi_k, s) \cap \text{④} = \emptyset$ (recall that $\text{④} = \text{Pan}(\phi_k, s_\phi)$). Hence, no backdoor from ⑦ to ④ arises.
- *Flaw 4:* In the S-execution, there is no mechanism for simulating backdoors from ⑧ to ④. However, backdoors from ⑧ to ④ do not occur in the R-execution either. By the definition of $\tilde{S}_\rho$, such a backdoor would correspond to an input of the form $(\text{ReplacedSession}, (m, M))$ received by $\tilde{S}_\rho$, where $M \in \text{Pan}(\phi_k, s_\phi)$. But $\tilde{S}_\rho$ explicitly rejects all such inputs when $M \in \text{Pan}(\phi_k, \tilde{s})$ for some $\text{sid } \tilde{s}$. Therefore, no backdoor from ⑧ to ④ can occur.

Argument for case1.iv:

This is perfectly simulated in the S-execution. Indeed, when an ITI of ④ deactivates, the environment $(\mathcal{E}_\phi^l, 0)$ simulates the activation of $(\mathcal{E}_\rho, 0)$ with no message.

Argument for case1.v:

This is perfectly simulated in the S-execution. Indeed, when the control function disallows an external operation, the environment $(\mathcal{E}_\phi^l, 0)$ simulates the activation of $\textcircled{I}$ (i.e. $(\mathcal{E}_\rho, 0)$) with no message.

Case 2: Correspondence for Messages Sent to $\textcircled{4}$ and $\textcircled{IV}$

We now consider communications from an ITI outside $\textcircled{4}$ to an ITI within $\textcircled{4}$. These interactions fall into the following categories:

- case2.i Inputs from an ITI in $\textcircled{2}$, $\textcircled{3}$, or $\textcircled{5}$.
- case2.ii Outputs from an ITI in $\textcircled{2}$, $\textcircled{3}$, or $\textcircled{5}$.
- case2.iii Backdoors from the adversary (originating from $\textcircled{6}$, $\textcircled{7}$, or $\textcircled{8}$).

Argument for case2.i:

In the R-execution, ITIs belonging to $\textcircled{2}$, $\textcircled{3}$, or $\textcircled{5}$ may send inputs to any ITI within the panoptic session $\textcircled{4}$. However, since ϕ_k is both sub-party respecting and separated, only inputs addressed to main parties of $\textcircled{4}$ are accepted. All others are rejected in accordance with Item **spr.3** of Definition 9. Furthermore, these rejections cannot be exploited by the adversary to inject backdoors, as guaranteed by Item **spr.1**.

In the S-execution, the protocol execution rules ensure that simulated ITIs in $\textcircled{II}$, $\textcircled{III}$, or $\textcircled{V}$ may only send inputs to main parties of $\textcircled{IV}$. Any attempt to send such an input to a panoptic sub-party of $\textcircled{IV}$ results in a deactivation.

The manner this deactivation is made depends on whether the environment $(\mathcal{E}_\phi^\ell, \diamond)$ has already determined the value of the test sid s_ϕ . This identification occurs only when a simulated ITI first attempts to interact with a main party of $\textcircled{IV}$.

Let N be a simulated ITI in $\textcircled{II}$, $\textcircled{III}$, or $\textcircled{V}$, and suppose N tries to send an input to a panoptic sub-party $M \in \text{PanSub}(\phi_k, s_\phi) \subset \textcircled{IV}$. Two cases must be considered:

- If s_ϕ is already known as the test sid, then $(\mathcal{E}_\phi^\ell, \diamond)$ directly simulates the deactivation of N .
- If s_ϕ has not yet been determined as the test sid, the simulation proceeds and send the input to a simulation of M . Since ϕ is sub-party respecting and separated, the message is rejected, and $(\mathcal{E}_\phi^\ell, 0)$ then simulates the deactivation of M .

Thus, in both the R-execution and the S-execution, the result is the same from the point of view of $(\mathcal{E}_\rho, 0)$.

Argument for case2.ii:

In the R-execution, ITIs belonging to $\textcircled{2}$, $\textcircled{3}$, or $\textcircled{5}$ may send outputs to any ITI within the panoptic session $\textcircled{4}$. However, since ϕ_k is sub-party respecting and separated, each of these outputs are rejected. This follows from Item **spr.4** of Definition 9. Moreover, as guaranteed by Item **spr.1**, these rejections cannot be exploited by the adversary to transmit backdoors.

In the S-execution, the execution rules ensure that simulated ITIs from $\textcircled{II}$, $\textcircled{III}$, or $\textcircled{V}$ cannot send outputs to ITIs of $\textcircled{IV}$. Any attempt to send such an output results in a deactivation.

The manner this deactivation is made depends on whether the environment $(\mathcal{E}_\phi^\ell, \diamond)$ has already determined the value of the test sid s_ϕ .

Let N be a simulated ITI in $\textcircled{II}$, $\textcircled{III}$, or $\textcircled{V}$, and suppose N tries to send an output to an ITI $M \in \textcircled{4}$. Three cases arise:

- If $M \in \text{MainParty}(\phi_k, s_\phi)$, then the environment necessarily knows that s_ϕ is the test sid, and it simulates the deactivation of M .
- If $M \in \text{ExtSub}(\phi_k, s_\phi)$ and it is already known that s_ϕ is the test sid, the environment simulates the deactivation of M .
- If $M \in \text{ExtSub}(\phi_k, s_\phi)$ and s_ϕ is not yet known as the test sid, the simulation proceeds and send the output to a simulation of M . Since ϕ is sub-party respecting and separated, the message is rejected, and $(\mathcal{E}_\phi^\ell, 0)$ then simulates the deactivation of M .

Thus, in both the R-execution and the S-execution, the result is the same from the point of view of $(\mathcal{E}_\rho, 0)$.

Argument for case2.iii:

This case concerns backdoors from the adversary to $\textcircled{4}$. It is already treated by the argument for case1.iii (page 76).

Argument for case2.iii:

This case concerns backdoors sent by the adversary (*i.e.*, from $\textcircled{6}$, $\textcircled{7}$, or $\textcircled{8}$) to ITIs of $\textcircled{4}$. This is addressed in the argument for case1.iii (page 76).

Case 3: Correspondence for Messages Sent by $\textcircled{6}$ and $\textcircled{VI}$

We now consider communications from $\textcircled{6}$ to an ITI distinct of $\textcircled{6}$. These interactions fall into the following categories:

- case3.i Backdoors to the environment $\textcircled{1}$.
- case3.ii Backdoors to protocol ITIs (*i.e.*, ITIs in $\textcircled{2}$, $\textcircled{3}$, $\textcircled{4}$, or $\textcircled{5}$).
- case3.iii If $\textcircled{6}$ sends a backdoor to an ITI that does not exist, this results in the activation of the environment with no message.
- case3.iv If $\textcircled{6}$ attempts an external write that is disallowed by the control function, this results in the activation of the environment with no message.

Argument for case3.i:

As explained in the argument for case1.iii on page 75, all backdoor communications from $\textcircled{6}$ to the environment $\textcircled{1}$ in the R-execution are perfectly simulated in the S-execution by communications from $\textcircled{VI}$ to $\textcircled{I}$.

Argument for case3.ii:

In the R-execution, any adversary $(\mathcal{S}_{\phi, s}, \diamond)$ can only send backdoors to ITIs in the panoptic session $\text{Pan}(\phi_k, s)$. In particular, $(\mathcal{S}_{\phi, s_\phi}, \diamond)$ can only send backdoors to ITIs within $\text{Pan}(\phi_k, s_\phi)$.

In the S-execution, the adversary $(\mathcal{S}_\phi, \diamond)$ is not similarly restricted, but the protocol execution rules guarantee that any backdoor not targeting the environment or an ITI in $\text{Ext}(\phi_k, s_\phi)$ is disallowed.

The only potential discrepancy arises when $(\mathcal{S}_{\phi, s_\phi}, \diamond)$ sends a backdoor to an ITI $M \in \text{Pan}(\phi_k, s_\phi) \setminus \text{Ext}(\phi_k, s_\phi)$, which is permitted in the R-execution.

However, Proposition 3 ensures that such an M must be sleeping, and Item **spr.1** of the sub-party respecting definition guarantees that the message is rejected. Thus, the behaviors are indistinguishable from the point of view of $(\mathcal{E}_\rho, 0)$.

Argument for case3.iii:

Assume that ⑥ sends a backdoor to a nonexistent ITI M . By definition of protocol execution, and since the adversary always unsets **forced-write**, the environment ① = $(\mathcal{E}_\rho, 0)$ is activated with no message in the R-execution.

In the S-execution, the simulation mirrors this behavior.

If $M \notin \text{Pan}(\phi, \mathbf{s}_\phi)$, by definition, $(\mathcal{S}_{\phi, \mathbf{s}_\phi}, \diamond)$ will not send the backdoor and reject.

We now suppose that $M \in \text{Pan}(\phi, \mathbf{s}_\phi)$.

From the argument in case2.i (page 77), the set of awake ITIs in ④ and $\overline{IV}$ coincide, so M is also sleeping in the S-execution.

Two cases arise:

- If M exists but is sleeping, the message is rejected by Item **spr.1** of the sub-party respecting property. This leads to the activation of $(\mathcal{E}_\phi^\ell, 0)$ with no message, which then simulates the activation of $\overline{I} = (\mathcal{E}_\rho, 0)$ with no message.
- If M does not exist at all, the adversary again unsets **forced-write**, and $(\mathcal{E}_\phi^\ell, 0)$ activates with no message, simulating an activation of $\overline{I} = (\mathcal{E}_\rho, 0)$ with no message.

In both cases, the outcome is an activation of $(\mathcal{E}_\rho, 0)$ with no message, matching the R-execution.

Argument for case3.iv:

This situation is perfectly simulated in the S-execution. Whenever the control function disallows an external write in the R-execution, the environment ① = $(\mathcal{E}_\rho, 0)$ is activated with no message. The same happens in the S-execution: $(\mathcal{E}_\phi^\ell, 0)$ simulates this activation of $\overline{I} = (\mathcal{E}_\rho, 0)$ with no message.

Case 4: Correspondence for Messages Sent to ⑥ and $\overline{VI}$

We now examine the communications received by the simulated adversary ⑥ = $(\mathcal{S}_{\phi, \mathbf{s}_\phi}, \diamond)$ from ITIs outside ⑥. These fall into two categories:

- case4.i Inputs from the environment ① (i.e., inputs sent to $(\tilde{\mathcal{S}}_\rho, \diamond)$ that are internally routed to the simulation of $(\mathcal{S}_{\phi, \mathbf{s}_\phi}, \diamond)$).
- case4.ii Backdoors from protocol ITIs (i.e., backdoors sent to $(\tilde{\mathcal{S}}_\rho, \diamond)$ by ITIs in ②, ③, ④, or ⑤, that are internally routed to the simulation of $(\mathcal{S}_{\phi, \mathbf{s}_\phi}, \diamond)$).

Argument for case4.i:

As explained in the argument for case1.iii page 75, the communications between ① and ⑥ are perfectly simulated by the communications between $\overline{I}$ and $\overline{VI}$.

Argument for case 4.ii:

In the R-execution, $\tilde{\mathcal{S}}_\rho$ only forwards backdoors to $\textcircled{6} = (\mathcal{S}_{\phi, \mathbf{s}_\phi}, \diamond)$ if they originate from an ITI $M \in \text{Pan}(\phi_k, \mathbf{s}_\phi)$.

In the S-execution, the only ITIs authorized to send backdoors to $\textcircled{VI} = (\mathcal{S}_\phi, \diamond)$ are those belonging to $\text{Ext}(\phi_k, \mathbf{s}_\phi)$.

These two cases are equivalent. Proposition 5 guarantees that $\text{Ext}(\phi_k, \mathbf{s}_\phi) \subset \text{Pan}(\phi_k, \mathbf{s}_\phi)$, and Proposition 3 ensures that any ITI in the set difference $\text{Pan}(\phi_k, \mathbf{s}_\phi) \setminus \text{Ext}(\phi_k, \mathbf{s}_\phi)$ is necessarily sleeping. Since sleeping ITIs cannot send messages, there is no loss in simulation fidelity.

Addition of the counter:

We now argue that adding a counter to $\mathcal{E}_\phi^\ell$ —as specified in Line 1 of Figure 22—has no effect on the correspondence between R- and S-executions.

By construction of the polynomial Q and the assumption that $(k, z) \in \mathbb{N} \times \bigcup_{0 < a \leq k^d} \{0, 1\}^a$ with $0 \leq \ell \leq Q(k)$, we know that every (ϕ, ℓ, π) -execution of ρ_k with adversary $\tilde{\mathcal{S}}_\rho$ and environment $\mathcal{E}_\rho$ on input (k, z) terminates within fewer than $Q(k)$ computational steps.

In particular, this implies that all R-executions are bounded by $Q(k)$ steps. Since we have established a faithful correspondence between R-executions and S-executions, the same bound applies to the total number of simulated steps performed by $\mathcal{E}_\phi^\ell$.

Hence, the counter within $\mathcal{E}_\phi^\ell$ is never exceeded in any of the executions under consideration. The presence of the counter has no effect on the behavior of the simulation and does not alter the outcome of the execution.

This completes the proof

C.8 Proof of the Second Auxiliary Lemma

Restatement of Lemma 3 (Second Auxiliary Lemma) *Let d be the constant described in Section C.4, whose existence follows from the assumption that Equation (1) is false. For $(k, z) \in \mathbb{N} \times \bigcup_{0 < a \leq k^d} \{0, 1\}^a$:*

$$\begin{aligned} \text{EXEC}_{\phi, \mathcal{S}_\phi, \mathcal{E}_\phi^0}(k, z) &= \text{EXEC}_{\rho, \mathcal{S}_\rho, \mathcal{E}_\rho}(k, z), \\ \text{EXEC}_{\pi, \mathcal{D}, \mathcal{E}_\phi^{Q(k)-1}}(k, z) &= (\phi, \infty, \pi)\text{-EXEC}_{\rho, \mathcal{D}^{\text{DROP}}, \mathcal{E}_\rho}(k, z). \end{aligned}$$

Proof. Let us fix $(k, z) \in \mathbb{N} \times \bigcup_{0 < a \leq k^d} \{0, 1\}^a$.

Thanks to Equations (5) and (6), established in the proof of Lemma 2, it suffices to prove:

$$\begin{aligned} (\phi, 0, \pi)\text{-EXEC}_{\rho, \tilde{\mathcal{S}}_\rho, \mathcal{E}_\rho}(k, z) &= \text{EXEC}_{\rho, \mathcal{S}_\rho, \mathcal{E}_\rho}(k, z), \\ (\phi, Q(k), \pi)\text{-EXEC}_{\rho, \tilde{\mathcal{S}}_\rho, \mathcal{E}_\rho}(k, z) &= (\phi, \infty, \pi)\text{-EXEC}_{\rho, \mathcal{D}^{\text{DROP}}, \mathcal{E}_\rho}(k, z). \end{aligned}$$

First equation:

In any $(\phi, 0, \pi)$ -execution of ρ , the set `ReplacedSession` is empty by definition. Comparing the algorithms of $\mathcal{S}_\rho$ and $\tilde{\mathcal{S}}_\rho$ (Figures 16 and 19, respectively), we can directly conclude.

Second equation:

By construction of Q , we know that any $(\phi, Q(k), \pi)$ -execution of ρ on input (k, z) halts in fewer than $Q(k)$ computational steps. This implies that all extended sessions of ϕ_k are replaced by corresponding sessions of π_k within this time bound. Hence, such executions are equivalent to the (ϕ, ∞, π) -executions of ρ .

Furthermore, from the definition of $\tilde{\mathcal{S}}_\rho$, we observe that when all sessions of ϕ_k have been replaced, $\tilde{\mathcal{S}}_\rho$ behaves exactly like $\mathcal{D}^{\text{DROP}}$. The result follows.

D Details About Issues, Limitations and Weaknesses Described in the Main Body

This section provides complete counterexamples and detailed proofs for Issue (I.2), Issue (I.3), and Issue (I.4) introduced in the main body.

Preliminary notations and conventions relevant to these constructions are given in Appendix D.1.

D.1 Additional Notations and Conventions

Additional Notations for ITIs.

Let `SetITI` denote the set of all ITIs.

For an ITI M , we denote its code by `code`(M) and its identifier by `id`(M). If this identifier is a pair $(s, p) \in \{0, 1\}^* \times \{0, 1\}^*$, we write `sid`(M) for its `sid` and `pid`(M) for its `pid`.

For any external-write w , `mess`(w) denotes its message and `tape`(w) its destination tape.

Additional Conventions and Instructions for Algorithms.

LOCAL FUNCTIONS. Each local function has access to the local variables of the main function and may modify them.

SENDING MESSAGES. The instructions `apply` and `send` both indicate that the ITI performs an external-write instruction w . More precisely, `apply` is used in the form “`apply` w ”, and `send` appears as “`send` ‘ m ’ to tape t of ITI N with flags (...)”. If the context is clear, flags may be omitted. When the destination tape t is known, we often use `sendinput`, `sendoutput`, or `sendbackdoor`.

DEACTIVATION, REJECT, AND HALT. The instruction `deactivate` deactivates the ITI, `reject` causes it to reject, and `halt` places it in a halt state. Once halted, the ITI deactivates and rejects any subsequent messages.

PARSING. The instruction “`checkorchalt` m is (...)” checks whether a message m matches the specified pattern. If so, execution proceeds; otherwise, the ITI halts.

This instruction is typically used to parse m as a tuple and halt if parsing fails. The instruction **checkorreject** performs the same check but rejects instead of halting.

IN/OUT ACTIVATION. An in/out activation refers to an activation triggered by the reception of an input or output. For example, the “second in/out activation” of an ITI refers to the second time it is activated via an input or output.

ENDING EXECUTION. In environment algorithms, for $b \in \{0, 1\}$, the instruction “**return** b ” causes $\mathcal{E}$ to write b to the outgoing message tape and halt. This instruction thus terminates the execution with result b . The result of an execution with environment $\mathcal{E}$ is sometimes referred to as the “result of $(\mathcal{E}, 0)$ ”.

DEFAULT BEHAVIOR. If an ITM’s algorithm omits a response to certain events (*e.g.* it specifies a reaction to inputs but not to outputs or backdoors), the ITM is assumed to reject by default.

Use of the New Extended Session Definition. Due to the limitation highlighted in (L.1) (Section 5.1), we adopt our own extended session definition (Definition 3).

No Use of Parametrized Systems. The counterexamples we construct are not parametrized systems, but rather ITMs.

Although UC-emulation is formally defined only for parametrized systems, the notion naturally extends to ITMs. For any ITMs π and ϕ , we define $\pi_k = \pi$ and $\phi_k = \phi$ for all k , and apply the UC-emulation definition to the parametrized systems $\pi = (\pi_k)_k$ and $\phi = (\phi_k)_k$.

Simplification of the Subroutine Exposing Property. Certain protocols ϕ we will construct require the subroutine exposing property. In every such case, the exposing ITI for the extended session $\text{Ext}(\phi, \mathbf{s})$ will be $(\phi, (\mathbf{s}, 0))$, and each ITI $(\phi, (\mathbf{s}, \mathbf{p}))$ with $\mathbf{p} \neq 0$ will halt upon its first activation.

In our counterexamples, these latter ITIs are not monitored by the exposing ITI, as implementing such functionality would compromise readability and inflate the size of the algorithm.

Nevertheless, this feature could be added without affecting any relevant properties: an ITI $(\phi, (\mathbf{s}, \mathbf{p}))$ may notify $(\phi, (\mathbf{s}, 0))$ of its invocation by sending it an output. This difference has no impact on any of the properties considered in our counterexamples.

Use of Flags. Unless explicitly stated otherwise, the following conventions apply:

- Messages exchanged between protocol ITIs (*i.e.* any ITI except the adversary and environment) have both flags **forced-write** and **reveal-ext-id** set.
- Backdoors sent by the adversary have both **forced-write** and **reveal-ext-id** unset.
- Inputs sent by the environment have both flags **forced-write** and **reveal-ext-id** set.

Use of Autoreferences. For clarity, we permit the code of an ITI to include its own code—despite this being formally undefined.

This abstraction simplifies the frequent situation involving two ITIs, say (ψ, id) and (α, id') , such that:

- (ψ, id) sends a message to (α, id') ,
- and (α, id') sends a message back to (ψ, id) .

In this scenario, the code of ψ would embed that of α , which in turn embeds the code of ψ , introducing a circular definition.

To circumvent this, we rely on the fact that an ITI has access to its own code via its identity tape, as defined in [11, (Section 3.1.1)]. We implement the following workaround in ψ :

- Replace every occurrence of ψ in its own code with a reserved sequence `seq`.
- Modify ψ 's code so that, at runtime, the ITI replaces `seq` with the code stored on its identity tape.

Structured Protocols' sid and pid Match Their Body.

For structured protocols such as the UC operator $\rho^{\phi \rightarrow \pi}$, we require their sid and pid to match those of their body.

This convention is not respected in Plain UC, as pointed out in Issue (I.5) (see Appendix F.1), and leads to a counterexample of the UC Theorem.

Compliance. We omit the full definition of compliance from the main body of the article; it is reproduced here for completeness.

Definition 19 (Compliance, ([11, (Section 6.1)])). *A protocol ρ is called (π, ϕ, ξ) -compliant if:*

- *All inputs sent by any ITI of any extended session of ρ are made with the flag forced-write set. All outputs sent to any ITI of any extended session of ρ have the flag reveal-ext-id set.*
- *No two external-write instructions, out of all the external-write instructions made by the members of an extended session of ρ , where one instruction has target code π , and the other instruction has target code ϕ , have the same target sid.*
- *The extended identities of all the ITIs in any extended session of ρ , that pass inputs to ITIs with code either π or ϕ , satisfy the predicate ξ .*

When $\xi = \xi_{\text{true}}$, the always-true predicate, we say that the protocol is (π, ϕ) -compliant.

D.2 Details Regarding Issue (I.2)

In this section, we present a comprehensive counterexample for Issue (I.2).

Figure 25 defines the protocols ϕ , π , and α – adaptations of those introduced in the discussion of Issue (I.2) (Section 5.1) – modified to satisfy the conditions required for the application of the UC theorem. These protocols rely on an additional protocol $\rho \notin \{\pi, \phi, \alpha\}$, whose definition is given in Figure 26.

Proposition 6. *ϕ and π are subroutine respecting.*

Proof. It is clear that π is subroutine respecting. We thus focus on ϕ .

For a sid $s \neq 0$, the extended session $\text{Ext}(\phi, s)$ contains no sub-party, so we focus on $(\phi, 0)$. The extended session $\text{Ext}(\phi, 0)$ can have one sub-party $(\alpha, (0, 0))$, depending on which ITI sends first an input to $(\alpha, (0, 0))$. More precisely, there are two cases:

- Case 1: if $(\alpha, (0, 0))$ is activated by an input from $(\phi, (0, 0))$, it is a sub-party of $(\phi, 0)$. We thus need to ensure that:
 - $(\alpha, (0, 0))$ does not send inputs or outputs to ITIs outside the extended session $\text{Ext}(\phi, 0)$. This is indeed the case, as $(\alpha, (0, 0))$ only sends an input to $(\phi, (0, 0))$ then halts.
 - $(\alpha, (0, 0))$ rejects any inputs or outputs from ITIs outside the extended session $\text{Ext}(\phi, 0)$. This is indeed the case, as $(\alpha, (0, 0))$ halts after its first activation.
- Case 2: if $(\alpha, (0, 0))$ is activated by an input from an ITI that is not $(\phi, (0, 0))$, we need to ensure that:
 - $(\alpha, (0, 0))$ does not send outputs to main parties of the extended session $\text{Ext}(\phi, 0)$, or input to the sub-parties of the extended session $\text{Ext}(\phi, 0)$. This is the case, as it only sends an input to $(\phi, (0, 0))$ and then halts.

Proposition 7. ϕ and π are subroutine exposing.

Proof. It is direct by construction, the exposing ITI of session (π, s) (*resp.* (ϕ, s)) being $(\pi, (s, 0))$ (*resp.* $(\phi, (s, 0))$).

Proposition 8. π UC-emulates ϕ .

Proof. We consider an execution of π with dummy adversary $\mathcal{D}$ and an environment $\mathcal{E}$. We note s_{test} the test sid of the execution. The proof is made in two steps.

Step 1: We can assume that $\mathcal{D}$ neither receives nor sends any input or backdoor.

Note that the only interactions between the protocols ϕ , π , and their adversaries occur through requests to the exposing ITI. We define an adversary $\mathcal{S}$ for ϕ by modifying $\mathcal{D}$ only in the way it handles messages related to the subroutine exposing property:

- The code of the exposing ITI $(\pi, (s_{\text{test}}, 0))$ is replaced by ϕ , both for messages from the exposing ITI to the adversary and from the adversary to the exposing ITI.
- Requests about the presence of $(\pi, (s_{\text{test}}, 0))$ in the session are replaced by requests about the presence of $(\phi, (s_{\text{test}}, 0))$.
- Requests about the presence of an ITI M with code α are not sent to the exposing ITI of $(\phi, (s_{\text{test}}, 0))$; instead, $\mathcal{S}$ simulates the response as if the request were made and the exposing ITI returned false.

It is immediate that $\mathcal{S}$ perfectly simulates the queries made by $\mathcal{D}$ to the exposing ITI $(\pi, (s_{\text{test}}, 0))$.

Furthermore, the environment $\mathcal{E}$ does not need to send any message to the dummy adversary in order to learn which ITIs belong to $\text{Ext}(\phi, s_{\text{test}})$: the existence of these ITIs is entirely determined by the input messages sent by $\mathcal{E}$.

We have therefore established that:

- $\mathcal{E}$ cannot exploit any behavioral difference between $\mathcal{D}$ and $\mathcal{S}$ to distinguish the execution of π from that of ϕ .
- $\mathcal{E}$ does not need to interact with $\mathcal{D}$, as it already knows all the information that $\mathcal{D}$ could provide.

Consequently, we may assume without loss of generality that $\mathcal{D}$ never receives or sends any input or backdoor.

Step 2: $\mathcal{E}$ cannot distinguish the execution of π or ϕ by sending inputs.

The only possible observable behavioral difference between $(\phi, (0, 0))$ and $(\pi, (0, 0))$ from the environment's perspective arises from the fact that $(\alpha, (0, 0))$ may belong to the extended session $\text{Ext}(\phi, 0)$, but never belongs to $\text{Ext}(\pi, 0)$.

In particular, in the execution of ϕ with test sid 0, the behavior of $(\alpha, (0, 0))$ upon receiving an input depends on whether it was previously invoked by $(\phi, (0, 0))$.

However, for this behavioral difference to be visible to the environment, it would have to send an input to $(\alpha, (0, 0))$. This is impossible, since the environment is restricted to sending inputs only to main parties.

N is an ITI with code $\notin \{\phi, \pi, \alpha, \rho\}$	
$(\tilde{\mathcal{E}}, 0)$	(ρ, id)
1 : <u>At first activation :</u>	1 : if $\text{id} \neq (0, 0)$ then halt
2 : send input “0” to $(\rho, (0, 0))$	2 : <u>At reception of an input :</u>
3 : <u>At activation with no message :</u>	3 : if it is first activation then
// This condition occurs if:	4 : send input “0” to $(\phi, (0, 0))$
// - An ITI deactivates itself.	5 : else halt
// - The adversary sends a message	6 : <u>At reception of an output</u>
// to a non-existent ITI.	<u>from an ITI M :</u>
// - The control function disallows	7 : if it is second activation then
// an external write.	8 : send input “0” to $(\alpha, (0, 0))$
4 : return 0	9 : else if it is third activation then
5 : <u>At reception of an output</u>	10 : send output “0” to N
<u>from an ITI M :</u>	11 : else halt
6 : return 1	

Fig. 26. Definition of ρ and $\tilde{\mathcal{E}}$ for Issue (I.2).

Proposition 9. *The protocol ρ defined in Figure 26 is (π, ϕ) -compliant.*

Proof. All conditions can directly be verified.

We have shown that ϕ and π are subroutine respecting, subroutine exposing, that π UC-emulates ϕ and that ρ is (π, ϕ) -compliant. The UC theorem of Plain UC should thus apply and ensure that $\rho^{\phi \rightarrow \pi}$ UC-emulates ρ .

Nevertheless, this conclusion is false, as shown by the next proposition.

Proposition 10. $\rho^{\phi \rightarrow \pi}$ does not UC-emulate ρ .

Proof. The environment $\tilde{\mathcal{E}}$ defined in Figure 26 can distinguish them. More precisely, in the execution of ρ with environment $\tilde{\mathcal{E}}$, any adversary and any input, $\tilde{\mathcal{E}}$ will return a uniformly random bit, whereas it will always return 1 in the execution of $\rho^{\phi \rightarrow \pi}$ with environment $\tilde{\mathcal{E}}$ any adversary and any input.

Detail of messages sent in the execution of ρ with environment $\tilde{\mathcal{E}}$, any adversary and any input:

- $\tilde{\mathcal{E}}$ sends input “0” to $(\rho, (0, 0))$.
- $(\rho, (0, 0))$ sends input “0” to $(\phi, (0, 0))$.
- If the first bit of randomness of $(\phi, (0, 0))$ is 0:
 1. $(\phi, (0, 0))$ sends input “0” to $(\alpha, (0, 0))$.
 2. $(\alpha, (0, 0))$ sends output “0” to $(\phi, (0, 0))$.
 3. $(\phi, (0, 0))$ sends output “0” to $(\rho, (0, 0))$.
 4. $(\rho, (0, 0))$ sends input “0” to $(\alpha, (0, 0))$.
 5. Because α halted, it deactivates.
 6. $\tilde{\mathcal{E}}$ is activated with no message and therefore returns 0.
- If the first bit of randomness of $(\phi, (0, 0))$ is 1:
 1. $(\phi, (0, 0))$ sends output “0” to $(\rho, (0, 0))$.
 2. $(\rho, (0, 0))$ sends input “0” to $(\alpha, (0, 0))$.
 3. It is the first activation of α by an ITI that is not $(\phi, (0, 0))$, it sends input “0” to $(\phi, (0, 0))$.
 4. $(\phi, (0, 0))$ sends output “0” to $(\rho, (0, 0))$.
 5. $(\rho, (0, 0))$ sends output “0” to an ITI N that does belong to the extended session $\text{Ext}(\rho, 0)$. The output is thus sent to $\tilde{\mathcal{E}}$, as well as extended identity N and identity $(0, 0)$.
 6. $\tilde{\mathcal{E}}$ returns 1.

Detail of messages sent in the execution of $\rho^{\phi \rightarrow \pi} = (\rho, \sigma_{\pi, \phi})$ with environment $\tilde{\mathcal{E}}$, any adversary and any input:

- $\tilde{\mathcal{E}}$ sends input “0” to $((\rho, \sigma_{\pi, \phi}), (0, 0))$.
- $((\rho, \sigma_{\pi, \phi}), (0, 0))$ sends input “(0, (Shell:main: $\rho, 0$; subr: $\pi, 0$))” to the ITI $((\pi, \sigma_{\pi, \phi}), (0, 0))$.
- $((\pi, \sigma_{\pi, \phi}), (0, 0))$ sends output “(0, (Shell:main: $\rho, 0$; subr: $\pi, 0$))” to the ITI $((\rho, \sigma_{\pi, \phi}), (0, 0))$.
- $((\rho, \sigma_{\pi, \phi}), (0, 0))$ sends input “(0, (Shell:main: $\rho, 0$; subr: $\perp$))” to the ITI $((\alpha, \sigma_{\pi, \phi}), (0, 0))$.
- $((\alpha, \sigma_{\pi, \phi}), (0, 0))$ sends input “(0, (Shell:main: $\rho, 0$; subr: $\pi, 0$))” to the ITI $((\pi, \sigma_{\pi, \phi}), (0, 0))$.
- $((\pi, \sigma_{\pi, \phi}), (0, 0))$ sends output “(0, (Shell:main: $\rho, 0$; subr: $\pi, 0$))” to the ITI $((\rho, \sigma_{\pi, \phi}), (0, 0))$.

- $((\rho, \sigma_{\pi, \phi}), (0, 0))$ wants to send output “0” to an ITI N that does belong to the extended session $\text{Ext}(\rho, \sigma_{\pi, \phi}, 0)$. The output is thus sent to $\tilde{\mathcal{E}}$, as well as extended identity N and identity $(0, 0)$.
- $\tilde{\mathcal{E}}$ returns 1.

D.3 Details Regarding Issue (I.3)

In this section, we present a comprehensive counterexample to issue (I.3).

We consider the protocols ϕ and π defined in Figure 27, the auxiliary protocols α and β defined in Figure 28, and the protocol ρ defined in Figure 29.

Proposition 11. *ϕ and π are subroutine respecting.*

Proof. The only extended session of ϕ that may contain a sub-party is $(\phi, 0)$, with sub-party $(\beta, (0, 0))$. By definition, $(\beta, (0, 0))$ rejects any message not originating from $(\phi, (0, 0))$. It follows that ϕ is subroutine respecting.

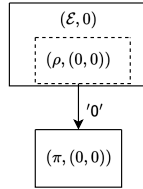
We now focus on π . The only session of π that can contain a sub-party is $(\pi, 0)$, with sub-party $(\alpha, (0, 0))$.

All other main parties $(\pi, (0, s))$, $s \neq 0$, only accept backdoor and never send any input or output, so we can focus on the study of $(\pi, (0, 0))$ and $(\alpha, (0, 0))$.

Let θ be an execution prefix – of any protocol with an environment $\mathcal{E}$ and some adversary – such that $(\alpha, (0, 0))$ is a sub-party of $(\pi, 0)$. We will show that, within θ , $(\alpha, (0, 0))$ never receives an input from any ITI other than $(\pi, (0, 0))$. This will complete the proof.

If $(\alpha, (0, 0))$ is a sub-party of $(\pi, 0)$, then the condition given in Line 4 of π ’s definition must hold. In particular, this implies that $(\pi, (0, 0))$ exists in θ and received an input “0” from some ITI with extended identity $(\rho, (0, 0))$.

However, by construction, ρ never sends inputs to $(\pi, (0, 0))$. Therefore, this situation can only arise if θ is a prefix of an execution in which the environment $(\mathcal{E}, 0)$ impersonates $(\rho, (0, 0))$ to send the input “0” to $(\pi, (0, 0))$:



In this situation, θ is therefore a prefix of an execution of the protocol π with test sid 0. By the rules governing protocol executions, $(\alpha, (0, 0))$ can only receive inputs from $(\pi, (0, 0))$, since the environment $(\mathcal{E}, 0)$ is not permitted to send inputs to the sub-party $(\alpha, (0, 0))$. This completes the proof.

Proposition 12. *ϕ and π are subroutine exposing.*

Proof. It is direct by construction, the exposing ITI of session (π, s) (*resp.* (ϕ, s)) being $(\pi, (s, 0))$ (*resp.* $(\phi, (s, 0))$).

Proposition 13. *π UC-emulates ϕ .*

$(\pi, (s, p))$	$(\phi, (s, p))$
local variable: boolean <code>alphalsSubparty</code> , initialized to false 1 : if $(s, p) \neq (0, 0) \wedge \neg \text{lsBackdoor}$ then 2 : reject 3 : <u>At first activation :</u> 4 : if it is input “0” from $(\rho, (0, 0))$ then 5 : <code>alphalsSubparty</code> $\leftarrow$ true 6 : send input “0” to $(\alpha, (0, 0))$ 7 : else reject any subsequent in/out activation 8 : <u>At second in/out activation :</u> 9 : if it is output “0” from $(\alpha, (0, 0))$ then 10 : send output “0” to $(\rho, (0, 0))$ 11 : else reject 12 : <u>For other in/out activations:</u> 13 : reject 14 : <u>At reception of a backdoor :</u> // subroutine exposing interface: 15 : if $p \neq 0$ then reject 16 : checkorreject message can be parsed as an ITI M 17 : if $M = (\pi, (s, p))$ then 18 : send backdoor “true” to adversary 19 : if $(s, p) = (0, 0) \wedge M = (\alpha, (0, 0))$ then 20 : send backdoor “ <code>alphalsSubparty</code> ” to adversary 21 : send backdoor “false” to adversary	local variable: boolean <code>betalsSubparty</code> , initialized to false if $(s, p) \neq (0, 0) \wedge \neg \text{lsBackdoor}$ then reject <u>At first activation :</u> if it is input “0” from $(\rho, (0, 0))$ then <code>betalsSubparty</code> $\leftarrow$ true send input “0” to $(\beta, (0, 0))$ else reject any subsequent in/out activation <u>At second in/out activation :</u> if it is output “0” from $(\beta, (0, 0))$ then send output “0” to $(\rho, (0, 0))$ else reject <u>For other in/out activations:</u> reject <u>At reception of a backdoor :</u> // subroutine exposing interface: if $p \neq 0$ then reject checkorreject message can be parsed as an ITI M if $M = (\phi, (s, p))$ then send backdoor “true” to adversary if $(s, p) = (0, 0) \wedge M = (\beta, (0, 0))$ then send backdoor “ <code>betalsSubparty</code> ” to adversary send backdoor “false” to adversary

Fig. 27. Definition of ϕ, π for Issue (I.3).

(α, id)	(β, id)
1 : if $\text{id} \neq (0, 0)$ then halt	1 : if $\text{id} \neq (0, 0)$ then halt
2 : <u>At first activation :</u>	2 : <u>At first activation :</u>
3 : if it is input “0” from $(\pi, (0, 0))$ then	3 : if it is input “0” from $(\phi, (0, 0))$ then
4 : send output “0” to $(\pi, (0, 0))$	4 : send output “0” to $(\phi, (0, 0))$
5 : else halt	5 : else halt
6 : <u>At second activation :</u>	6 : <u>For other activations:</u>
7 : if it is input “0” from $(\rho, (0, 0))$ then	7 : halt
8 : send output “0” to $(\rho, (0, 0))$	
9 : else halt	
10 : <u>For other activations:</u>	
11 : halt	

Fig. 28. Definition of α, β for Issue (I.3).

(ρ, id)
1 : <u>At first activation :</u>
2 : if $\text{id} \neq (0, 0)$ then halt
3 : send input “0” to $(\phi, (0, 0))$
4 : <u>At second activation :</u>
5 : if it is ouput “0” from $(\phi, (0, 0))$ then
6 : send input “0” to $(\alpha, (0, 0))$
7 : else reject any subsequent in/out activation
8 : <u>At third activation :</u>
9 : if it is ouput “0” from $(\alpha, (0, 0))$ then
// μ is a code distinct of ρ, π, ϕ
10 : send output “0” to $(\mu, (0, 0))$
11 : else halt
12 : <u>For other activations:</u>
13 : halt

Fig. 29. Definition of ρ for Issue (I.3). All messages are sent with flags forced-write and reveal-ext-id set.

Proof. We consider an execution of π with dummy adversary $\mathcal{D}$ and an environment $\mathcal{E}$. We note s_{test} the test sid of the execution.

The proof is made in two steps.

Step 1: We can assume that $\mathcal{D}$ neither receives nor sends any input or backdoor.

Observe that the only interactions between the protocols ϕ , π and their adversaries occur through requests to the exposing ITI. We construct an adversary $\mathcal{S}$ for ϕ as a variant of $\mathcal{D}$, with the only difference being that, whenever $\mathcal{D}$ sends or receives a message related to the subroutine exposing property, $\mathcal{S}$ modifies the message as follows:

- The code of the exposing ITI $(\pi, (s_{\text{test}}, 0))$ is replaced with ϕ , both for messages from the exposing ITI to the adversary and from the adversary to the exposing ITI.
- Requests about the presence of $(\alpha, (0, 0))$ in the session are replaced by requests about the presence of $(\beta, (0, 0))$.

It is immediate that $\mathcal{S}$ perfectly simulates the requests made by $\mathcal{D}$ to the exposing ITI $(\pi, (s_{\text{test}}, 0))$, since the subroutine $(\alpha, (0, 0))$ is invoked in the execution of ϕ under exactly the same conditions as $(\beta, (0, 0))$ in the execution of π .

Moreover, the environment $\mathcal{E}$ does not need to interact with the dummy adversary to determine which ITIs belong to $\text{Ext}(\phi, s_{\text{test}})$, as their existence depends solely on the input messages previously sent by $\mathcal{E}$.

We have therefore shown that:

- $\mathcal{E}$ cannot exploit any behavioral difference between $\mathcal{D}$ and $\mathcal{S}$ to distinguish the execution of π from that of ϕ .
- $\mathcal{E}$ does not need to communicate with $\mathcal{D}$, as it already knows all the information $\mathcal{D}$ could provide.

It follows that we may assume, without loss of generality, that $\mathcal{D}$ neither receives nor sends any input or backdoor.

Step 2: $(\mathcal{E}, 0)$ cannot distinguish the execution of π or ϕ by sending inputs.

The only behavioral difference that $(\mathcal{E}, 0)$ could potentially exploit lies in the second activation of the sub-parties: $(\beta, (0, 0))$ deactivates, whereas $(\alpha, (0, 0))$ sends an output to $(\rho, (0, 0))$ if it receives the input “0” from $(\rho, (0, 0))$. However, by construction, $(\rho, (0, 0))$ never sends such a message. Moreover, the environment’s impersonation ability cannot be leveraged in this context, as it is not allowed to send inputs to sub-parties.

Proposition 14. *The protocol ρ defined in Figure 29 is (π, ϕ) -compliant.*

Proof. It is direct, as ρ never writes to an ITI with code π and that the flag forced-write is always set.

We have shown that ϕ and π are subroutine respecting, subroutine exposing, that π UC-emulates ϕ and that ρ is (π, ϕ) -compliant. The UC theorem of Plain UC should thus apply and ensure that $\rho^{\phi \rightarrow \pi}$ UC-emulates ρ .

Nevertheless, this conclusion is false, as shown by the next proposition.

Proposition 15. *$\rho^{\phi \rightarrow \pi}$ does not UC-emulate ρ .*

Proof. Let us consider this environment:

```

 $(\tilde{\mathcal{E}}, 0)$ 


---


At first activation :
send input "0" to the protocol with identifier  $(0, 0)$ 
At second activation :
if it is activated with no message then
  // This condition occurs if:
  // - An ITI deactivates itself.
  // - The adversary sends a message to a non-existent ITI.
  // - The control function disallows an external write.
return 1
else return 0

```

We can see in the Figure 30 that for any adversary, $\tilde{\mathcal{E}}$ returns 0 for the execution of $\rho^{\phi \rightarrow \pi}$ and 1 for the execution of ρ .

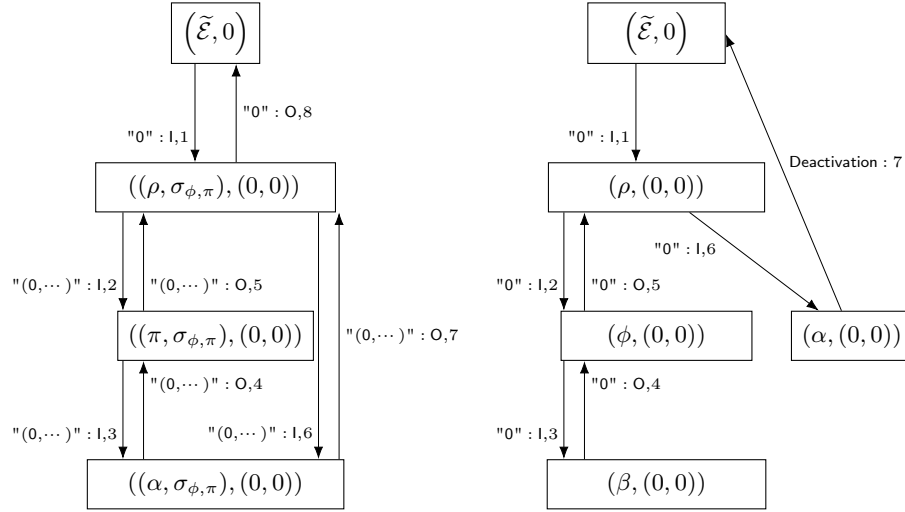


Fig. 30. I/O differentiate input from output, and the numbers indicate execution order. The left diagram shows the message sequence in the execution of $\rho^{\phi \rightarrow \pi}$ with $\tilde{\mathcal{E}}$, and the omitted right parts of messages indicated by “...” are the shell messages $\mathbf{m}_{\text{shell}}$ as discussed in the first paragraph of Section 5.2. The right diagram shows the sequence of messages in the execution of ρ with $\tilde{\mathcal{E}}$. The adversary is not used in both executions and is therefore not shown.

D.4 Details Regarding Issue (I.4)

In this section, we present a comprehensive counterexample to issue (I.4).

We consider the protocol ρ defined in Figure 31, which is constructed using an auxiliary protocol α .

Additionally, we consider any PPT protocols ϕ and π satisfying the following:

- $\phi \notin \{\pi, \rho, \alpha\}$,
- ϕ and π are subroutine respecting and subroutine exposing,
- π UC-emulates ϕ .

It is clear that ρ is (π, ϕ) -compliant.

According to the UC theorem in Plain UC, this should imply that $\rho^{\phi \rightarrow \pi}$ UC-emulates ρ . However, we will show that this is not the case: $\rho^{\phi \rightarrow \pi}$ does not UC-emulate ρ .

Proposition 16. $\rho^{\phi \rightarrow \pi}$ does not UC-emulate ρ .

Proof. We recall that the UC operator of Plain UC is defined in [11, (Section 6.1)]. Considering the environment $\tilde{\mathcal{E}}$ defined in Figure 32. We can see that, for any adversary $\mathcal{A}$:

- In the execution of $\rho^{\phi \rightarrow \pi} = (\rho, \sigma_{\pi, \phi})$ with $\tilde{\mathcal{E}}$ and the dummy adversary $\mathcal{D}$, using the definition of the replacement operator [11, (Section 6.1)], we see:
 - At reception of input 0, $((\rho, \sigma_{\pi, \phi}), (0, 0))$ sends output “(0, (**Shell:main**: $\rho, 0$; **subr**: $\perp$)))” to $((\alpha, \sigma_{\pi, \phi}), (0, 0))$.
 - At reception of this input $((\alpha, \sigma_{\pi, \phi}), (0, 0))$ deactivates.
 - $(\tilde{\mathcal{E}}, 0)$ is then activated, and sends input 0 to $((\rho, \sigma_{\pi, \phi}), (0, 1))$.
 - At reception of the input, $((\rho, \sigma_{\pi, \phi}), (0, 1))$ initializes its variables **main** to $(\rho, 0)$ and **subr** to $\perp$. The body $(\rho, (0, 1))$ is then activated with input 0 and intends to send output 0 to $(\alpha, (0, 0))$: the condition 2.(c) of the definition of UC operator [11, (Section 6.1)] is then verified, which implies that $((\rho, \sigma_{\pi, \phi}), (0, 1))$ sends output 0 to $(\alpha, (0, 0))$. Because $(\alpha, (0, 0))$ does not exist in the extended session $((\rho, \sigma_{\pi, \phi}), 0)$, this message is actually sent to $(\tilde{\mathcal{E}}, 0)$, as well as the extended identity $(\alpha, (0, 0))$ and the identity $(0, 1)$.
 - $(\tilde{\mathcal{E}}, 0)$ receives the output, and thus returns 1.
- In the execution of ρ with $\tilde{\mathcal{E}}$ and $\mathcal{A}$:
 - At reception of input 0, $(\rho, (0, 0))$ sends input 0 to $(\alpha, (0, 0))$.
 - At reception of this input, $(\alpha, (0, 0))$ deactivates.
 - $\tilde{\mathcal{E}}$ is then activated, and sends input 0 to $(\rho, (0, 1))$.
 - At reception of the input, $(\rho, (0, 1))$ sends output 0 to $(\alpha, (0, 0))$.
 - At reception this output, $(\alpha, (0, 0))$ deactivates.
 - As a result of this deactivation, $\tilde{\mathcal{E}}$ is activated with no message and returns 0.

(ρ, id)	(α, id)
1 : <u>At first activation:</u>	1 : halt
2 : if $\text{id} \notin \{(0, 0), (0, 1)\}$ then halt	
3 : if $\text{id} = (0, 0)$ then	
4 : send input “0” to $(\alpha, (0, 0))$	
5 : if $\text{id} = (0, 1)$ then	
6 : send output “0” to $(\alpha, (0, 0))$	
7 : <u>For other activations:</u>	
8 : halt	

Fig. 31. Algorithm ρ used for Issue (I.4).

$(\tilde{\mathcal{E}}, 0)$
1 : <u>At first activation:</u>
2 : send input “0” to $(\rho, (0, 0))$
3 : <u>At second activation:</u>
4 : send input “0” to $(\rho, (0, 1))$
5 : <u>At third activation:</u>
6 : if it is activated with no message then
// This condition occurs if:
// - An ITI deactivates itself.
// - The adversary sends a message to a non-existent ITI.
// - The control function disallows an external write.
return 0
7 : else return 1

Fig. 32. environment $\tilde{\mathcal{E}}$ used for Issue (I.4).

E Definition of Our Universal Composition Operator

We fix three distinct non-empty binary words: `newOperation`, `simulate`, and `apply`, which will be used throughout the construction.

Our goal is to define a new universal composition (UC) operator that satisfies Definition 13 and resolves the issues outlined in Section 5.2. In addition, we aim for the execution graph of $\rho^{\phi \rightarrow \pi}$ to closely match that of the (ϕ, ∞, π) -executions of ρ . For instance, we seek a bijective correspondence between ITIs in these two types of executions.

To this end, Sections E.1–E.3 define the construction. Section E.4 states and proves Proposition 17, asserting that the construction indeed defines a UC operator in the sense of Definition 13.

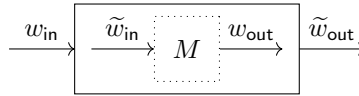
E.1 Main Ideas of the Construction and Definition of Our UC Operator

We define a UC operator on ITMs, written $\rho^{\phi \rightarrow \pi}$ for protocols ρ, ϕ, π . The definition naturally extends to parameterized systems (ρ, ϕ, π) by setting $\rho^{\phi \rightarrow \pi} = \left(\rho_k^{\phi_k \rightarrow \pi_k} \right)_k$.

We assume that ϕ and π are separated and disjoint, and that $\rho \notin \text{Pan}(\phi) \cup \text{Pan}(\pi)$.

In each extended session $\text{Ext}(\rho^{\phi \rightarrow \pi}, s_{\text{test}})$, there exists an ITI $\text{Coord}(s_{\text{test}}) = (\text{repl}[\phi \rightarrow \pi : \rho], (s_{\text{test}}, 0))$, referred to as the coordinator. The coordinator serves as a middleman, observing and modifying messages exchanged between ITIs of the extended session.

To ensure the ITIs of $\text{Ext}(\rho^{\phi \rightarrow \pi}, s_{\text{test}})$ – except for the coordinator itself – route their communication through the coordinator, they are implemented as structured protocols. These simulate a base ITI M – called the body – and may modify both incoming and outgoing external-write operations.



The structured protocols we use are of the form $\mu^{\rightleftharpoons \text{Coord}:\rho}$, where μ is the simulated code, $\text{Coord} = \text{repl}[\phi \rightarrow \pi : \rho]$ is the coordinator, and ρ, ϕ, π are the protocol to which we want to apply the UC operator.

The two upcoming subsections formally define these structured protocols and the coordinator. These two notions are sufficient to define $\rho^{\phi \rightarrow \pi}$.

Proposition 17 later defines a set S for which our operator qualifies as an S -UC Operator, as per Definition 13.

Definition 20 (Universal composition operator). *For ρ, ϕ, π such that ϕ, π are separated, disjoint and $\rho \notin \text{Pan}(\phi) \cup \text{Pan}(\pi)$, we define the protocol $\rho^{\phi \rightarrow \pi}$ as $\rho^{\rightleftharpoons \text{repl}[\phi \rightarrow \pi : \rho] : \rho}$.*

For ρ, π, ϕ parametrized systems such that ϕ, π are separated, disjoint and $\forall k \in \mathbb{N}, \rho_k \notin \text{Pan}(\phi_k) \cup \text{Pan}(\pi_k)$, we define the parametrized system $\rho^{\phi \rightarrow \pi}$ as $\left(\rho_k^{\phi_k \rightarrow \pi_k} \right)_k$.

E.2 Definition of the Structured Protocol $\mu^{\rightleftharpoons \text{Coord}:\rho}$

We define the behavior of $(\mu^{\rightleftharpoons \text{Coord}:\rho}, \text{id})$, its algorithm being given in Figure 33

We assume that id is structured as a couple (s, p) . Else, $(\mu^{\rightleftharpoons \text{Coord}:\rho}, \text{id})$ halts at first activation.

In a nutshell, $(\mu^{\rightleftharpoons \text{Coord}:\rho}, (s, p))$ first determines a $\text{sid}_{\text{coord}}$ for the coordinator, simulates the ITI $(\mu, (s, p))$ and performs the following:

- It forwards the backdoors it receives to its body $(\mu, (s, p))$.
- If $\mu = \rho$, it sends the inputs it receives to its body $(\mu, (s, p))$. If $\mu \neq \rho$ it rejects.
- It notifies $\text{Coord}(\text{sid}_{\text{coord}}) = (\text{Coord}, (\text{sid}_{\text{coord}}, 0))$ of any external-write the simulation of $(\mu, (s, p))$ intends to make. Nevertheless, it does not notify of deactivations or rejects.
- If requested by $(\text{Coord}, (\text{sid}_{\text{coord}}, 0))$, it simulates the application of any external-write w to the simulation of $(\mu, (s, p))$, provided the recipient of w is indeed $(\mu, (s, p))$.
- If requested by $(\text{Coord}, (\text{sid}_{\text{coord}}, 0))$, it applies any external-write w , provided the sender of w is indeed $(\mu^{\rightleftharpoons \text{Coord}:\rho}, (s, p))$.

The precise behavior of $(\mu^{\rightleftharpoons \text{Coord}:\rho}, (s, p))$ is described in next paragraphs and its code is given in Figure 33. Note that for an ITI M , we will denote $M^{\rightleftharpoons \text{Coord}:\rho} = (\text{code}(M))^{\rightleftharpoons \text{Coord}:\rho}, \text{id}(M)$.

Determination of the Sid $\text{sid}_{\text{coord}}$ for Coord. While $\text{sid}_{\text{coord}}$ is unset, the ITI $(\mu^{\rightleftharpoons \text{Coord}:\rho}, (s, p))$ rejects all messages, except in these two situations:

- choicesid.1 $\mu = \rho$ and $(\mu^{\rightleftharpoons \text{Coord}:\rho}, (s, p))$ receives an input. $\text{sid}_{\text{coord}}$ is then set to s . Next, the simulation of $(\mu, (s, p))$ is initialized, $(\mu^{\rightleftharpoons \text{Coord}:\rho}, (s, p))$ replaces the recipient of the input with $(\mu, (s, p))$, and simulates its sending to the simulation of $(\mu, (s, p))$.
- choicesid.2 $(\mu^{\rightleftharpoons \text{Coord}:\rho}, (s, p))$ receives an output from $(\text{Coord}, (\tilde{s}, 0))$, for some $\text{sid } \tilde{s} \in \{0, 1\}^*$. $\text{sid}_{\text{coord}}$ is then set to $\tilde{s}$. Next, the simulation of $(\mu, (s, p))$ is initialized, and $(\mu^{\rightleftharpoons \text{Coord}:\rho}, (s, p))$ proceeds as described below in the “Output reception” paragraph.

For next paragraphs, we suppose that the $\text{sid}_{\text{coord}}$ is set.

Input Reception. When $(\mu^{\rightleftharpoons \text{Coord}:\rho}, (s, p))$ receives an input and $\mu = \rho$, it replaces the recipient by $(\mu, (s, p))$ and simulates the sending of this input to the simulation of $(\mu, (s, p))$. If $\mu \neq \rho$, it rejects.

Backdoor Reception. When $(\mu^{\rightleftharpoons \text{Coord}:\rho}, (s, p))$ receives a backdoor, it replaces the recipient by $(\mu, (s, p))$ and simulates the sending of this backdoor to the simulation of $(\mu, (s, p))$.

$(\mu^{\rightleftharpoons \text{Coord}:\rho}, (\mathbf{s}, \mathbf{p}))$	
Local variable: a sid $\mathbf{s}_{\text{coord}}$, initialized to $\perp$	
1 :	<u>At reception of an external-write w that is an input from an ITI M :</u>
2 :	if $\mu \neq \rho$ then reject
3 :	if $\mathbf{s}_{\text{coord}} = \perp$ then $\mathbf{s}_{\text{coord}} \leftarrow \mathbf{s}$
4 :	$\tilde{w} \leftarrow$ the external-write w with recipient replaced by $(\mu, (\mathbf{s}, \mathbf{p}))$
5 :	SimulateOperation ($\tilde{w}$)
6 :	<u>At reception of an output “m” from an ITI $(\text{Coord}, (\tilde{\mathbf{s}}, 0))$:</u>
7 :	if $\mathbf{s}_{\text{coord}} = \perp$ then $\mathbf{s}_{\text{coord}} \leftarrow \tilde{\mathbf{s}}$
8 :	if $\mathbf{s}_{\text{coord}} \neq \perp \wedge \mathbf{s}_{\text{coord}} \neq \tilde{\mathbf{s}}$ then reject
9 :	if $\mathbf{m} = (\text{simulate}, w)$ for an external-write w with recipient $(\mu, (\mathbf{s}, \mathbf{p}))$ then
10 :	SimulateOperation (w)
11 :	if $\mathbf{m} = (\text{apply}, w)$ for an external-write w with sender $(\mu^{\rightleftharpoons \text{Coord}:\rho}, (\mathbf{s}, \mathbf{p}))$ then
12 :	apply w
13 :	reject
14 :	<u>At reception of an external-write w that is a backdoor from an ITI M :</u>
15 :	if $\mathbf{s}_{\text{coord}} = \perp$ then reject
16 :	$\tilde{w} \leftarrow$ the external-write w with recipient replaced by $(\mu, (\mathbf{s}, \mathbf{p}))$
17 :	SimulateOperation ($\tilde{w}$)
18 :	<u>In any other case :</u> reject
SimulateOperation (w)	
1 :	if the simulation of $(\mu, (\mathbf{s}, \mathbf{p}))$ has not been initialized then
2 :	initialize the simulation of $(\mu, (\mathbf{s}, \mathbf{p}))$
3 :	simulate the application of w to the simulation of $(\mu, (\mathbf{s}, \mathbf{p}))$
4 :	<u>Then simulation of $(\mu, (\mathbf{s}, \mathbf{p}))$ continues until one of the following events occurs :</u>
5 :	if the simulation of $(\mu, (\mathbf{s}, \mathbf{p}))$ rejects then reject
6 :	if the simulation of $(\mu, (\mathbf{s}, \mathbf{p}))$ deactivates (but doesn't reject) then deactivate
7 :	if the simulation of $(\mu, (\mathbf{s}, \mathbf{p}))$ wants to apply an external-write w then
8 :	send input “(newOperation, w)” to $(\text{Coord}, (\mathbf{s}_{\text{coord}}, 0))$

Fig. 33. Code of the ITM $\mu^{\rightleftharpoons \text{Coord}:\rho}$.

Output Reception. $(\mu^{\rightleftharpoons \text{Coord}:\rho}, (s, p))$ rejects any output “m” that does not originate from $(\text{Coord}, (s_{\text{coord}}, 0))$. When the sender is the ITI $(\text{Coord}, (s_{\text{coord}}, 0))$, $(\mu^{\rightleftharpoons \text{Coord}:\rho}, (s, p))$ behaves as follows:

- If m is a pair $(\text{simulate}, w)$ where w is an external-write with recipient $(\mu, (s, p))$, then $(\mu^{\rightleftharpoons \text{Coord}:\rho}, (s, p))$ simulates the application of w to the simulation of $(\mu, (s, p))$.
- If m is a pair (apply, w) where w is an external-write with sender $(\mu^{\rightleftharpoons \text{Coord}:\rho}, (s, p))$, then $(\mu^{\rightleftharpoons \text{Coord}:\rho}, (s, p))$ applies w .
- If neither of these conditions are met, $(\mu^{\rightleftharpoons \text{Coord}:\rho}, (s, p))$ rejects.

Action when the Simulation of $(\mu, (s, p))$ Rejects. If the simulation of $(\mu, (s, p))$ rejects, $(\mu^{\rightleftharpoons \text{Coord}:\rho}, (s, p))$ rejects too.

Action when the Simulation of $(\mu, (s, p))$ Deactivates But Does Not Reject. If the simulation of $(\mu, (s, p))$ deactivates without rejecting, the ITI $(\mu^{\rightleftharpoons \text{Coord}:\rho}, (s, p))$ deactivates too.

Action when the Simulation of $(\mu, (s, p))$ Intends to Make an External-Write. If the simulation of $(\mu, (s, p))$ intends to make an external-write w , $(\mu^{\rightleftharpoons \text{Coord}:\rho}, (s, p))$ sends input “(newOperation, w)” to $(\text{Coord}, (s_{\text{coord}}, 0))$.

E.3 Definition of the Coordinator $\text{Coord} = \text{repl}[\phi \rightarrow \pi : \rho]$

We describe $\text{Coord} = \text{repl}[\phi \rightarrow \pi : \rho]$, its algorithm being provided in Figures 34 and 35.

Note that $\text{repl}[\phi \rightarrow \pi : \rho]$ uses an array **ActualPosition**. This array stores information about each ITI of the execution by associating each ITI with an element of $\{\text{out}, \perp\} \cup \{0, 1\}^*$. Specifically, for an ITI M **ActualPosition**(M) is defined as:

- $\perp$ if M does not exist in the current execution prefix. This value can change if M is invoked.
- **out** if M exists but does not belong to an extended session $\text{Ext}(\pi, s)$ which replaced an extended session $\text{Ext}(\phi, s)$. This value can change because, if M sleeps, it may later belong to such an extended session.
- A sid $s \in \{0, 1\}^*$ if M belongs to an extended session $\text{Ext}(\pi, s)$ that replaced an extended session $\text{Ext}(\phi, s)$. Once set, this value cannot change as an ITI cannot “leave” an extended session.

We will use the notation $\text{Coord}(s) = (\text{Coord}, (s, 0))$.

We focus on the behavior of **Coord** during the execution of $\rho^{\phi \rightarrow \pi} = \rho^{\rightleftharpoons \text{Coord}:\rho}$ with an adversary $\mathcal{A}$ and environment $\mathcal{E}$, assuming they are fixed. Let s_{test} the test sid of this execution. In this context, $\text{Coord}(s_{\text{test}})$ acts as a middleman, capable of making the same code replacements as in the (ϕ, ∞, π) -execution of ρ because:

- Any ITI U invoked by the environment $(\mathcal{E}, 0)$ will have code $\rho^{\rightleftharpoons \text{Coord}:\rho}$ and sid s_{test} , thus choosing to communicate with $\text{Coord}(s_{\text{test}})$, as explained in Item **choicesid.1** of paragraph “Determination of the sid s_{coord} of **Coord**”, page 96.

$(\text{repl}[\phi \rightarrow \pi : \rho], (s, p))$	
Local variable: ActualPosition, initialized to out for all ITIs $M \in \text{MainParty}(\rho^{\text{repl}[\phi \rightarrow \pi : \rho]}, s)$ and to $\perp$ for all other ITIs	
1 :	if $p \neq 0$ then halt
2 :	<u>At reception of a backdoor or output message : reject</u>
3 :	<u>At reception of an input m from an ITI U :</u>
4 :	checkorreject there is an ITI M with $U = M^{\text{repl}[\phi \rightarrow \pi : \rho]}$
5 :	checkorreject $m = (\text{newOperation}, w)$ where w is an external-write
6 :	with sender M and recipient some ITI N
7 :	we note $V = N^{\text{repl}[\phi \rightarrow \pi : \rho]}$, $\text{pos}_U = \text{ActualPosition}[U]$ and $\text{pos}_V = \text{ActualPosition}[V]$
8 :	if $\text{pos}_U = \perp \wedge \text{code}(M) \neq \rho$ then reject
9 :	// Message to adversary:
10 :	if w is a backdoor then
11 :	$\tilde{w} \leftarrow w$ where sender replaced by U
12 :	send output “(apply, $\tilde{w}$)” to U
	// Message relayed to environment:
13 :	if w is an output $\wedge \text{code}(M) = \rho \wedge \text{sid}(M) = s \wedge \text{pos}_V = \perp$ then
14 :	// This is here that the replacement compatible condition (Definition 21)
15 :	// is needed. See paragraph “Outputs to Environment”, page 102
16 :	$\tilde{w} \leftarrow w$ where sender replaced by U
17 :	send output “(apply, $\tilde{w}$)” to U
	// End of the algorithm is given in Figure 35

Fig. 34. First part of code of $\text{repl}[\phi \rightarrow \pi : \rho]$.

```

(repl  $[\phi \rightarrow \pi : \rho], (s, p)$ )

    // Begin of the algorithm is given in Figure 34
    // Message with recipient code replacement  $\phi \rightarrow \pi$ :
18 : if ( $w$  is an input or output)  $\wedge$   $\text{code}(N) = \phi \wedge \text{pos}_U = \text{out}$  then
19 :    $\tilde{N} \leftarrow N$  with code replaced by  $\pi$ 
20 :    $\text{ActualPosition}[\tilde{N}^{\text{repl}[\phi \rightarrow \pi : \rho]}] \leftarrow \text{sid}(N)$ 
21 :    $\tilde{w} \leftarrow w$  with recipient replaced by  $\tilde{N}$ 
22 :   send output “(simulate,  $\tilde{w}$ )” to  $\tilde{N}^{\text{repl}[\phi \rightarrow \pi : \rho]}$ 
    // Message with sender code replacement  $\pi \rightarrow \phi$ :
23 : if  $w$  is an output  $\wedge$   $\text{code}(M) = \pi \wedge \text{pos}_U \notin \{\perp, \text{out}, \text{pos}_V\}$  then
24 :    $\tilde{M} \leftarrow M$  with code replaced by  $\phi$ 
25 :    $\tilde{w} \leftarrow w$  with sender replaced by  $\tilde{M}$ 
26 :   if  $\text{pos}_V = \perp$  then  $\text{ActualPosition}[V] \leftarrow \text{out}$ 
27 :   send output “(simulate,  $\tilde{w}$ )” to  $V$ 
    // Message from two protocols ITI with no replacement:
    // Here, we know that no external-write, deactivation or
    // reject has been asked before
28 : if  $\text{pos}_V = \perp$  then  $\text{ActualPosition}[V] \leftarrow \text{ActualPosition}[U]$ 
29 : send output “(simulate,  $w$ )” to  $V$ 

```

Fig. 35. Second part of code of $\text{repl}[\phi \rightarrow \pi : \rho]$.

- Any ITI not invoked by $(\mathcal{E}, 0)$ will be invoked by $\text{Coord}(\mathbf{s}_{\text{test}})$ and will have the form $N^{\rightleftharpoons \text{Coord}:\rho}$. This ITI will thus choose to communicate with $\text{Coord}(\mathbf{s}_{\text{test}})$, as explained in Item choicesid.2 of paragraph “Determination of the sid $\mathbf{s}_{\text{coord}}$ of Coord”, page 96.

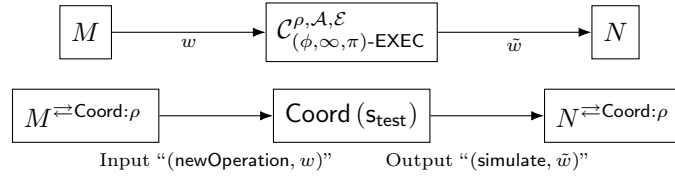
As a consequence, $\text{Coord}(\mathbf{s}_{\text{test}})$ will always know which ITIs are in the extended session $\text{Ext}(\rho^{\phi \rightarrow \pi}, \mathbf{s}_{\text{test}})$, except for the main-parties of $(\rho^{\phi \rightarrow \pi}, \mathbf{s}_{\text{test}})$ that are sleeping. This allows $\text{Coord}(\mathbf{s}_{\text{test}})$ to make the replacements of code $\phi \rightarrow \pi$, $\pi \rightarrow \phi$ that correspond to those in the (π, ∞, ϕ) -execution of ρ . However, we must impose a condition on ρ, ϕ and π to prevent a collision issue, needed in “Messages to Environment” paragraph.

Definition 21 (Replacement compatible). *A triplet of protocols (ρ, ϕ, π) is said to be replacement compatible if there is no $(\rho, \phi, \infty, \pi)$ -prefix of execution θ , sid $\mathbf{s}$ and ITI M such that $M^{\rightleftharpoons \text{Coord}:\rho} \in \text{Ext}(\rho, \mathbf{s}, \theta)$ or $\text{Coord}(\mathbf{s}) = (\text{repl}[\phi \rightarrow \pi : \rho], (\mathbf{s}, 0)) \in \text{Ext}(\rho, \mathbf{s}, \theta)$.*

A triplet of parametrized systems (ρ, ϕ, π) is said to be replacement compatible if (ρ_k, ϕ_k, π_k) is replacement compatible for any $k \in \mathbb{N}$.

Next paragraphs go into detail on how $\text{Coord}(\mathbf{s}_{\text{test}})$ behaves in each possible situation, and compare the result to the behavior observed on a (ϕ, ∞, π) -execution of ρ .

Messages Between Two Protocol ITIs. A message between two ITIs M and N of the (ϕ, ∞, π) -execution corresponds to a message between $M^{\rightleftharpoons \text{Coord}:\rho}$ and $N^{\rightleftharpoons \text{Coord}:\rho}$ with the coordinator as a middle man. The coordinator acts as the control function of the (ϕ, ∞, π) -execution regarding the replacement of codes.



Note that even if N deactivates – and thus $N^{\rightleftharpoons \text{Coord}:\rho}$ too – $\text{Coord}(\mathbf{s}_{\text{test}})$ is aware of the existence of $N^{\rightleftharpoons \text{Coord}:\rho}$ as it has been a middleman.

Messages to Adversary. Upon receiving input “(newOperation, w)” from ITI $M^{\rightleftharpoons \text{Coord}:\rho}$, w being a backdoor with sender M , $\text{Coord}(\mathbf{s}_{\text{test}})$ creates $\tilde{w}$ by replacing sender of w with $M^{\rightleftharpoons \text{Coord}:\rho}$ and sends output “(apply, $\tilde{w}$)” to $M^{\rightleftharpoons \text{Coord}:\rho}$.

As a consequence, when an ITI M sends a backdoor to the adversary in the (ϕ, ∞, π) -execution of ρ , $M^{\rightleftharpoons \text{Coord}:\rho}$ will send the same backdoor to the adversary in the execution of $\rho^{\phi \rightarrow \pi}$. Moreover, $\text{Coord}(\mathbf{s}_{\text{test}})$ is aware of M .

Messages from Adversary. Messages from the adversary are sent directly to their intended recipient. The coordinator thus does not see the messages, but this is not needed as such messages do not require modification.

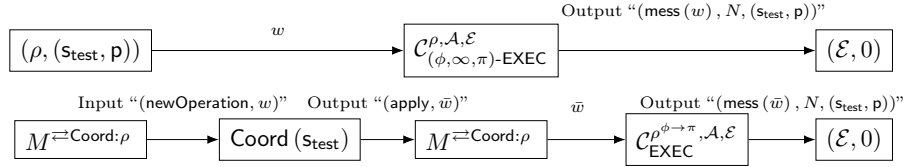
Messages from Environment. Messages from environment are sent directly to their intended recipient. The coordinator thus does not see the messages, but

this is not needed as such messages do not require modification (here, we still suppose that $\rho \notin \text{Pan}(\pi) \cup \text{Pan}(\phi)$).

Additionally, if the environment invokes a main party of (ρ, s_{test}) that deactivates, the coordinator will remain unaware of the existence of the corresponding main party of $(\rho^{\phi \rightarrow \pi}, s_{\text{test}})$, as the corresponding main party will deactivate too. However, this does not impact the knowledge of $\text{Ext}(\rho^{\phi \rightarrow \pi}, s_{\text{test}})$ by $\text{Coord}(s_{\text{test}})$ as $\text{MainParty}(\rho^{\phi \rightarrow \pi}, s_{\text{test}}) \subset \text{Ext}(\rho^{\phi \rightarrow \pi}, s_{\text{test}})$.

Outputs to Environment. The environment receives an output when a main party sends an output to a recipient not present in the extended session. In such cases, the control function sends the message, the sender's identity and the recipient's extended identity to the environment.

This rule was the source of issue (I.4) in the UC operator of Plain UC. The following figure shows how the situation is handled by $\rho^{\phi \rightarrow \pi}$. Specifically, it compares a (ϕ, ∞, π) -execution of ρ with environment $\mathcal{E}$ and an adversary (upper part) and the corresponding execution of $\rho^{\phi \rightarrow \pi}$ (lower part). A detailed description of the messages is provided after the figure.



In the (ϕ, ∞, π) execution of ρ :

An external-write w , which is an output, is made to an ITI $N \notin \text{Ext}(\rho, s_{\text{test}})$ by a main party $M = (\rho, (s_{\text{test}}, p))$. This external-write is given to the control function $\mathcal{C}_{(\phi, \infty, \pi)\text{-EXEC}}^{\rho, \mathcal{A}, \mathcal{E}}$, which decides to send output “(mess(w), N , (s_{test} , p))” to $(\mathcal{E}, 0)$.

In the corresponding execution of Coord :

- The ITI $M = (\rho, (s_{\text{test}}, p))$ simulated by the main party $M^{\Leftarrow \text{Coord}: \rho} = (\rho^{\phi \rightarrow \pi}, (s_{\text{test}}, p))$ intends to send w to an ITI N , such that $N^{\Leftarrow \text{Coord}: \rho} \notin \text{Ext}(\rho^{\phi \rightarrow \pi}, s_{\text{test}})$. $M^{\Leftarrow \text{Coord}: \rho}$ sends the input “(newOperation, w)” to the ITI $\text{Coord}(s_{\text{test}})$.
- $\text{Coord}(s_{\text{test}})$ observes that $N^{\Leftarrow \text{Coord}: \rho} \notin \text{Ext}(\rho^{\phi \rightarrow \pi}, s_{\text{test}})$ and that the message is an output from a main party of extended session $\text{Ext}(\rho, s_{\text{test}})$. It creates $\bar{w}$, which is w with sender replaced by $M^{\Leftarrow \text{Coord}: \rho}$, while keeping the receiver as N . It then sends output “(apply, $\bar{w}$)” to $M^{\Leftarrow \text{Coord}: \rho}$.
- $M^{\Leftarrow \text{Coord}: \rho}$ applies $\bar{w}$. Since (ρ, π, ϕ) is replacement compatible, we have $N \notin \text{Ext}(\rho^{\phi \rightarrow \pi}, s_{\text{test}})$, as each ITI of this extended session is of the form $U^{\Leftarrow \text{Coord}: \rho}$ for some ITI U or is $\text{Coord}(s_{\text{test}})$. Additionally, $\bar{w}$ is an output from a main party of extended session $\text{MainParty}(\rho^{\phi \rightarrow \pi}, s_{\text{test}})$. Therefore, the control function $\mathcal{C}_{\text{EXEC}}^{\rho^{\phi \rightarrow \pi}, \mathcal{A}, \mathcal{E}}$ sends output “(mess($\bar{w}$), N , (s_{test} , p))” to $(\mathcal{E}, 0)$.

The output received by $(\mathcal{E}, 0)$ is the same in both cases.

E.4 Proof that this Operator is a UC Operator

Proposition 17. *Let $S \subset \text{PPT-Param}^3$ be the subset of triplets (ρ, ϕ, π) that are replacement compatible (see Definition 21). Then, the operator $(\phi, \pi, \rho) \rightarrow \rho^{\phi \rightarrow \pi}$, constructed in Definition 20, is an S-UC Operator in the sense of Definition 13.*

Proof. By construction, we can immediately observe that if ρ, ϕ, π are PPT parametrized systems and that (ρ, ϕ, π) is a PPT triplet of parametrized systems, then $\rho^{\phi \rightarrow \pi}$ is a PPT parametrized system.

We need to show that for any replacement compatible triplets (ρ, ϕ, π) , where π and ϕ are separated, sub-party respecting, sid-exposing, disjoint, and ρ is (π, ϕ) -compliant, then $\rho^{\phi \rightarrow \pi}$ perfectly UC-emulates (ρ, ϕ, π) .

For any security parameter k , we consider the injection:

$$\begin{aligned} \text{AddStruct}_k : \text{SetITI} &\rightarrow \text{SetITI} \\ M &\mapsto M^{\overleftrightarrow{\text{repl}}[\phi_k \rightarrow \pi_k : \rho_k] : \rho_k} . \end{aligned}$$

AddStruct_k can be efficiently computed. Moreover, for any ITI M , it can efficiently be decided if $M \in \text{AddStruct}_k(\text{SetITI})$ and if so, the unique ITI N such that $M = \text{AddStruct}_k(N)$ can be efficiently found.

We can thus efficiently compute $\text{InvAddStruct}_k : \text{SetITI} \rightarrow \text{SetITI}$ defined by:

$$\text{InvAddStruct}_k(M) = \begin{cases} \perp & \text{if } M \notin \text{AddStruct}_k(\text{SetITI}) , \\ \text{The unique } N \text{ such that } \text{AddStruct}_k(N) = M & \text{else.} \end{cases}$$

For ρ, ϕ, π three parametrized systems, we define an adversary $\mathcal{D}_{(\rho, \phi, \pi)}$. We describe it for a fixed security parameter k (we recall that the security parameter is provided to the adversary at invocation, on a dedicated tape):

- When activated with an input “(ReplacedSession, (m, M))” from the environment $(\mathcal{E}, 0)$, where ReplacedSession is a subset of $\{0, 1\}^*$, m is a message, and M an extended identity, $\mathcal{D}_{(\rho, \phi, \pi)}$ computes $N = \text{InvAddStruct}_k(M)$. If $N = \perp$, $\mathcal{D}_{(\rho, \phi, \pi)}$ rejects. Else, $\mathcal{D}_{(\rho, \phi, \pi)}$ sends backdoor “m” to N .
- When activated with a message (ReplacedSession, m) on its backdoor tape, sent by an ITI M , $\mathcal{D}_{(\rho, \phi, \pi)}$ sends backdoor (m, $\text{AddStruct}_k(M)$) to the environment.

It is direct by construction that $\mathcal{D}_{(\rho, \phi, \pi)}$ is dropping : it never uses the set ReplacedSession.

Using the explanations provided in Section E.3, we can conclude that for any PPT environment $\mathcal{E}$, any security parameter k and any $z \in \{0, 1\}^*$, we have

$$\text{EXEC}_{\rho^{\phi \rightarrow \pi}, \mathcal{D}, \mathcal{E}}(k, z) = (\phi, \infty, \pi)\text{-EXEC}_{\rho, \mathcal{D}_{(\rho, \phi, \pi)}, \mathcal{E}}(k, z) .$$

This completes the proof.

F Additional Issues, Limitations, Weaknesses and Discussions

This appendix addresses additional issues, limitations, weaknesses, and discussions concerning Plain UC, with all identified problems listed in Table 4⁴⁶. They constitute supplementary material that is not necessary to understand the article but valuable in its own right.

The first two sections outline an issue and a limitation related to the UC operator and the subroutine respecting property, respectively. Despite being connected to previously discussed notions, they offer innovative insights.

Section F.3 highlights a weakness in the Plain UC UC Theorem proof, justifying the simulator modification in the Updated UC proof.

Section F.4 examines the possibility in Plain UC to specify the receiver of an external-write via a predicate rather than its extended identity. While appealing, this approach leads to multiple issues, which is why it was excluded from Updated UC.

Section F.5 explores the notion of time-complexity in Plain UC and details the flaws that warranted our use of a different concept for Updated UC.

Lastly, Section F.6 identifies a weakness tied to the balanced environment notion, excluded in Updated UC for this reason.

Table 4. All Issues, limitations and weaknesses described in the article, with a highlight on those detailed in this section.

Notions	Related Issues, limitations or weaknesses
Extended Session and Subroutine Respecting	(L.1) (I.1) (I.2) (I.3) (Section 5.1) (L.3) (Appendix F.2)
UC Operator	(I.4) (W.1) (Section 5.2) (I.5) (Appendix F.1), (I.6) (Appendix F.1)
Subroutine Exposing	(L.2) (W.2) (Section 5.3)
Compliance	(W.3) (W.4) (Section 5.4)
Use of Predicates to Designate Receiver	(I.7) (L.4) (Appendix F.4)
Time Complexity	(W.6) (W.7) (Appendix F.5)
Balanced Environment	(W.8) (Appendix F.6)
Other	(W.5) (Appendix F.3)

⁴⁶ Note that the category “other” used in the table refers to a weakness in the proof of the UC Theorem that is not directly related to any specific concept of UC. It addresses a deficiency in the definition of an adversary.

F.1 Two additional Issues Regarding the UC Operator

(I.5) Structured Protocols' sid and pid Should Match Their Body. In Plain UC, for structured protocols, such as the UC operator $\rho^{\phi \rightarrow \pi}$, it is indicated that their sid and pid to not match those of their body. For example, as described in [11, (Section 5.1)]:

The extended identity of μ represents the extended identity of the shell followed by that of the body.

However, this discrepancy presents a direct issue, which we briefly describe here. For instance, consider $(\rho, (\mathbf{s}, \mathbf{p}))$ which halts for any $(\mathbf{s}, \mathbf{p}) \neq (0, 0)$ and outputs "0" when $(\rho, (0, 0))$ receives an input. An environment sending an input to the protocol with identifier $(0, 0)$ and checking for an output could distinguish between ρ (where output is received) and $(\rho^{\phi \rightarrow \pi}, (0, 0))$ (where output is not received because it doesn't simulate $(\rho, (0, 0))$).

(I.6) The Shell-messages Lead to a Distinguisher. We can construct simple protocols ρ such that for any protocols ϕ and π distinct of ρ , $\rho^{\phi \rightarrow \pi}$ does not UC-emulate ρ . The problem arises when the environment sends a message $(\mathbf{m}_{\text{body}}, \mathbf{m}_{\text{shell}})$ to the protocol $(\rho^{\phi \rightarrow \pi} \text{ or } \rho)$ with the correct impersonation. The shell of the protocol $\rho^{\phi \rightarrow \pi}$ will interpret this as a valid message and forward $\mathbf{m}_{\text{body}}$ to its body ρ . However, ρ does not expect a shell-part $\mathbf{m}_{\text{shell}}$ and will reject the message.

We think this Issue as a large impact as it can occur with any protocols ϕ and π . In this section, we present a comprehensive counterexample for Issue (I.6).

Detailed counterexample:

We consider the protocol ρ defined in Figure 36 and a subroutine exposing subroutine respecting protocol π . Moreover, we take a subroutine exposing subroutine respecting protocol $\phi \notin \{\pi, \rho, \text{code}(N)\}$ and such that π UC-emulates ϕ .

N is an ITI with code $\neq \rho$	
(ρ, id)	
1 :	if $\text{id} \neq (0, 0)$ then halt
2 :	<u>At first activation:</u>
3 :	if it is an input with message “0” then
4 :	send output “0” to N
5 :	else halt
6 :	<u>For other activations:</u>
7 :	halt

Fig. 36. Protocol ρ used for Issue (I.6).

It is clear that ρ is (π, ϕ) -compliant and that π UC-emulates ϕ . The UC theorem of Plain UC should thus apply and ensure that $\rho^{\phi \rightarrow \pi}$ UC-emulates ρ . Nevertheless, this conclusion is false, as shown by the next proposition.

Proposition 18. $\rho^{\phi \rightarrow \pi}$ does not UC-emulate ρ .

Proof. We recall that the UC operator of Plain UC is defined in [11, (Section 6.1)]. We consider the environment $\tilde{\mathcal{E}}$ defined in Figure 37. We can see that, for any adversary $\mathcal{A}$:

- In the execution of $\rho^{\phi \rightarrow \pi} = (\rho, \sigma_{\pi, \phi})$ with $\tilde{\mathcal{E}}$ and the dummy adversary, using the definition of the replacement operator [11, (Section 6.1)], we see that:
 - At reception of input “(0, (Shell:main: $\rho, 0$; subr: $\perp$))”, the conditions 1.(a).i then 1.(b).ii of the definition of UC operator [11, (Section 6.1)] are verified and applied. As a result, the message 0 is passed to the body $(\rho, (0, 0))$, with sender identity $(\rho, (0, 1))$.
 - By definition of ρ , the body wants to send output 0 to N .
 - The condition 2.(c) of the definition of UC operator is verified and applied, leading to $(\rho, \sigma_{\pi, \phi})$ sending output 0 to N .
 - Because N does not exist in the extended session $((\rho, \sigma_{\pi, \phi}), 0)$ and that it is an output, $(\tilde{\mathcal{E}}, 0)$ receives the message 0 with the information that it was sent to N .
 - As the second activation of $(\tilde{\mathcal{E}}, 0)$ is not due to the deactivation of an ITI, $(\tilde{\mathcal{E}}, 0)$ returns 1.
- In the execution of ρ with environment $\tilde{\mathcal{E}}$ and adversary $\mathcal{A}$, $(\rho, (0, 0))$ rejects the input “(0, (Shell:main: $\rho, 0$; subr: $\perp$))” sent by $(\tilde{\mathcal{E}}, 0)$ because it is different of “0”. As a result, $(\tilde{\mathcal{E}}, 0)$ returns 0.

$(\tilde{\mathcal{E}}, 0)$ <pre style="margin: 0;"> 1 : <u>At first activation:</u> 2 : send input “(0, (Shell:main: ρ, 0; subr: $\perp$))” to the ITI with id = (0, 0) impersonating the structured ITI $((\rho, \sigma_{\pi, \phi}), (0, 1))$ 3 : <u>At second activation:</u> 4 : if it is activated with no message then // This condition occurs if: // - An ITI deactivates itself. // - The adversary sends a message to a non-existent ITI. // - The control function disallows an external write. return 0 5 : else return 1 </pre>
--

Fig. 37. Environment $\tilde{\mathcal{E}}$ used for Issue (I.6).

F.2 One additional Limitation Regarding the Subroutine Respecting property

(L.3) The Case of “Unusual Sub-parties”. Considering θ the prefix of execution of some protocol and $(\pi, \mathbf{s})$ the extended session of some protocol π in this prefix. We define an “unusual sub-party” as an ITI $M \in \text{Ext}(\pi, \mathbf{s})$ that is neither a main party nor a subsidiary of π (the notion of subsidiary is defined in [11, (Section 3.1.2, Paragraph “Subroutines, etc.”)] while the notion of sub-party is defined in [11, (Section 3.1.2, Paragraph “Protocol sessions.”)]). This can happen, for example, when M is invoked with an output by a sub-party.

In Plain UC, some extra-restrictions are given to unusual sub-parties in the subroutine respecting property [11, (Definition 19)]:

- Any input sent by an unusual sub-party to a sub-party must be rejected.
- Any output sent by an unusual sub-party to a member of the extended session must be rejected.

This limits the expressiveness of the model.

In the sub-party respecting property of Updated UC (Definition 9), “unusual parties” are not considered differently than other sub-parties.

F.3 (W.5) A Missing Case in the Simulator of the Proof of UC Theorem of Plain UC

In this section, we describe a situation where the simulator used in the proof of the UC theorem of Plain UC [11, (Theorem 22)] should reject an input from the environment instead of acting as the dummy adversary.

This issue is intended for readers familiar with Plain UC and the proof of the UC Theorem in Plain UC [11, Theorem 22]]. For simplicity, we do not consider imports.

Recall that in this proof, an adversary $\mathcal{S}$ for ρ is defined in [11, (Figure 10)].

The goal of the demonstration is to show that for any balanced, PPT environment $\mathcal{E}$, the following holds:

$$\text{EXEC}_{\rho^{\phi \rightarrow \pi}, \mathcal{D}, \mathcal{E}} \approx \text{EXEC}_{\rho, \mathcal{S}, \mathcal{E}} . \quad (7)$$

By definition, $\mathcal{S}$ simulates multiple, disjoint copies of an adversary $\mathcal{S}_\pi$ for ϕ . Moreover, when $\mathcal{S}$ receives an input (m, M) from the environment:

- By item 1.(a) of [11, (Figure 10)], $\mathcal{S}$ will request $(\phi, (\mathbf{s}, \mathbf{p}))$ from the subroutine exposing ITIs of each copy of $\mathcal{S}_\pi$.
- If none of these ITIs respond positively, and additionally, if M is not a main party of ϕ , item 1.(c).ii from [11, (Figure 10)] shows that $\mathcal{S}$ will send backdoor $\mathbf{m}$ to M .

We consider protocols ρ, π, ϕ , a balanced PPT environment $\mathcal{E}$, sids and pid $\mathbf{s}, \mathbf{p}$, and a message $\mathbf{m}$ that satisfy the following reasonable hypotheses, illustrated in Figure 38:

- All possible extended sessions of π and ϕ are disjoint. - There is a prefix θ of the execution of ρ with environment $\mathcal{E}$ and adversary $\mathcal{S}$ such that $(\mathcal{E}, 0)$ sends input $(\mathbf{m}, (\phi, \mathbf{s}, \mathbf{p}))$ to $(\mathcal{S}, \diamond)$. - At the moment $(\mathcal{E}, 0)$ sends this input, $(\phi, \mathbf{s}, \mathbf{p})$ exists in the prefix θ .

Using the description of $\mathcal{S}$ upon the reception of an input and these hypotheses, we see that when $(\mathcal{S}, \diamond)$ receives the input, it sends backdoor $\mathbf{m}$ to $(\phi, \mathbf{s}, \mathbf{p})$, which already exists in the prefix.

We now consider how the same situation is handled in the corresponding execution of $\rho^{\phi \rightarrow \pi}$ with the same environment $\mathcal{E}$ and dummy adversary $\mathcal{D}$.

By the definition of $\rho^{\phi \rightarrow \pi}$ and because all possible extended sessions of π and ϕ are disjoint, when $\mathcal{E}$ sends input $(\phi, \mathbf{s}, \mathbf{p})$ to $\mathcal{D}$, we know that $(\phi, (\mathbf{s}, \mathbf{p}))$ doesn't exist, as illustrated in Figure 39.

The backdoor $\mathbf{m}$ that $(\mathcal{D}, \diamond)$ will try to send to $(\phi, (\mathbf{s}, \mathbf{p}))$ will thus be rejected by the control function, as it is sent with **forced-write** unset.

To summarize, we identified a common situation where:

- In the execution of ρ with $\mathcal{E}$ and $\mathcal{S}$, $(\phi, (\mathbf{s}, \mathbf{p}))$ receives a backdoor.
- In the execution of $\rho^{\phi \rightarrow \pi}$ with $\mathcal{E}$ and $\mathcal{D}$, the control function disallows the sending of the corresponding backdoor.

For a well-chosen ϕ , this contradicts Equation (7).

As a consequence, we see that $\mathcal{S}$ should reject in such a situation. In the proof of the UC Theorem of Updated UC (Appendix C), these situations are taken into account.

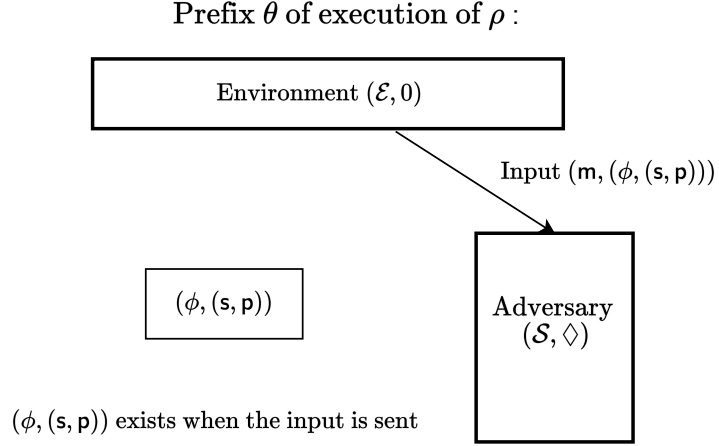


Fig. 38. Illustration of a prefix θ that satisfies the hypotheses given in the description of Weakness (W.5).

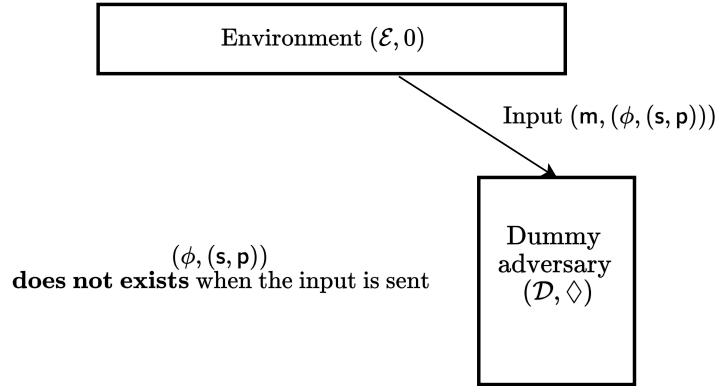


Fig. 39. Illustration of an execution of $\rho^{\phi \rightarrow \pi}$ with $\mathcal{E}$ and $\mathcal{D}$, corresponding to θ . This figure is used in the description of Weakness (W.5).

F.4 Problems Related to Predicates

In Plain UC, since the revision [10], a message sent with flag `forced-write` unset may designate its recipient using a predicate on ITIs instead of an extended identity [11, (Section 3.1.2)].

This predicate is evaluated by the control function to determine the recipient's extended identity. If multiple ITIs in the execution match the predicate, the first one invoked is selected. If no ITI matches, the environment is activated with no message, as though the sender had performed a deactivation.

This feature can be of interest to support cases where the sender has only partial information about the recipient. To our knowledge, both the adversary and protocol ITIs may use this mechanism during protocol execution: the adversary always unsets `forced-write`, and protocol ITIs are not required to set it [11, (Section 4.1)].

However, we show in this section that this mechanism leads to several issues. Specifically:

- The first subsection identifies a problem arising from the UC operator's incompatibility with predicate-based addressing.
- The second subsection presents a pair of protocols π and ϕ such that π UC-emulates ϕ against adversaries that do not use predicates, but fails to UC-emulate ϕ when adversaries are permitted to use them.
- The third subsection assumes a naïve fix to the problem discussed in the first subsection and introduces another counterexample to the UC Theorem, this time involving protocols that use predicates.

To account for these counterexamples, this section departs from our earlier conventions and permits protocol ITIs to unset the flag `forced-write` for outputs.

(I.7) Non-Adaptation of UC Operator to Predicates. The definition of the UC operator assumes that the ITIs use extended identities when they want to send a message. This can for example be seen in the item 2 of [11, (Section 6.1)]. As a result, the operator cannot be applied to protocols that use predicates.

(L.4) UC-Emulation With Adversary Able to Use Predicates. To our knowledge, the vast majority of adversaries considered in the proofs of UC-emulation are not allowed to use predicates.

In this section, we show with a counterexample that the UC-emulation for such adversaries does not imply the UC-emulation for adversaries able to use predicates.

The idea of the counterexample is to use predicates that hide the extended identity the adversary will write to.

We define π and ϕ in Figure 40, their subroutines (respectively α and β) are defined on Figure 41.

In a nutshell:

- π is such that if $(\pi, (0, 0))$ receives as input a bit b , it activates $(\alpha, (0, 0))$ first, then $(\alpha, (0, 1))$.

- ϕ is such that if $(\phi, (0, 0))$ receives as input a bit b , it activates $(\beta, (0, b))$ first, then $(\beta, (0, 1 - b))$.
- For $u \in \{0, 1\}$, if the second message received by $(\alpha, (0, u))$ (*resp.* $(\beta, (0, u))$) is a backdoor, it sends output “0” to $(\pi, (0, 0))$ (*resp.* $(\phi, (0, 0))$).
- For $u \in \{0, 1\}$, if the third message received by $(\pi, (0, 0))$ (*resp.* $(\phi, (0, 0))$) is “0” from $(\alpha, (0, u))$ (*resp.* $(\beta, (0, u))$), then it sends output “u” to an ITI N with code $\notin \{\phi, \psi, \alpha, \beta\}$.

Proposition 19. π and ϕ are subroutine respecting.

Proof. The proof is straightforward for both protocols. We briefly discuss π . Only $(\pi, (0, 0))$ can have subroutine that are $(\alpha, (0, 0))$ and $(\alpha, (0, 1))$, these subroutines only send outputs to $(\pi, (0, 0))$ and accept input or output only from $(\pi, (0, 0))$. Moreover, $(\pi, (0, 0))$ accepts outputs only from these subroutines.

Proposition 20. π and ϕ are subroutine exposing.

Proof. It is direct by construction, the exposing ITI of extended sessions (π, s) (*resp.* (ϕ, s)) being $(\pi, (s, 0))$ (*resp.* $(\phi, (s, 0))$).

Proposition 21. π UC-emulates ϕ .

Proof. If we consider an adversary $\mathcal{A}$ for the execution of ϕ that always designates the recipient by its extended identity, we can easily create a corresponding adversary $\mathcal{S}$ for the execution of π that ensure UC-emulation.

$\mathcal{S}$ simulates $\mathcal{A}$ and when $\mathcal{A}$ wants to write a backdoor to $(\alpha, (0, u))$ ($u \in \{0, 1\}$), $\mathcal{S}$ pass the message to $(\beta, (0, u))$.

Proposition 22. π does not UC-emulate ϕ if we allow the adversary to use predicates.

Proof. We construct an adversary $\mathcal{A}$ and an environment $\tilde{\mathcal{E}}$.

$\mathcal{A}$ is such that when $(\mathcal{A}, \diamond)$ receives an input from the environment, it sends a backdoor to an ITI by using the predicate “code is α ”.

$\tilde{\mathcal{E}}$ acts as following:

- $(\tilde{\mathcal{E}}, 0)$ samples a random bit $b \leftarrow_{\$} \{0, 1\}$ and sends b to the protocol with identifier $(0, 0)$.
- At reception of an output, $(\tilde{\mathcal{E}}, 0)$ sends an input 0 to $(\mathcal{A}, \diamond)$.
- The next time that $(\tilde{\mathcal{E}}, 0)$ is activated. It returns 0 if it receives output b , and it returns 1 else.

We note that, in the execution of $(\pi, (0, 0))$ with adversary $\mathcal{A}$ and environment $\tilde{\mathcal{E}}$, $(\tilde{\mathcal{E}}, 0)$ will always output 0. Indeed, the predicate used by $(\mathcal{A}, \diamond)$ is both respected by $(\alpha, (0, 0))$ and $(\alpha, (0, 1))$. As a consequence, the recipient will be the earliest ITI invoked among them, which is $(\alpha, (0, b))$. $\tilde{\mathcal{E}}$ will thus receive output b and return 0.

Now, we claim that it is impossible to construct an adversary $\mathcal{S}$ such that the execution of ϕ with $\mathcal{S}$ and $\tilde{\mathcal{E}}$ will always return 0. Indeed, the input b sent by $\tilde{\mathcal{E}}$ is independent of any information that $\mathcal{S}$ can obtain, even using predicates on the extended identity of ITIs. $\tilde{\mathcal{E}}$ will thus return 0 with probability 1/2.

(π, id)	(ϕ, id)
Local variable: boolean GoodInput initialized to true	Local variable: boolean GoodInput initialized to true
1 : if $\text{id} \neq (0, 0) \wedge \neg \text{lsBackdoor}$ then	1 : if $\text{id} \neq (0, 0) \wedge \neg \text{lsBackdoor}$ then
2 : reject	2 : reject
3 : <u>At first activation :</u>	3 : <u>At first activation :</u>
4 : if it is an input “b”, $b \in \{0, 1\}$ then	4 : if it is an input “b”, $b \in \{0, 1\}$ then
5 : $\text{bit} \leftarrow b$	5 : send input “0” to $(\beta, (0, 0))$
6 : send input “0” to $(\alpha, (0, \text{bit}))$	6 : else then
7 : else then	7 : GoodInput $\leftarrow$ false
8 : GoodInput $\leftarrow$ false	8 : reject any subsequent
9 : reject any subsequent	in/out activation
in/out activation	9 : <u>At second in/out activation :</u>
10 : <u>At second in/out activation :</u>	10 : if it is output “0”
11 : if it is output “0”	from $(\beta, (0, 0))$ then
from $(\alpha, (0, \text{bit}))$ then	11 : send input “0” to $(\beta, (0, 1))$
12 : send input “0” to $(\alpha, (0, 1 - \text{bit}))$	12 : else reject any subsequent
13 : else reject any subsequent	in/out activation
in/out activation	13 : <u>At third in/out activation :</u>
14 : <u>At third in/out activation :</u>	14 : if it is output “0”
15 : if it is output “0”	from $(\beta, (0, u)), u \in \{0, 1\}$ then
from $\alpha, (0, u)), u \in \{0, 1\}$ then	15 : send output “u” to N
16 : send output “u” to N	16 : else reject
17 : else reject	17 : <u>For other in/out activations:</u> reject
18 : <u>For other in/out activations:</u> reject	18 : <u>At reception of a backdoor :</u>
19 : <u>At reception of a backdoor :</u>	// subroutine exposing interface:
// subroutine exposing interface:	19 : if $p \neq 0$ then reject
20 : if $p \neq 0$ then reject	20 : checkorreject message can be
21 : checkorreject message can be	parsed as an ITI M
parsed as an ITI M	21 : if $M = (\phi, \text{id})$ then
22 : if $M = (\pi, \text{id})$ then	22 : send backdoor “true” to adversary
23 : send backdoor “true” to adversary	23 : if $\text{id} = (0, 0)$
24 : if $\text{id} = (0, 0)$	$\wedge M \in \{(\beta, (0, 0)), (\beta, (0, 1))\}$ then
$\wedge M \in \{(\alpha, (0, 0)), (\alpha, (0, 1))\}$ then	24 : send backdoor “GoodInput”
25 : send backdoor “GoodInput”	to adversary
to adversary	25 : send backdoor “false” to adversary
26 : send backdoor “false” to adversary	

Fig. 40. Codes of ϕ and π for Limitation (L.4). The relevant differences are highlighted. All messages are sent with reveal-sender-id and forced-write set. N is an ITI with code $\notin \{\phi, \psi, \alpha, \beta\}$.

All messages are sent with reveal-sender-id and forced-write set reveal-sender-id and forced-write set	
$(\alpha, (s, p))$	$(\beta, (s, p))$
1 : if $(s, p) \notin \{(0, 0), (0, 1)\}$ then halt	1 : if $(s, p) \notin \{(0, 0), (0, 1)\}$ then halt
2 : <u>At first activation :</u>	2 : <u>At first activation :</u>
3 : if it is input “0” from $(\pi, (0, 0))$ then	3 : if it is input “0” from $(\phi, (0, 0))$ then
4 : send output “0” to $(\pi, (0, 0))$	4 : send output “0” to $(\phi, (0, 0))$
5 : else halt	5 : else halt
6 : <u>At second activation :</u>	6 : <u>At second activation :</u>
7 : if it is a backdoor then	7 : if it is a backdoor then
8 : send output “0” to $(\pi, (0, 0))$	8 : send output “0” to $(\pi, (0, 0))$
9 : else halt	9 : else halt
10 : <u>For any other activation :</u>	10 : <u>For any other activation :</u>
11 : halt	11 : halt

Fig. 41. Codes of α and β for Limitation (L.4).

Using Predicates On Protocols. Issue F.4 (page 110) demonstrated that the UC operator must be adapted for protocols using predicates. In this section, we consider a “naive” adaptation of the UC operator and examine some of its concerns.

We will assume in the definition of $\rho^{\phi \rightarrow \pi}$, that when the body of a structured protocol of an extended session of $\rho^{\phi \rightarrow \pi}$ uses a predicate P , the structured protocol replaces it by a predicate $\tilde{P}$. The modified predicate is satisfied for structured protocols $((\mu, \sigma_{\pi, phi}), id)$ such that (μ, id) satisfies P .

With this solution, we will construct a counterexample to the UC theorem. We believe this example demonstrates that adding support for predicates to the UC operator must be accompanied by new constraints on the protocols ϕ, π and ρ . These constraints could, for instance, be incorporated into the notion of compliance.

We will create three protocols ϕ, π and ρ such that:

- ϕ and π are subroutine respecting and subroutine exposing.
- ϕ UC-emulates π .
- ρ is (ϕ, π) -compliant.
- The adaptation of UC operator to predicates is such that $\rho^{\phi \rightarrow \pi}$ does not UC-emulate π .

The definitions of ϕ and π are given on Figure 42. In a nutshell, when $(\pi, (0, 0))$ (*resp.* $(\phi, (0, 0))$) receives an input 0 from the environment, $(\pi, (0, 0))$ (*resp.* $(\phi, (0, 0))$) will go back and forth two times with its only subroutine $(\alpha, (0, 0))$ (*resp.* $(\beta, (0, 0))$) with four distinct messages:

- An input 0 from $(\pi, (0, 0))$ to $(\alpha, (0, 0))$ (*resp.* $(\phi, (0, 0))$ to $(\beta, (0, 0))$).
- An output 0 from $(\alpha, (0, 0))$ to $(\pi, (0, 0))$ (*resp.* $(\beta, (0, 0))$ to $(\pi, (0, 0))$).
- An output 1 from $(\pi, (0, 0))$ to $(\alpha, (0, 0))$ (*resp.* $(\phi, (0, 0))$ to $(\beta, (0, 0))$).
- An output 0 from $(\alpha, (0, 0))$ to $(\pi, (0, 0))$ (*resp.* $(\beta, (0, 0))$ to $(\pi, (0, 0))$).

Then, $(\pi, (0, 0))$ (*resp.* $(\phi, (0, 0))$) halts.

Nevertheless, while $(\phi, (0, 0))$ sends the output 1 to $(\alpha, (0, 0))$ by indicating $(\beta, (0, 0))$ as recipient, $(\pi, (0, 0))$ uses instead a predicate ζ to designate $(\alpha, (0, 0))$, where $\zeta(M) = (\text{code}(M) \neq \pi) \wedge ((\text{sid}(M), \text{pid}(M)) = (0, 0))$. This will form the basis of the counterexample, as this predicate will only hold for $(\alpha, (0, 0))$ in the execution of π with test sid 0, but will apply to other ITIs when executed in $\rho^{\phi \rightarrow \pi}$.

Finally, we insist that the third message is an output: this will be necessary to ensure that π is subroutine respecting (Proposition 23).

Proposition 23. *π and ϕ are subroutine respecting.*

Proof. This is direct for ϕ , and for π , we note that the only difficulty is that, in some executions, $(\pi, (0, 0))$ could send a message to an ITI different of $(\alpha, (0, 0))$ as the predicate ζ could be verified by another ITI invoked before. Nevertheless, as this message is an output, the subroutine respecting property is respected.

Proposition 24. *π and ϕ are subroutine exposing.*

(α, id)	(β, id)
1 : if $\text{id} \neq (0, 0)$ then halt	1 : if $\text{id} \neq (0, 0)$ then halt
2 : <u>At first and second activations :</u>	2 : <u>At first and second activations :</u>
3 : if it is input “0” from $(\pi, (0, 0))$ then	3 : if it is input “0” from $(\phi, (0, 0))$ then
4 : send output “0” to $(\pi, (0, 0))$	4 : send output “0” to $(\phi, (0, 0))$
5 : else halt	5 : else halt
6 : <u>For other activations:</u>	6 : <u>For other activations:</u>
7 : halt	7 : halt

Fig. 43. Codes of subroutines α and β used respectively by π and ϕ for concern of Section F.4. The code of π and ϕ is provided in Figure 42. All messages are sent with reveal-sender-id and forced-write set.

Proof. It is direct by construction, the exposing ITI of session (π, s) (*resp.* (ϕ, s)) being $(\pi, (s, 0))$ (*resp.* $(\phi, (s, 0))$).

Proposition 25. π UC-emulates ϕ .

Proof. We consider the execution of π with dummy adversary $\mathcal{D}$ and some environment $\mathcal{E}$. The test sid is denoted by s_{test} .

The proof is made in two steps.

Step 1: we can suppose that $\mathcal{D}$ never receives any input or backdoor.

Note that the only communications between protocols ϕ, π and their adversaries are through the requests to the exposing ITI. We create $\mathcal{S}$, adversary of ϕ , as $\mathcal{D}$, with the only difference being that when $\mathcal{D}$ sends or receives a message linked to the subroutine exposing property, $\mathcal{S}$ modifies the message as follows:

- The code of the exposing ITI $(\pi, (s_{\text{test}}, 0))$ is replaced by ϕ , both for messages from the exposing ITI to the adversary and from the adversary to the exposing ITI.
- Requests about the presence of $(\alpha, (0, 0))$ in the session, are replaced by requests about the presence of $(\beta, (0, 0))$.

We can directly see that $\mathcal{S}$ simulates perfectly the requests made through $\mathcal{D}$ to the exposing ITI $(\pi, (s_{\text{test}}, 0))$ because the subroutine $(\alpha, (0, 0))$ is invoked in the execution of ϕ exactly under the same conditions as $(\beta, (0, 0))$ in the execution of π .

Moreover, $\mathcal{E}$ does not need to send messages to the dummy adversary in order to know which ITIs are on the extended session $\text{Ext}(\phi, s_{\text{test}})$: the existence of the ITIs depends solely on the inputs $\mathcal{E}$ sent.

We have thus shown that:

- $\mathcal{E}$ cannot exploit a difference of behavior between $\mathcal{D}$ and $\mathcal{S}$ to distinguish the execution of π from ϕ .

- $\mathcal{E}$ does not need to send messages to $\mathcal{D}$, as it already knows the information $\mathcal{D}$ would give.

Step 2: $(\mathcal{E}, 0)$ cannot distinguish the execution of π or ϕ by sending inputs.

Moreover, the only difference of behavior between π and ϕ that could detect the environment is the fact that π uses a predicate on the second input it sends (Line 11). Nevertheless, in an execution of π with any environment and adversary, this predicate will always designate the ITI $(\alpha, (0, 0))$.

The definition of ρ is given in Figure 44.

(ρ, id)
1 : if $\text{id} \neq (0, 0)$ then halt
2 : <u>At first activation :</u>
3 : send input “0” to $(\phi, (0, 0))$
4 : <u>At second activation :</u>
5 : if it is an output “1” then
6 : send output “1” to N
7 : else halt
8 : <u>For other activations :</u>
9 : halt

Fig. 44. Code of ρ for concern of Section F.4. Note that all messages are sent with reveal-sender-id and forced-write set. Moreover, N is an ITI with code $\notin \{\phi, \pi, \alpha, \rho\}$.

Proposition 26. ρ is (π, ϕ) -compliant.

Proof. It is direct, as ρ never writes to an ITI with code π and that the flag forced-write is always set when sending inputs.

Proposition 27. With the proposed adaptation of the UC operator to support predicates, $\rho^{\phi \rightarrow \pi}$ does not UC-emulate ρ .

Proof. Consider an environment $\tilde{\mathcal{E}}$ that sends any message to the protocol with identifier $(0, 0)$. We can observe the following:

- In the execution of ρ , $\tilde{\mathcal{E}}$ will be activated due to the deactivation of an ITI.
- In the execution of $\rho^{\phi \rightarrow \pi}$, $\tilde{\mathcal{E}}$ will receive the output “1”. This is because when $(\pi, (0, 0))$ sends output “1” using the predicate $\zeta(M) = (\text{code}(M) \neq \pi) \wedge ((\text{sid}(M), \text{pid}(M)) = (0, 0))$, the first invoked ITI that satisfies this condition is $(\rho, (0, 0))$, which is invoked before $(\alpha, (0, 0))$. As a result, $(\rho, (0, 0))$ receives the output “1” and sends output “1” to the environment.

F.5 Problems Related to the Notion of T -Bounded Protocols

In this section, we review the definition of a T -bounded protocol and discuss several aspects, including one that introduces a weakness in the proof of the UC Theorem in Plain UC. We note that other works—such as [21, Section 3]—offer valuable discussions of this notion. However, the points we raise here are distinct from those.

Definition of a T -Bounded Protocol. The notion of T -bounded protocol is defined in [11, (Section 3.2)] and discussed in [11, (Section 3.3.4)]. We briefly recall it here.

To account for resource-bounded computation in systems of ITM, each message sent by any ITM is a pair “ (i, m) ”, where m is the actual message and $i \in \mathbb{N}$ a number called its import.

The import can be thought of as a number of “tokens” that an ITI can “consume” in exchange for computation time or transfer to another ITI. In the formal definition provided below, the number of tokens an ITI has at a given moment is called its total budget.

More formally, let’s recall the definition of locally T -bounded, T -bounded and PPT bounded protocols, where T is a function $T : \mathbb{N} \rightarrow \mathbb{N}$.

Definition 22 (T-bounded [11, (Definition 6)]). *Recall that the import of a message is the value written in the import field of the message. Let $T : \mathbb{N} \rightarrow \mathbb{N}$. An ITM μ is locally T -bounded if, at any prefix θ of an execution of a system of ITMs, any ITI M with program μ satisfies the following conditions:*

- *Budget(M, θ) ≥ 0 , where Budget(M, θ) = $n_I - n_O$, n_I is the overall imports of the messages written by other machines on the externally writable tapes of M , and n_O is the overall imports of the messages written by M on externally writable tapes of other machines.*
- *The overall number of computational steps taken by M so far is at most $T(\text{Budget}(M, \theta))$.*

T -bounded ITMs are defined inductively: A locally T -bounded ITM that does not make external-writes with forced-write is T -bounded. In addition, a locally T -bounded ITM, all of whose external-writes with forced-write specify a recipient ITM which is T -bounded, is T -bounded as well. ITM μ is PPT if there exists a polynomial P such that μ is P -bounded. A protocol is PPT if it is PPT as an ITM.

Problems with 0-Import Messages. For any function $T : \mathbb{N} \rightarrow \mathbb{N}$ and number u such that $T(u)$ is high enough, we can create a system of ITMs that is T -bounded and whose execution with an initial message of import u contains more than $T(u)$ distinct ITIs.

This gives a counterexample of [11, (Proposition 7)], as its proof shows that this number should be bounded by $T(u)$ if T is also super-additive and increasing.

In the next paragraphs, we will present this counterexample, propose a modification of the definition of T-bounded ITI and prove a variant of [11, (Proposition 7)] with this new definition.

The Counterexample:

We fix $u \in \mathbb{N}$. We will use two constants $\tau, t_u > 0$ that will be fixed at the end. We consider a protocol $\mu = \mu(u, \tau, t_u)$ that acts as follows. For any identifier id and any activation:

- If its identifier id cannot be interpreted as a number $0 \leq \text{id} < t_u + \tau$, (μ, id) halts.
- If its import is not u , (μ, id) halts.
- If the two first conditions are verified, (μ, id) invokes $(\mu, \text{id} + 1)$ with input “0” and import u , then halts.

An execution of $(\mu, 0)$ is shown in Figure 45.

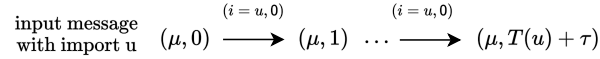


Fig. 45. Note that all messages are invocations.

As the reading and writing of t_u, id, τ and u is linear in the size of id and the values $\log_2(t_u), \log_2(u)$, if t_u is high enough, we can find a carefully chosen $\tau > 0$ and $t_u > 0$ such that each ITI with code μ will take less than t_u operations in any execution. As a consequence, μ is T -bounded for any function T with $T(u) = t_u$.

Nevertheless, by construction, the execution of $(\mu, 0)$ contains $T(u) + \tau + 1$ ITIs.

First Try to a Solution:

We could avoid this counterexample by forbidding any ITI with budget u to send messages with forced-write flag set and import u – in particular, it would not be able to invoke an ITI using all its budget.

Nevertheless, more subtle counterexamples could still be created, where (μ, id) invokes (μ, id) with import $u - 1$, and only gives later the last token it kept to (μ, id) , as shown in Figure 46.

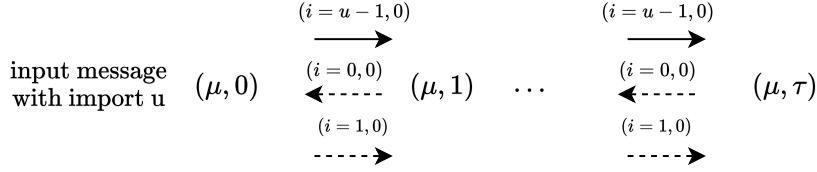


Fig. 46. The three messages are sent from up to bottom. The dotted arrows represent messages with forced-write flag unset, solid arrows are invocations.

A Solution:

We propose some additions to the T -bounded notion that aims to forbid the two counterexamples we described.

Definition 23 (T-bounded, revisited). *An ITM is locally T -bounded in our new sense if it is locally T -bounded as in Definition 22 and if, in addition, in any prefix of execution of a system of ITMs, any ITI M with code μ satisfies the following conditions:*

- *At any moment, if its budget is strictly positive, M does not send a message with import equal to its budget.*
- *At any moment, if its budget is 0, M does not send a message with flag forced-write set – in particular, it cannot invoke ITIs.*

T -bounded ITMs are defined inductively: A locally T -bounded ITM that does not make external-writes with forced-write is T -bounded. In addition, a locally T -bounded ITM, all of whose external-writes with forced-write specify a recipient ITM which is T -bounded, is T -bounded as well. ITM μ is PPT if there exists a polynomial P such that μ is P -bounded. A protocol is PPT if it is PPT as an ITM.

We now prove a variant of the claims of [11, (Proposition 7)] related to the execution time of systems of ITIs and made in its proof.

Recall that a function $T : \mathbb{N} \rightarrow \mathbb{N}$ is super additive if, for any $n, n' \in \mathbb{N}$, $T(n + n') \geq T(n) + T(n')$.

Proposition 28 ([11, (Proposition 7)], revisited). *Let $T : \mathbb{N} \rightarrow \mathbb{N}$ a super additive, increasing function and a system of ITMs (I, C) such that the initial ITM is T -bounded in the sense of Definition 23. We consider any prefix of execution θ of (I, C) where the import of the initial input is $n \in \mathbb{N}$. Then:*

1. *The time of execution of θ , defined as the number of operation made by each ITI in θ , is bounded by $T(n)(1 + T(0))$.*
2. *The total number of ITIs of θ is bounded by $T(n) + 1$.*

Proof. Note that at any moment of the execution, the sum of budgets of each of the active ITIs is equal to n .

Let $\mathcal{M}_{\geq 1}^t$ the set of ITIs that exists in θ , with a budget ≥ 1 and $\mathcal{M}_{=0}^t$ the set of ITIs that exists in θ , with budget 0 (note that budget is always ≥ 0).

For each $N \in \mathcal{M}_{\geq 1}^t \sqcup \mathcal{M}_{=0}^t$, we note $\text{Budget}(N, \theta)$ its budget.

We have:

$$\sum_{N \in \mathcal{M}_{\geq 1}^t} \text{Budget}(N, \theta) = n.$$

Moreover, because all theses ITIs are locally T -bounded and that T is super-additive and increasing, we deduce that, in θ , the added execution time of all the ITMs of $\mathcal{M}_{\geq 1}^t$ is bounded by $T(n)$. Indeed:

$$\begin{aligned} \sum_{N \in \mathcal{M}_{\geq 1}^t} T(\text{ExecTime}(N, \theta)) &\leq \sum_{N \in \mathcal{M}_{\geq 1}^t} T(\text{Budget}(N, \theta)) \\ &\leq T\left(\sum_{N \in \mathcal{M}_{\geq 1}^t} \text{Budget}(N, \theta)\right) = T(n). \end{aligned}$$

We now study the addition of the execution time of all ITMs of $\mathcal{M}_{=0}^t$.

First note there are only two possibilities for an ITM N to be in $\mathcal{M}_{=0}^t$:

- $n = 0$ and it is the initial ITI. The results of the proposition are thus direct, as there is only the initial ITI in the system.
- $n > 0$, N has been invoked with import 0 and never received any messages with an import different of 0.

We focus on the second case. Note that an ITI of $\mathcal{M}_{=0}^t$ can only have been invoked by an ITI of $\mathcal{M}_{\geq 0}^t$. As a result, we can roughly upper-bound the number of ITIs of $\mathcal{M}_{=0}^t$ by $T(n)$, as we have shown that the added execution time of all ITIs of $\mathcal{M}_{\geq 1}^t$ is bounded by $T(n)$. We obtain that the addition of the execution time of the ITIs of $\mathcal{M}_{=0}^t$ is bounded by $T(0)T(n)$.

The sum of time of execution of all the ITIs of the system, at time t , is thus bounded by $T(n)(1 + T(0))$. This proves the first point of the proposition.

We now prove the second point. At any time t , except the initial ITI, each ITI of the system has been invoked by an ITI of $\mathcal{M}_{\geq 1}^t$. As we have previously shown that the total time of execution of each ITI of $\mathcal{M}_{\geq 1}^t$ is bounded than $T(n)$, this implies that the total number of ITI of the system is bounded by $T(n) + 1$.

(W.6) The Case of Protocol Execution. The definition of T -bounded ITI is based on the execution of systems of ITMs, where no impersonation occurs. However, UC-emulation takes place in the context of a protocol execution, which is an extended system where the environment can perform impersonation.

As a result, it is possible to create a protocol π that is T -bounded but for which the execution time can become infinite under certain environment and adversaries.

For example, lets ν an ITM that halts upon its first activation. We construct π such that, for any identity id , (π, id) rejects any message that does not originate

from the ITI $(\nu, (0, 0))$. However, if the message does come from $(\nu, (0, 0))$, π enters an infinite loop.

π is T -bounded because, in any execution of a system of ITMs, it never receives any message from $(\nu, (0, 0))$. However, during the protocol execution of π , an environment can send an input to π by impersonating $(\nu, (0, 0))$, triggering the infinite loop.

The solution of this problem is to consider the execution of protocols instead of the execution of systems of ITMs. Nevertheless, this improvement is not enough to deal with structured-protocols, as shown in the next section.

(W.7) The Case of Structured Protocols. In the previous section, we established that the definition of T -bounded protocols should be considered in the context of protocol executions rather than systems of ITMs. From now on, we assume this adjustment is in place.

In this section, we demonstrate that this improvement is still insufficient to handle structured protocols, which are used, for example, by the replacement protocol. Specifically, we cannot conclude that a structured protocol is T -bounded simply because both its body and shell are. This can affect several constructions in Plain UC that rely on structured protocols, such as the universal composition operator.

For instance, consider μ a locally T -bounded ITM and N , an ITI that halts after its first activation. Now, let's construct $\tilde{\mu}$, which is identical to μ , except that if, for any identifier id , $(\tilde{\mu}, \text{id})$ receives an output from N , $(\tilde{\mu}, \text{id})$ enters an infinite loop and never halts.

We claim that $\tilde{\mu}$ is locally T -bounded because for any execution prefix of any protocol, no ITI with code $\tilde{\mu}$ never receives an output from N . Indeed, N never sends such an output, so this could only occur if the environment impersonates N . However, the rules of execution rules of a protocol prohibit an environment from sending outputs to protocols.

Nevertheless, we can easily create a structured protocol with body code $\tilde{\mu}$, that never halts upon receiving any output. It is sufficient for the shell to replace the sender of each output with N .

This counterexample is possible because we cannot infer, from an execution prefix involving a structured protocol with body M and shell S , a corresponding execution prefix that involves both M and S .

This issue leads to several weaknesses in the proof of the UC theorem of Plain UC, as the UC operator relies on structured protocols, and various ITMs constructed in its proof simulates multiples ITIs.

The Variable Value of a Token. Here, we present a minor remark about the T -bounded notion, with no impact on the correctness of Plain UC.

We will speak of tokens of an ITI M for its total budget, and we will say that an ITI receives a token if it receives a message with import 1.

When a token is given to an ITI M , the additional time it gives to M isn't the same depending on how many tokens M already has. Indeed, if we note i

the number of tokens that M already has, M was limited to a total number of computational steps $T(i)$, and will be now limited to a total number of computational steps $T(i + 1)$, which is a gain of $T(i + 1) - T(i)$ computational steps. If, for example, $T(n) = n^2$, the additional time allowed to M for a new token is thus equal to $T(i + 1) - T(i) = 2i + 1$.

We believe it would be more natural for each additional token to provide the same additional number of computational steps. However, modifying the definition to ensure this may not be straightforward.

The Rarity of Tokens. Here, we present a minor remark about the T-bounded notion, with no impact on the correctness of Plain UC.

We will use our variant of T-bounded definition (Definition 23). More precisely, we will consider that a T-bounded ITM cannot use all its budget to invoke another ITM, as this behavior is used to create a counterexample in Section F.5.

Let's $u \in \mathbb{N}$. We will call “ u -chain”, in the execution of a protocol, a situation where:

- An ITM I_0 is invoked by an input or output with import u .
- I_0 then invokes another ITI I_1 with an input or output.
- I_1 then invokes another ITI I_2 with an input or output.
- This continues until some ITI I_ℓ performs a different action (*e.g.* deactivates, sends a message to the adversary, communicates with an already invoked ITI, etc.).

We define the length of a u -chain as the number ℓ . We call “member of the chain” of the ITM $I_0, \dots, I_{\ell-1}$. We call chain a u -chain for some $u \in \mathbb{N}$.

Let $u \in \mathbb{N}$ and a protocol μ that is T -bounded (Definition 23) for some function $T : \mathbb{N} \rightarrow \mathbb{N}$. We claim that for any execution of μ , any u -chain has a size bounded by u . Indeed, as each member of the chain invokes another ITI, the import it gives is strictly less than the import it has been invoked with.

We believe it would be more natural for the size of these chains to be bounded by $T(u)$ instead of u , although changing the definition to ensure this might be challenging.

F.6 (W.8): A Weakness Related to the Notion of Balanced Environment

In Updated UC, we do not adopt an analogue of the balanced environment notion. In Plain UC, this condition ensures that environments used to test UC-emulation provide sufficient computational resources to the adversary, thereby preventing starvation. More precisely, the notion aims to rule out scenarios where “the computational resources of the protocol, hence the protocol’s computational and communication complexity, are arbitrarily large relative to the computational resources available to the adversary” [11, (Section 4.2)].

While this property is functionally useful, we argue that it introduces a weakness in the proof of the UC Theorem in Plain UC. For this reason, it is omitted in Updated UC. The nature of this weakness is explained in the next subsection.

To partially offset the absence of this condition, the definition of a PPT adversary in Updated UC guarantees that the adversary is granted at least $n_M \cdot P(k)$ computational steps, where P is a polynomial, n_M is the number of input or backdoor messages received, and k is the security parameter.

Description of the Weakness. This subsection outlines a technical flaw in the proof of the UC Theorem in Plain UC, intended for readers familiar with its formalism. We use the notion of parametrized systems and time complexity from [11, (Section 3.2)]. All notations below refer to the Plain UC framework.

In Plain UC, UC-emulation is defined only with respect to a restricted class of environments known as *balanced environments* [11, (Section 4.2)]. A key consequence of this definition is that the first input message sent by the environment with nonzero import must be addressed to the adversary.

Let us consider an execution of ρ with adversary $\mathcal{S}$, a balanced environment $\mathcal{E}$, and input (k, z) , where $k > 0$. We will demonstrate with an example that the proof does not guarantee that, for a given session $(\pi, \mathfrak{s})$, the corresponding copy of $\mathcal{S}_\pi$ receives any message with nonzero import before one of the main parties of the session $(\phi, \mathfrak{s})$ is activated.

As a consequence, the behavior of this copy of $\mathcal{S}_\pi$ cannot be meaningfully related to its behavior in executions used to define UC-emulation of ϕ by π . Indeed, in an execution of ϕ with adversary $\mathcal{S}_\pi$ and a balanced environment, $(\mathcal{S}_\pi, (0, \diamond))$ never receives its first message after $(\phi, (0, 0))$ has already received an input with of import at least k .

Therefore, even if $\mathcal{S}_\pi$ appropriately emulates $\mathcal{D}$ in such settings, this does not establish the correctness of the simulation within the UC Theorem proof. This gap constitutes a structural weakness.

We demonstrate this lack of guarantee with the example illustrated in Figure 47. The execution proceeds as follows:

- $(\mathcal{E}, 0)$ sends an initial input $((i =)2k, (m_1, M, 0))$ to $(\mathcal{S}, 0)$, where M is an ITI distinct from ρ , ϕ , π , and $\mathcal{E}$, and m_1 is some message. The notation “ $(i =)$ ” indicates the import value.
- By definition, $\mathcal{S}$ initially behaves like the dummy adversary $\mathcal{D}$ and attempts to send a backdoor message $((i =)0, m_1)$ to M . Since M does not exist in the execution, $(\mathcal{E}, 0)$ is activated with no message.
- The environment $(\mathcal{E}, 0)$ then sends an input $((i =)k, m_2)$ to $(\rho, (0, 0))$.
- The ITI $(\rho, (0, 0))$ processes this input and sends an input $((i =)k, m_3)$ to $(\phi, (0, 0))$.

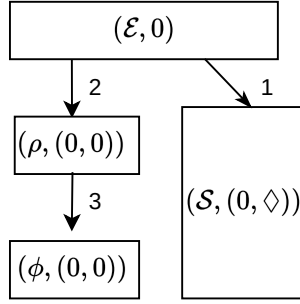


Fig. 47. Illustration of Weakness (W.8). Numbers indicate message creation order.

In this example, $\mathcal{E}$ is balanced the initial import to the adversary is $2k$, and the only subsequent import is $k < 2k$. However, $(\phi, (0, 0))$ receives a message with import k before the copy of $\mathcal{S}_\pi$ associated with sid 0 is created.

Table of Contents

Diving Deep Into UC: Uncovering and Resolving Issues in Universal Composability	1
<i>Céline Chevalier and Éric Sageloli</i>	
1 Introduction.....	1
1.1 Our Contributions	3
1.2 Related Work	5
2 Technical Overview	6
2.1 A Flaw in the Extended Session Definition Making the UC Theorem Unusable for Practical Protocols	7
2.2 Protocols with Dynamic Sub-Parties: A Family of Protocols Unfit for Plain UC	8
2.3 Panoptic Session: A Static Variant of Extended Session	9
2.4 Problems with the Subroutine Exposing Property	10
2.5 Extending Security Definitions Beyond Protocol Executions.....	12
2.6 Identifying Concerns in the UC Operator and Reimagining It for Enhanced Modularity	13
2.7 Revealing Hidden Scope Constraints in the UC Theorem	14
2.8 Proof of the UC Theorem Accounting for the Updated Definitions	15
2.9 Additional Contributions	15
3 Preliminary Notations	16
4 Overview of Plain UC	16
5 Problems Within The Universal Composition Theorem	24
5.1 Problems Related to Extended Sessions and Subroutine Respecting Property	24
5.2 Problems Related to the Universal Composition Operator	28
5.3 Problems Related to the Subroutine Exposing Property	30
5.4 Problem Related to the Compliance Property	32
6 Updated UC: An Update of Plain UC	33
6.1 A New Notion of Extended Session	34
6.2 Replacement Execution: A (Not Entirely) New Notion of Execution	35
6.3 Panoptic Session	36
6.4 Sid-Exposing	37
6.5 Sub-party Respecting Protocol	38
6.6 Compliance	39
6.7 Universal Composition Operator and Extension of UC-emulation	40
7 Universal Composition Theorem	42
A Quick Guide to Appendices	45
B User-guide to updated UC	45
B.1 Proving UC-emulation	46

B.2	Ensuring Preservation of UC-Emulation Under Composition by Reasonable Protocols	46
B.3	Ensuring Preservation of UC-Emulation Under Composition by a Specific Protocol ρ	47
C	Complete Proof of UC Theorem of Updated UC (Theorem 1)	47
C.1	Additional Notations and Conventions	48
C.2	Detailed Definitions of Time Complexity	49
C.3	Relevant Prefixes	50
C.4	Proof of the Theorem Relying on the Use of a Central Lemma ..	56
C.5	Proof of the Central Lemma (Lemma 1) Using two Auxiliary Lemmas	64
C.6	Definition of Environment $\mathcal{E}_\phi$ and Polynomial Q	65
C.7	Proof of the First Auxiliary Lemma	69
C.8	Proof of the Second Auxiliary Lemma	80
D	Details About Issues, Limitations and Weaknesses Described in the Main Body	81
D.1	Additional Notations and Conventions	81
D.2	Details Regarding Issue (I.2)	83
D.3	Details Regarding Issue (I.3)	88
D.4	Details Regarding Issue (I.4)	93
E	Definition of Our Universal Composition Operator	95
E.1	Main Ideas of the Construction and Definition of Our UC Operator	95
E.2	Definition of the Structured Protocol $\mu^{\rightleftharpoons \text{Coord}:\rho}$	96
E.3	Definition of the Coordinator $\text{Coord} = \text{repl}[\phi \rightarrow \pi : \rho]$	98
E.4	Proof that this Operator is a UC Operator	103
F	Additional Issues, Limitations, Weaknesses and Discussions	104
F.1	Two additional Issues Regarding the UC Operator	105
F.2	One additional Limitation Regarding the Subroutine Respecting property	107
F.3	(W.5) A Missing Case in the Simulator of the Proof of UC Theorem of Plain UC	107
F.4	Problems Related to Predicates	110
F.5	Problems Related to the Notion of T -Bounded Protocols	118
F.6	(W.8): A Weakness Related to the Notion of Balanced Environment	123